Web アプリ開発に役立つ API 利用ガイド

Python と uv で試す 94 の API

河合 勝彦

名古屋市立大学大学院経済学研究科

kkawai@econ.nagoya-cu.ac.jp

2026 年 10 月 9 日

概要

本書は、Web アプリケーションの開発でよく利用される API を「無償」と「有償（無料枠ありを含む）」に分けて紹介し、それぞれについて Python による短いサンプルスクリプトと、その実行結果を示した利用ガイドである。スクリプトはすべて PEP 723 のインラインメタデータ形式で書かれており、パッケージ管理ツール uv を使えば `uv run` の一行で依存関係の導入から実行までが完了する。API キーなしで実行できるものは実際に実行し、その出力を掲載して解説した。API キーや契約が必要なものについては、必要な環境変数と典型的な使い方を示した。

目次

第 1 章　はじめに ・・・

6.1 Unsplash API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 36
6.2 Pexels API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 37
6.3 Pixabay API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 37
6.4 DiceBear ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 38
6.5 Iconify API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 39
第 7 章 知識・コンテンツ ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 41
7.1 Wikipedia / MediaWiki API ・・・・・・・・・・・・・・・・・・・・・・・・・・・ 41
7.2 Open Library API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 42
7.3 Google Books API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 43
7.4 Hacker News API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 44
7.5 PokéAPI ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 45
7.6 GitHub REST API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 45
第 8 章 金融・為替 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 47
8.1 Frankfurter ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 47
8.2 CoinGecko API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 48
第 9 章 ユーティリティ ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 49
9.1 goqr.me QR Code API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 49
9.2 Microlink ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 50
9.3 Google Fonts ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 50
9.4 World Time API / Time API ・・・・・・・・・・・・・・・・・・・・・・・・・・・ 52
第 II 部 有償 API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 53
第 10 章 生成 AI・機械学習 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 55
10.1 Claude API (Anthropic) ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 55
10.2 OpenAI API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 56
10.3 Google Gemini API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 57
10.4 Replicate ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 58
10.5 Hugging Face Inference Providers ・・・・・・・・・・・・・・・・・・・・・・・ 59
第 11 章 決済 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 61
11.1 Stripe ・・ 61
11.2 PAY.JP ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 62
11.3 Square ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 63
11.4 PayPal ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 64
11.5 KOMOJU ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 65
第 12 章 認証・ユーザー管理 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 67
12.1 Firebase Authentication ・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 67
12.2 Auth0 ・・ 68
12.3 Clerk ・・ 69
12.4 Supabase Auth ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 69
第 13 章 データベース・BaaS ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 71
13.1 Cloud Firestore (Firebase) ・・・・・・・・・・・・・・・・・・・・・・・・・・・ 71
13.2 Supabase ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 72
13.3 Neon ・・ 72

13.4 Upstash ・・ 73
13.5 MongoDB Atlas ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 74
第 14 章 メール配信 ・・・ 76
14.1 Resend ・・・ 76
14.2 Amazon SES ・・ 77
14.3 SendGrid ・・ 77
14.4 Postmark ・・ 78
14.5 Mailgun ・・・ 79
第 15 章 SMS・通話・メッセージング ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 81
15.1 Twilio ・・・ 81
15.2 Vonage ・・・ 82
15.3 LINE Messaging API ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 82
15.4 Slack API ・・ 83
第 16 章 地図・位置情報（有償） ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 85
16.1 Google Maps Platform ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 85
16.2 Mapbox ・・ 86
16.3 HERE ・・ 87
第 17 章 ストレージ・画像処理 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 88
17.1 Amazon S3 ・・ 88
17.2 Cloudflare R2 ・・ 89
17.3 Cloudinary ・・ 90
17.4 imgix ・・ 90
17.5 UploadThing ・・・ 92
第 18 章 検索 ・・ 93
18.1 Algolia ・・・ 93
18.2 Meilisearch ・・ 94
18.3 Typesense ・・・ 95
第 19 章 翻訳・音声 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 96
19.1 DeepL API ・・・ 96
19.2 Google Cloud Translation ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 97
19.3 ElevenLabs ・・ 97
19.4 Google Cloud Speech-to-Text ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 98
第 20 章 監視・分析 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 100
20.1 Sentry ・・・ 100
20.2 PostHog ・・ 101
20.3 Mixpanel ・・・ 101
20.4 Datadog ・・ 102
第 21 章 リアルタイム通信 ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 104
21.1 Pusher Channels ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 104
21.2 Ably ・・ 105
21.3 LiveKit ・・ 105
第 22 章 その他データ ・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・・ 107

22.1 OpenWeather ・・・

第 1 章　はじめに

1.1　本書の構成

本書では、Web アプリケーション開発に役立つ 94 の API を次の 2 部に分けて紹介する。

- **第 I 部 無償 API**（40 種）：API キーなし、または無料登録だけで使える API。開発・テスト用、地図、天気、日本向けオープンデータ、画像、知識、金融、ユーティリティの 8 分野。
- **第 II 部 有償 API**（54 種）：従量課金やサブスクリプションで提供される API。多くは無料枠やテストモードを備える。生成 AI、決済、認証、データベース、メール、メッセージング、地図、ストレージ、検索、翻訳・音声、監視・分析、リアルタイム通信、その他データの 13 分野。

各 API の節は、概要と基本情報（公式サイト・認証方式・料金）、サンプルスクリプト、実行結果、解説の順に構成した。実行結果は 2026 年 10 月 9 日に筆者の環境で実際に取得したものである。天気・為替・ニュースなど時々刻々と変わるデータを扱う API では、読者が実行した時点の結果とは当然異なる。

実行結果の枠には 3 種類ある。黄色の枠は正常に実行できたときの出力、赤の枠はエラー終了したときの出力、灰色の枠は API キーなどの認証情報が未設定のため実行しなかったことを示す。有償 API のうち、筆者が API キーを保有していたもの（Claude、OpenAI、Gemini、Hugging Face、X）と、キーなしで動作を確認できるもの（imgix の URL 生成、LiveKit のトークン発行、Alpha Vantage のデモキー）は実際に実行した。

注意

料金体系・無料枠・利用規約は頻繁に変わる。本書の執筆中にも、REST Countries が API キー必須の新バージョンへ移行し、World Time API に接続できない状態が続いていた（いずれも本文で扱う）。API を本番で採用する前には、必ず公式サイトで最新の条件を確認してほしい。

1.2　実行環境の準備

1.2.1　uv のインストール

uv は Astral 社が開発する高速な Python パッケージ・プロジェクト管理ツールである。Python 本体のインストール、仮想環境の作成、パッケージの導入をひとつのコマンドで扱える。macOS / Linux では次のコマンドでインストールできる。

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
$ uv --version
uv 0.12.9
```

Windows では PowerShell で次のコマンドを実行する。

```
PS> powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
```

本書の実行結果は uv 0.12.9、Python 3.14.4 で取得した。

1.2.2 PEP 723：スクリプトに依存関係を書く

PEP 723（Inline script metadata）は、単一ファイルの Python スクリプトの先頭に、必要な Python のバージョンと依存パッケージを TOML 形式のコメントとして書く仕様である。本書のスクリプトはすべて次の形のヘッダーを持つ。

```
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""ここにスクリプトの説明を書く。"""
import httpx
```

`# /// script` から `# ///` までがメタデータで、`dependencies` に PyPI のパッケージ名を並べる。uv はこのヘッダーを読み取り、スクリプト専用の一時的な仮想環境を自動で作ってパッケージを導入してから実行する。`requirements.txt` や `pyproject.toml` を用意しなくてよく、スクリプト単体を配布しても同じように動くのが利点である。

1.2.3 スクリプトの実行

リポジトリを取得し、任意のスクリプトを `uv run` で実行する。

```
$ git clone https://github.com/katzkawai/kklab-api-list.git
$ cd kklab-api-list
$ uv run scripts/free/12_open_meteo.py
```

初回の実行時のみパッケージのダウンロードが発生し、2 回目以降はキャッシュが使われる。新しいスクリプトを書くときは、`uv init --script` でヘッダーの雛形を作り、`uv add --script` で依存関係を追加できる。

```
$ uv init --script hello.py --python 3.12
$ uv add --script hello.py httpx
$ uv run hello.py
```

全スクリプトをまとめて実行し、結果を `outputs/` に保存するツール `tools/run_all.py` も用意した。本書の実行結果はこのツールで取得している。

```
$ uv run tools/run_all.py              # すべて実行
$ uv run tools/run_all.py free/ paid/41  # 先頭一致で絞り込み
```

1.2.4　API キーの設定

API キーなどの認証情報は、スクリプトに直接書かず、環境変数から読み込む。キーを書いたファイルを誤って Git にコミットすると、公開リポジトリでは数分のうちに悪用されることがある。

```
$ export ANTHROPIC_API_KEY="sk-ant-..."
$ uv run scripts/paid/41_anthropic_claude.py
```

キーが多い場合は `.env` ファイルにまとめ、`.gitignore` に登録したうえで、`uv run --env-file .env scripts/...` のように読み込むとよい。本書のスクリプトは、必要な環境変数が未設定のときは `[SKIP]` と表示して何もせずに終了する。

1.3　API 呼び出しの共通パターン

本書のスクリプトの多くは、HTTP クライアントライブラリ httpx を使って REST API を直接呼び出している。個々の API に入る前に、繰り返し登場するパターンを整理しておく。

クライアントの使い回し
: 同じ API に複数回アクセスする場合は `httpx.Client(base_url=...)` を `with` 文で作り、使い回す。接続が再利用されるため速く、ヘッダーやタイムアウトの設定も一か所にまとめられる。

タイムアウト
: 外部 API は応答しないことがある。`timeout=` を必ず指定し、アプリ全体が止まらないようにする。

エラー処理
: `r.raise_for_status()` は、ステータスコードが 4xx / 5xx のときに例外を送出する。特に 429（Too Many Requests）はレート制限に達したことを示すので、時間をおいて再試行する。

クエリパラメータ
: URL に文字列を連結せず、`params={...}` に辞書で渡す。日本語や記号は自動でエンコードされる。

User-Agent
: Nominatim や Wikipedia のように、連絡先を含む User-Agent の指定を利用条件とする API がある。

非同期処理
: 多数のリクエストを並行して送る場合は `httpx.AsyncClient` と `asyncio.gather` を使う（Hacker News の例を参照）。

公式 SDK

生成 AI・決済・データベースなど、公式の Python SDK が提供されている API では SDK を使う。認証、再試行、ページネーションなどを SDK が肩代わりしてくれる。

Web アプリから API を呼ぶときの原則

秘密の API キーが必要な API は、ブラウザ（フロントエンド）から直接呼ばず、自前のサーバー（API ルート、Cloudflare Workers、Cloud Functions など）を経由して呼ぶ。フロントエンドの JavaScript に埋め込んだキーは、誰でも開発者ツールで読み取れるからである。本書のスクリプトは、そのままサーバー側の処理として使える形で書いている。

第I部　無償 API

第 2 章　開発・テスト用

フロントエンドの試作や学習の段階では、本物のバックエンドの代わりにダミーデータを返す API が役に立つ。いずれも API キーが不要で、ブラウザの `fetch` からも直接呼べる。

2.1　JSONPlaceholder

公式　`https://jsonplaceholder.typicode.com/`

認証　不要

料金　無料

投稿（posts）・コメント・ユーザー・TODO などのリソースを持つ、REST API の練習用サーバーである。GET だけでなく POST / PUT / DELETE も受け付けるが、データは実際には保存されず、成功したときのレスポンスだけが返る。

scripts/free/01_jsonplaceholder.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""JSONPlaceholder: ダミーの REST API で GET / POST を試す。"""
import httpx

BASE = "https://jsonplaceholder.typicode.com"

with httpx.Client(base_url=BASE, timeout=10) as client:
    # 1件取得 (GET)
    todo = client.get("/todos/1").json()
    print("GET /todos/1 ->", todo)

    # クエリパラメータで絞り込み (GET)
    posts = client.get("/posts", params={"userId": 1}).json()
    print(f"userId=1 の投稿数: {len(posts)}")
    for p in posts[:3]:
        print(f"  #{p['id']}: {p['title'][:40]}")

    # 新規作成 (POST) — 実際には保存されず、作成結果だけが返る
    r = client.post("/posts", json={"title": "こんにちは", "body": "テスト投稿", "userId": 1})
    print("POST /posts ->", r.status_code, r.json())
```

実行結果　`uv run scripts/free/01_jsonplaceholder.py`

```
GET /todos/1 -> {'userId': 1, 'id': 1, 'title': 'delectus aut autem', 'completed': False}
userId=1 の投稿数: 10
  #1: sunt aut facere repellat provident occae
  #2: qui est esse
  #3: ea molestias quasi exercitationem repell
POST /posts -> 201 {'title': ' こんにちは', 'body': ' テスト投稿', 'userId': 1, 'id': 101}
```

`/posts?userId=1` のように、クエリパラメータでの絞り込みにも対応している。POST の応答はステータス 201（Created）で、新しい ID として 101 が割り当てられた。ただし、もう一度 `/posts/101` を GET しても 404 になる。「作成に成功したら一覧を再取得する」といった UI の流れを試すには、このことを念頭に置く必要がある。

2.2 DummyJSON

公式 `https://dummyjson.com/`
認証 不要（ログイン API でトークン発行）
料金 無料

商品・カート・ユーザー・レシピなど、EC サイト風のリッチなダミーデータを返す。検索、ページネーション（`limit` / `skip`）、返す項目の選択（`select`）、JWT によるログインまで模擬できるのが特徴である。

scripts/free/02_dummyjson.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""DummyJSON: 商品検索・ページネーション・ログインを模擬する。"""
import httpx

BASE = "https://dummyjson.com"

with httpx.Client(base_url=BASE, timeout=10) as client:
    # 検索 + ページネーション (limit / skip)
    r = client.get("/products/search", params={"q": "phone", "limit": 3, "skip": 0,
                                               "select": "title,price,rating"})
    data = r.json()
    print(f"検索ヒット総数: {data['total']}")
    for p in data["products"]:
        print(f"  {p['title']:<30} ${p['price']:>8}  ★{p['rating']}")

    # ログイン (公開されているテスト用アカウント)
    r = client.post("/auth/login", json={"username": "emilys", "password": "emilyspass",
                                         "expiresInMins": 30})
    user = r.json()
    token = user["accessToken"]
    print(f"ログイン成功: {user['firstName']} {user['lastName']} / token={token[:20]}...")

    # 取得したトークンで認証付きエンドポイントを呼ぶ
    me = client.get("/auth/me", headers={"Authorization": f"Bearer {token}"}).json()
    print(f"/auth/me -> username={me['username']}, email={me['email']}")
```

実行結果 `uv run scripts/free/02_dummyjson.py`

```
検索ヒット総数: 23
  Apple AirPods Max Silver       $  549.99  ★3.47
  Apple iPhone Charger           $   19.99  ★4.15
  Apple MagSafe Battery Pack     $   99.99  ★3.62
ログイン成功: Emily Johnson / token=eyJhbGciOiJIUzI1NiIs...
/auth/me -> username=emilys, email=emily.johnson@x.dummyjson.com
```

`total` にヒット総数が返るので、ページ送り UI の実装練習にちょうどよい。`/auth/login` で得たアクセストークンを `Authorization: Bearer` ヘッダーに付けると、認証が必要なエンドポイント `/auth/me` からログイン中のユーザー情報を取得できた。トークン認証を使うアプリの流れを、バックエンドを作らずに一通り試せる。

2.3 httpbin

公式 `https://httpbin.org/`

認証 不要

料金 無料（OSS。Docker でローカル起動も可能）

受け取った HTTP リクエストの内容（クエリ、ヘッダー、ボディ）をそのまま JSON で返すサービスである。自分のコードが「実際にどんなリクエストを送っているか」を確かめるデバッグに使う。

scripts/free/03_httpbin.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""httpbin: 送信したリクエストの内容をそのまま確認する。"""
import httpx

BASE = "https://httpbin.org"
UA = "kklab-api-guide/1.0"

with httpx.Client(base_url=BASE, timeout=20, headers={"User-Agent": UA}) as client:
    # クエリとヘッダーがどう届いたかを確認
    r = client.get("/get", params={"q": "名古屋", "page": 2}, headers={"X-Demo": "hello"})
    d = r.json()
    print("args   :", d["args"])
    print("X-Demo :", d["headers"].get("X-Demo"))
    print("UA     :", d["headers"].get("User-Agent"))

    # JSON ボディの送信
    r = client.post("/post", json={"name": "河合", "items": [1, 2, 3]})
    print("json   :", r.json()["json"])

    # 任意のステータスコードを返させる (エラー処理のテストに便利)
    for code in (200, 404, 503):
        print(f"/status/{code} ->", client.get(f"/status/{code}").status_code)
```

実行結果 `uv run scripts/free/03_httpbin.py`

```
args   : {'page': '2', 'q': '名古屋'}
X-Demo : hello
UA     : kklab-api-guide/1.0
json   : {'items': [1, 2, 3], 'name': '河合'}
/status/200 -> 200
/status/404 -> 404
/status/503 -> 503
```

日本語のクエリ `q=名古屋` が正しくエンコードされて届いていること、独自ヘッダー `X-Demo` と User-Agent が送られていることが確認できる。`/status/{code}` は指定

したステータスコードを返すので、404 や 503 のときのエラー処理をテストできる。公開サーバーは混雑すると遅くなるため、頻繁に使う場合は `docker run -p 80:80 kennethreitz/httpbin` でローカルに立てるとよい。

2.4 Random User Generator

公式　https://randomuser.me/

認証　不要

料金　無料

名前・メールアドレス・住所・顔写真などを含む、架空のユーザープロフィールを生成する。ユーザー一覧画面やプロフィールカードのデザイン確認に使える。

scripts/free/04_randomuser.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Random User Generator: テスト用のユーザープロフィールを生成する。"""
import httpx

params = {
    "results": 3,
    "nat": "us,gb",                    # 国籍
    "seed": "kklab",                   # seed を固定すると毎回同じ結果になる
    "inc": "name,email,location,picture", # 必要な項目だけ取得
}
r = httpx.get("https://randomuser.me/api/", params=params, timeout=15)
r.raise_for_status()
for u in r.json()["results"]:
    name = f"{u['name']['first']} {u['name']['last']}"
    loc = f"{u['location']['city']}, {u['location']['country']}"
    print(f"{name:<22} {u['email']:<36} {loc}")
    print(f"  写真: {u['picture']['thumbnail']}")
```

実行結果　`uv run scripts/free/04_randomuser.py`

```
Myrtle Herrera         myrtle.herrera@example.com           Lousville, United States
  写真: https://randomuser.me/api/portraits/thumb/women/42.jpg
Hunter Howard          hunter.howard@example.com            Akron, United States
  写真: https://randomuser.me/api/portraits/thumb/men/45.jpg
Charlene Davis         charlene.davis@example.com           West Covina, United States
  写真: https://randomuser.me/api/portraits/thumb/women/73.jpg
```

`seed` を固定すると毎回同じユーザーが返るため、画面のスクリーンショットを比べるテストにも使える。`inc` で必要な項目だけを取得すれば、レスポンスが小さくなる。国籍（`nat`）に日本は含まれていないので、日本人名のダミーデータが必要な場合は別途用意する必要がある。

2.5 Lorem Picsum

公式 https://picsum.photos/

認証 不要

料金 無料

URL にサイズを指定するだけで写真を返す、プレースホルダー画像サービスである。HTML に `<img src="https://picsum.photos/600/400">` と書くだけで使える。

scripts/free/05_picsum.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Lorem Picsum: サイズ指定でプレースホルダー画像を取得する。"""
from pathlib import Path

import httpx

OUT = Path(__file__).resolve().parents[2] / "outputs" / "img"
OUT.mkdir(parents=True, exist_ok=True)

# /seed/{任意の文字列}/{幅}/{高さ} — seed を固定すると同じ画像になる
url = "https://picsum.photos/seed/kklab/600/400"
r = httpx.get(url, follow_redirects=True, timeout=20)
r.raise_for_status()
path = OUT / "picsum.jpg"
path.write_bytes(r.content)
print(f"保存: {path.name} ({len(r.content):,} bytes, {r.headers['content-type']})")
print("リダイレクト先:", r.url)

# 画像一覧 API (撮影者などのメタデータ)
for item in httpx.get("https://picsum.photos/v2/list", params={"limit": 3}, timeout=20).json():
    print(f"  id={item['id']:<4} {item['author']:<20} {item['width']}x{item['height']}")
print("その他の例: グレースケール  https://picsum.photos/600/400?grayscale")
print("            ぼかし          https://picsum.photos/600/400?blur=2")
```

実行結果 uv run scripts/free/05_picsum.py

```
保存: picsum.jpg (24,662 bytes, image/jpeg)
リダイレクト先: https://fastly.picsum.photos/id/345/600/400.jpg?hmac=mQIeR-cLPRvFvjl_e3Jbx」
↪  3vU2e2qYgWomR2qi0DyVII
  id=0    Alejandro Escamilla  5000x3333
  id=1    Alejandro Escamilla  5000x3333
  id=2    Alejandro Escamilla  5000x3333
その他の例: グレースケール  https://picsum.photos/600/400?grayscale
            ぼかし          https://picsum.photos/600/400?blur=2
```



`/seed/kklab/600/400` は CDN 上の実画像（`fastly.picsum.photos/id/345/...`）へリダイレクトされる。httpx はデフォルトではリダイレクトを追わないため、`follow_redirects=True` を指定している点に注意する。`/v2/list` からは撮影者名や元のサイズを取得できる。

第 3 章　地図・位置情報

3.1　OpenStreetMap タイル + Leaflet

公式　https://wiki.openstreetmap.org/wiki/Tiles

認証　不要（User-Agent の指定が必須）

料金　無料。大量アクセスはタイル利用ポリシーで禁止

OpenStreetMap（OSM）は、有志が作成している自由に利用できる世界地図である。地図画像は 256×256 ピクセルの「タイル」に分割され、ズームレベル z とタイル番号 (x, y) を含む URL で配信される（XYZ 方式）。Web ページで地図を表示するには、JavaScript ライブラリ Leaflet を使うのが定番である。

scripts/free/06_osm_tiles.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""OpenStreetMap タイル: 緯度経度からタイル番号を計算して地図画像を取得する。

Web ページで地図を表示するときは Leaflet を使うのが一般的 (末尾の HTML 例を参照)。
"""
import math
from pathlib import Path

import httpx

OUT = Path(__file__).resolve().parents[2] / "outputs" / "img"
OUT.mkdir(parents=True, exist_ok=True)
UA = "kklab-api-guide/1.0 (+https://github.com/katzkawai/kklab-api-list)"


def deg2tile(lat: float, lon: float, zoom: int) -> tuple[int, int]:
    """Web メルカトル (XYZ) のタイル番号を求める。"""
    n = 2 ** zoom
    x = int((lon + 180.0) / 360.0 * n)
    y = int((1.0 - math.asinh(math.tan(math.radians(lat))) / math.pi) / 2.0 * n)
    return x, y


lat, lon, z = 35.1407, 136.9354, 15  # 名古屋市立大学 滝子(山の畑)キャンパス付近
x, y = deg2tile(lat, lon, z)
url = f"https://tile.openstreetmap.org/{z}/{x}/{y}.png"
print(f"タイル番号 z={z} x={x} y={y}")
r = httpx.get(url, headers={"User-Agent": UA}, timeout=20)  # User-Agent の指定は必須
r.raise_for_status()
(OUT / "osm_tile.png").write_bytes(r.content)
print(f"保存: osm_tile.png ({len(r.content):,} bytes)")

LEAFLET_HTML = f"""<!-- Leaflet で地図を表示する最小例 -->
```

```
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css">
<div id="map" style="height:400px"></div>
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
<script>
  const map = L.map('map').setView([{lat}, {lon}], {z});
  L.tileLayer('https://tile.openstreetmap.org/{{z}}/{{x}}/{{y}}.png', {{
    attribution: '&copy; OpenStreetMap contributors'
  }}).addTo(map);
  L.marker([{lat}, {lon}]).addTo(map).bindPopup('名古屋市立大学');
</script>"""
print(LEAFLET_HTML)
```

実行結果　`uv run scripts/free/06_osm_tiles.py`

```
タイル番号 z=15 x=28848 y=12963
保存: osm_tile.png (36,238 bytes)
<!-- Leaflet で地図を表示する最小例 -->
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css">
<div id="map" style="height:400px"></div>
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
<script>
  const map = L.map('map').setView([35.1407, 136.9354], 15);
  L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {
    attribution: '&copy; OpenStreetMap contributors'
  }).addTo(map);
  L.marker([35.1407, 136.9354]).addTo(map).bindPopup(' 名古屋市立大学');
</script>
```



`deg2tile` 関数は、緯度経度をタイル番号に変換する Web メルカトル図法の式である。ただし実際の Web アプリでタイル番号を自分で計算することはほとんどない。出力の後半に示した HTML のように、Leaflet にタイル URL のテンプレートを渡せば、表示範囲に応じたタイルを自動で読み込んでくれる。出典表記（`attribution`）は利用条件なので省略しないこと。アクセスの多いサービスでは、OSM 公式サーバーではなく商用のタイル配信サービスや、自前のタイルサーバーを使う。

3.2　Nominatim

公式　`https://nominatim.org/`

認証　不要（User-Agent の指定が必須）

料金　無料（公式サーバーは最大 1 リクエスト/秒）

OSM のデータを使ったジオコーディング API である。地名や住所から座標を求める

「正引き」と、座標から住所を求める「逆引き」ができる。

scripts/free/07_nominatim.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Nominatim: 住所・施設名と座標を相互に変換する（ジオコーディング）。"""
import time

import httpx

UA = "kklab-api-guide/1.0 (+https://github.com/katzkawai/kklab-api-list)"
BASE = "https://nominatim.openstreetmap.org"

with httpx.Client(base_url=BASE, headers={"User-Agent": UA}, timeout=20) as client:
    # 正引き: 名称 → 座標
    r = client.get("/search", params={"q": "名古屋城", "format": "jsonv2", "limit": 1,
                                      "accept-language": "ja"})
    hit = r.json()[0]
    lat, lon = float(hit["lat"]), float(hit["lon"])
    print(f"名古屋城 -> lat={lat:.5f}, lon={lon:.5f}")
    print(f"  {hit['display_name']}")

    time.sleep(1)  # 利用ポリシー: 最大 1 リクエスト/秒

    # 逆引き: 座標 → 住所
    r = client.get("/reverse", params={"lat": 35.1709, "lon": 136.8815, "format": "jsonv2",
                                       "accept-language": "ja", "zoom": 18})
    d = r.json()
    print("(35.1709, 136.8815) ->", d["display_name"])
```

実行結果　`uv run scripts/free/07_nominatim.py`

```
名古屋城 -> lat=35.18440, lon=136.90010
  名古屋城, 丸ノ内線支線第２号, 三の丸一丁目, 本丸, 中区, 名古屋市, 愛知県, 460-8511, 日本
(35.1709, 136.8815) -> 名古屋, 椿町線, 太閤一丁目, 椿町, 中村区, 名古屋市, 愛知県, 453-0015,
↪  日本
```

`accept-language=ja` を指定すると日本語の住所が返る。利用ポリシーにより 1 秒に 1 リクエストまでに制限されているため、連続で呼ぶ間に `time.sleep(1)` を入れている。入力のたびに候補を出すオートコンプリートには使えない（ポリシーで禁止されている）。その用途には有償のジオコーディング API を使うか、Nominatim を自前で立てる。

3.3　国土地理院 地理院タイル

公式　`https://maps.gsi.go.jp/development/ichiran.html`

認証　不要

料金　無料（「出典：地理院タイル」の表記が必要）

国土地理院が配信する日本の地図タイルである。標準地図・淡色地図・航空写真・陰影起伏図など、多くの種類がある。URL 形式は OSM と同じ XYZ 方式なので、Leaflet の `tileLayer` の URL を差し替えるだけで使える。

scripts/free/08_gsi_tiles.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""国土地理院 地理院タイル: 標準地図と航空写真のタイルを取得する。

URL 形式は OSM と同じ XYZ 方式なので、Leaflet の tileLayer にそのまま指定できる。
利用時は「出典: 地理院タイル」の表記が必要。
"""
import math
from pathlib import Path

import httpx

OUT = Path(__file__).resolve().parents[2] / "outputs" / "img"
OUT.mkdir(parents=True, exist_ok=True)


def deg2tile(lat: float, lon: float, zoom: int) -> tuple[int, int]:
    n = 2 ** zoom
    x = int((lon + 180.0) / 360.0 * n)
    y = int((1.0 - math.asinh(math.tan(math.radians(lat))) / math.pi) / 2.0 * n)
    return x, y


z = 16
x, y = deg2tile(35.1847, 136.8995, z)  # 名古屋城
layers = {
    "std": "https://cyberjapandata.gsi.go.jp/xyz/std/{z}/{x}/{y}.png",          # 標準地図
    "seamlessphoto": "https://cyberjapandata.gsi.go.jp/xyz/seamlessphoto/{z}/{x}/{y}.jpg",  # 航空写真
}
for name, tmpl in layers.items():
    url = tmpl.format(z=z, x=x, y=y)
    r = httpx.get(url, timeout=20)
    r.raise_for_status()
    ext = url.rsplit(".", 1)[1]
    (OUT / f"gsi_{name}.{ext}").write_bytes(r.content)
    print(f"{name:<14} {url}  ({len(r.content):,} bytes)")
```

実行結果　`uv run scripts/free/08_gsi_tiles.py`

```
std            https://cyberjapandata.gsi.go.jp/xyz/std/16/57689/25917.png  (26,841 bytes)
seamlessphoto  https://cyberjapandata.gsi.go.jp/xyz/seamlessphoto/16/57689/25917.jpg
↪  (14,259 bytes)
```





左が標準地図、右が航空写真で、いずれも名古屋城付近のズームレベル 16 のタイルである。画像形式はレイヤーによって異なり、標準地図は PNG、航空写真は JPEG で配信

される。

3.4　国土地理院 住所検索・標高 API

公式　https://maps.gsi.go.jp/development/api.html

認証　不要

料金　無料

国土地理院は、地名・住所の検索 API と、指定地点の標高を返す API も公開している。

scripts/free/09_gsi_address_elevation.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""国土地理院 API: 住所検索と標高の取得。"""
import httpx

with httpx.Client(timeout=20) as client:
    # 住所検索 (結果は GeoJSON 形式。座標は [経度, 緯度] の順)
    r = client.get("https://msearch.gsi.go.jp/address-search/AddressSearch",
                   params={"q": "名古屋市瑞穂区瑞穂町"})
    for f in r.json()[:3]:
        lon, lat = f["geometry"]["coordinates"]
        print(f"{f['properties']['title']:<20} lat={lat:.5f} lon={lon:.5f}")

    # 標高 API
    points = {"名古屋駅": (35.1709, 136.8815), "富士山頂": (35.3606, 138.7274)}
    for name, (lat, lon) in points.items():
        r = client.get("https://cyberjapandata2.gsi.go.jp/general/dem/scripts/getelevation.php",
                       params={"lon": lon, "lat": lat, "outtype": "JSON"})
        d = r.json()
        print(f"{name}: 標高 {d['elevation']} m (データソース: {d['hsrc']})")
```

実行結果　uv run scripts/free/09_gsi_address_elevation.py

```
愛知県名古屋市瑞穂区瑞穂町          lat=35.13939 lon=136.93524
名古屋駅: 標高 2.2 m (データソース: 5m（レーザ）)
富士山頂: 標高 3770.6 m (データソース: 1m（レーザ）)
```

住所検索の結果は GeoJSON 形式で、座標が [経度, 緯度] の順に並ぶ点に注意する（緯度・経度の順ではない）。標高 API はレーザ測量などによる数値標高モデルを使っており、名古屋駅で 2.2 m、富士山頂で 3770.6 m という値が得られた。hsrc は、その値を求めるのに使ったデータの種類（解像度）を示す。

3.5　REST Countries

公式　https://restcountries.com/

認証　API キー（v5 から必須）

料金　無料プランは月 1,000 リクエスト

国名・首都・人口・通貨・言語・国旗などを返す API で、国選択のプルダウンなどによく使われてきた。長らくキー不要で使えたが、2026 年に旧バージョン（v2 / v3.1）が廃

止され、新しい v5 は API キーが必須となった。本節では、旧 URL にアクセスすると何が返るかを確認する。

scripts/free/10_restcountries.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""REST Countries: 国の基本情報 API。

2026年に旧バージョン (v2 / v3.1) が廃止され、新しい v5 は API キー必須になった
(無料プランは月 1,000 リクエスト)。ここでは旧 URL にアクセスしたときの応答を確認し、
API の「廃止・移行」がどのように通知されるかを見る。
"""
import httpx

r = httpx.get("https://restcountries.com/v3.1/alpha/jp", params={"fields": "name,capital"},
              follow_redirects=True, timeout=15)
print("HTTP ステータス:", r.status_code)
body = r.json()
print("success:", body.get("success"))
for e in body.get("errors", []):
    print("message:", e["message"])

# v5 はキーなしだと認証エラーになる
r = httpx.get("https://api.restcountries.com/v5/alpha/jp", timeout=15)
print("\nv5 (キーなし):", r.status_code, r.json())
```

実行結果　`uv run scripts/free/10_restcountries.py`

```
HTTP ステータス: 200
success: False
message: This API version has been deprecated. Please visit
↪  https://restcountries.com/docs/countries/legacy-api-deprecation to migrate to our new
↪  version (v5).

v5 (キーなし): 401 {'errors': [{'message': 'Authorization key required.', 'code':
↪  'authKeyMissing'}]}
```

旧 URL へのアクセスは新しいサーバーへリダイレクトされ、HTTP ステータスは 200 なのに `success: false` とエラーメッセージが返った。v5 にキーなしでアクセスすると 401（`authKeyMissing`）となる。「ステータスコードが 200 でも成功とは限らない」こと、そして無償 API は条件が変わりうることを示す好例である。国の一覧のように変化の少ないデータは、API を毎回呼ぶよりも、JSON ファイルとしてアプリに同梱するほうが堅牢な場合も多い。

3.6　ip-api

公式　`https://ip-api.com/`

認証　不要

料金　無料版は非商用のみ・HTTP のみ・45 リクエスト/分

IP アドレスから国・地域・都市・ISP を推定する。アクセス元の国に応じて言語や通

貨の初期値を切り替える、といった用途に使える。

scripts/free/11_ip_api.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""ip-api: IP アドレスからおおよその位置を推定する。

無料版は非商用のみ・HTTP のみ・45 リクエスト/分まで。
IP を省略すると「アクセス元の IP」が調べられる (ここでは公開 DNS の IP を指定)。
"""
import httpx

for ip in ["8.8.8.8", "1.1.1.1"]:
    r = httpx.get(f"http://ip-api.com/json/{ip}",
                  params={"lang": "ja", "fields": "status,country,regionName,city,isp,query"},
                  timeout=15)
    d = r.json()
    print(f"{d['query']:<10} {d['country']} {d['regionName']} {d['city']}  ISP: {d['isp']}")
```

実行結果　`uv run scripts/free/11_ip_api.py`

```
8.8.8.8    アメリカ合衆国 バージニア州 Ashburn  ISP: Google LLC
1.1.1.1    オーストラリア Queensland South Brisbane  ISP: Cloudflare, Inc
```

`lang=ja` で国名・地域名が日本語になり、`fields` で必要な項目だけを返させている。IP を省略するとアクセス元自身の IP が調べられる。IP からの位置推定は都市レベルでも外れることが多く、VPN やモバイル回線ではさらに不正確になる。あくまで「初期値の推測」にとどめ、ユーザーが変更できるようにしておくべきである。無料版は HTTPS に対応していないため、HTTPS のページからブラウザで直接呼ぶことはできない。

第 4 章　天気・自然

4.1　Open-Meteo

公式　https://open-meteo.com/

認証　不要

料金　非商用は無料（1 日 1 万リクエストまで）。商用は有料プラン

各国の気象機関の数値予報モデルをもとにした天気予報 API である。API キーなしで、現在の気象、時間別・日別の予報、過去の気象データを取得できる。

scripts/free/12_open_meteo.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Open-Meteo: API キー不要で天気予報を取得する。"""
import httpx

params = {
    "latitude": 35.1815, "longitude": 136.9066,    # 名古屋
    "current": "temperature_2m,relative_humidity_2m,wind_speed_10m",
    "daily": "weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max",
    "timezone": "Asia/Tokyo",
    "forecast_days": 5,
}
r = httpx.get("https://api.open-meteo.com/v1/forecast", params=params, timeout=15)
r.raise_for_status()
d = r.json()

cur = d["current"]
print(f"現在 ({cur['time']}): {cur['temperature_2m']}℃, 湿度 {cur['relative_humidity_2m']}%, "
      f"風速 {cur['wind_speed_10m']} km/h")

# WMO 天気コードの一部を日本語に変換
WMO = {0: "快晴", 1: "晴れ", 2: "一部曇り", 3: "曇り", 45: "霧", 51: "霧雨", 61: "小雨",
       63: "雨", 65: "大雨", 71: "小雪", 80: "にわか雨", 95: "雷雨"}
day = d["daily"]
for i, date in enumerate(day["time"]):
    w = WMO.get(day["weather_code"][i], f"コード{day['weather_code'][i]}")
    print(f"{date}  {w:<6} 最高 {day['temperature_2m_max'][i]:>5}℃ / 最低 "
          f"{day['temperature_2m_min'][i]:>5}℃  降水確率 {day['precipitation_probability_max'][i]}%")
```

実行結果　uv run scripts/free/12_open_meteo.py

```
現在 (2026-10-09T08:15): 18.9℃, 湿度 77%, 風速 2.1 km/h
2026-10-09  晴れ     最高  26.0℃ / 最低  16.1℃  降水確率 2%
2026-10-10  晴れ     最高  25.9℃ / 最低  17.6℃  降水確率 0%
2026-10-11  晴れ     最高  26.9℃ / 最低  17.8℃  降水確率 0%
2026-10-12  霧雨     最高  28.3℃ / 最低  17.7℃  降水確率 6%
```

```
2026-10-13  霧雨      最高  28.3℃ / 最低  18.5℃  降水確率 10%
```

`current` と `daily` に、欲しい変数名をカンマ区切りで指定する。天気は WMO（世界気象機関）の天気コードで返るため、表示用の文字列やアイコンに変換する対応表を用意する。`timezone=Asia/Tokyo` を指定しないと、日付の区切りが UTC 基準になる点に注意する。

4.2　気象庁 防災情報 JSON

公式　`https://www.jma.go.jp/bosai/`

認証　不要

料金　無料（公式 API としての仕様公開はない）

気象庁の Web サイトは、天気予報や警報を JSON ファイルから読み込んで表示している。この JSON を直接取得すれば、気象庁の発表そのままの予報文が得られる。

scripts/free/13_jma.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""気象庁 防災情報 JSON: 気象庁 Web サイトが内部で使う JSON から天気予報を取得する。

正式に仕様が公開された API ではないため、予告なく変わる可能性がある点に注意。
地域コード: 230000 = 愛知県
"""
import httpx

r = httpx.get("https://www.jma.go.jp/bosai/forecast/data/forecast/230000.json", timeout=15)
r.raise_for_status()
data = r.json()

latest = data[0]  # [0] が短期予報、[1] が週間予報
print(f"{latest['publishingOffice']} 発表: {latest['reportDatetime']}")
series = latest["timeSeries"][0]
for area in series["areas"]:
    print(f"■ {area['area']['name']}")
    for t, w in zip(series["timeDefines"], area["weathers"]):
        print(f"  {t[:10]}: {' '.join(w.split())}")

# 概況テキスト
ov = httpx.get("https://www.jma.go.jp/bosai/forecast/data/overview_forecast/230000.json",
               timeout=15).json()
print("概況:", ov["text"].split("\n")[0])
```

実行結果　`uv run scripts/free/13_jma.py`

```
名古屋地方気象台 発表: 2026-10-09T05:00:00+09:00
■ 西部
  2026-10-09: 晴れ
  2026-10-10: 晴れ 朝晩 くもり
■ 東部
  2026-10-09: 晴れ
  2026-10-10: 晴れ 朝晩 くもり
```

```
概況:　東海地方は、高気圧に覆われていますが、湿った空気の影響を受けている所があります。
```

地域コード 230000（愛知県）の予報を取得した。発表官署（名古屋地方気象台）、発表時刻、地域（西部・東部）ごとの天気の文章が得られる。気象庁の文面には全角スペースが含まれるため、`' '.join(w.split())` で空白を整えている。仕様書が公開された API ではないため、URL や構造は予告なく変わりうる。業務で使う場合は、気象業務支援センターなどの正式な配信サービスを検討する。

4.3　USGS Earthquake API

公式　`https://earthquake.usgs.gov/fdsnws/event/1/`
認証　不要
料金　無料

米国地質調査所（USGS）が提供する、世界の地震カタログの検索 API である。地震の規模・期間・範囲を指定して検索でき、結果は GeoJSON で返る。

scripts/free/14_usgs_earthquake.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""USGS Earthquake API: 世界の地震情報を GeoJSON で取得する。"""
from datetime import datetime, timezone

import httpx

params = {
    "format": "geojson",
    "minmagnitude": 5,
    "orderby": "time",
    "limit": 5,
}
r = httpx.get("https://earthquake.usgs.gov/fdsnws/event/1/query", params=params, timeout=20)
r.raise_for_status()
for f in r.json()["features"]:
    p = f["properties"]
    lon, lat, depth = f["geometry"]["coordinates"]
    t = datetime.fromtimestamp(p["time"] / 1000, tz=timezone.utc)
    print(f"{t:%Y-%m-%d %H:%M} UTC  M{p['mag']:.1f}  深さ{depth:>5.1f}km  {p['place']}")

# 日本周辺（緯度経度の範囲指定）の地震数
r = httpx.get("https://earthquake.usgs.gov/fdsnws/event/1/count",
              params={"format": "geojson", "minmagnitude": 4, "starttime": "2026-01-01",
                      "minlatitude": 24, "maxlatitude": 46, "minlongitude": 122,
                      "maxlongitude": 146}, timeout=20)
print("2026年以降 日本周辺の M4 以上:", r.json()["count"], "件")
```

実行結果　`uv run scripts/free/14_usgs_earthquake.py`

```
2026-10-08 10:07 UTC  M5.1  深さ 602.3km  236 km E of Levuka, Fiji
2026-10-08 09:00 UTC  M6.3  深さ  10.0km  102 km NE of Norsup, Vanuatu
2026-10-08 08:07 UTC  M5.5  深さ  52.1km  17 km SW of Burias, Philippines
```

```
2026-10-08 06:46 UTC  M5.5  深さ 10.0km  west of Macquarie Island
2026-10-08 04:48 UTC  M5.0  深さ 10.0km  92 km N of Ruteng, Indonesia
2026 年以降 日本周辺の M4 以上: 985 件
```

`time` は UNIX 時間（ミリ秒）なので、1000 で割ってから `datetime` に変換する。`/count` エンドポイントは件数だけを返し、緯度経度の範囲で絞り込める。2026 年以降、日本周辺で M4 以上の地震が 985 件記録されていることがわかる。GeoJSON はそのまま Leaflet の `L.geoJSON()` に渡して地図に描画できる。

4.4 NASA Open APIs

公式 `https://api.nasa.gov/`

認証 API キー（無料。`DEMO_KEY` で試用可）

料金 無料（登録キーは 1 時間 1,000 リクエスト）

NASA は、今日の天文写真（APOD）、地球に接近する小天体（NeoWs）、火星探査車の画像など、多くの API を公開している。

scripts/free/15_nasa.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""NASA Open APIs: 今日の天文写真 (APOD) と地球近傍天体 (NeoWs) を取得する。

API キーは https://api.nasa.gov で無料発行できる。未設定時は DEMO_KEY (回数制限が厳しい) を使う。
"""
import os
from datetime import date

import httpx

key = os.environ.get("NASA_API_KEY", "DEMO_KEY")

with httpx.Client(base_url="https://api.nasa.gov", timeout=30) as client:
    r = client.get("/planetary/apod", params={"api_key": key})
    if r.is_success:
        apod = r.json()
        print(f"APOD {apod['date']}: {apod['title']}")
        print(f"  種類: {apod['media_type']}  URL: {apod.get('url')}")
        print(f"  解説: {apod['explanation'][:120]}...")
    else:
        # APOD は一時的に 5xx を返すことがある。429 は DEMO_KEY の回数制限
        print(f"APOD 取得失敗: HTTP {r.status_code} {r.text[:80]}")

    today = date.today().isoformat()
    neo = client.get("/neo/rest/v1/feed",
                     params={"api_key": key, "start_date": today, "end_date": today}).json()
    print(f"\n{today} に地球へ接近する小天体: {neo['element_count']} 個")
    for obj in neo["near_earth_objects"][today][:3]:
        ca = obj["close_approach_data"][0]
        km = float(ca["miss_distance"]["kilometers"])
        print(f"  {obj['name']:<22} 最接近距離 {km:,.0f} km  "
              f"危険判定: {obj['is_potentially_hazardous_asteroid']}")
```

実行結果　uv run scripts/free/15_nasa.py

```
APOD 取得失敗: HTTP 500 {"code":500,"msg":"Internal Service Error","service_version":"v1"}


2026-10-09 に地球へ接近する小天体: 7 個
  (2001 UP)              最接近距離 62,620,486 km  危険判定: False
  (2003 QF70)            最接近距離 46,792,545 km  危険判定: False
  (2015 KT121)           最接近距離 46,791,579 km  危険判定: False
```

実行時、APOD は HTTP 500 を返した（数分おいても回復しなかった）。無償の公開 API ではこのような一時的な障害が珍しくないため、スクリプトでは `r.is_success` でステータスを確認し、失敗しても処理全体が止まらないようにしている。一方、NeoWs からは当日に地球へ接近する 7 個の小天体と、その最接近距離が得られた。`DEMO_KEY` は IP アドレス単位で 1 時間 30 回程度に制限されるので、継続して使う場合は無料のキーを発行する。

4.5　Sunrise-Sunset

公式　`https://sunrise-sunset.org/api`

認証　不要

料金　無料（クレジット表記が必要）

緯度経度と日付から、日の出・日の入り・南中・薄明の時刻を計算して返す。

scripts/free/16_sunrise_sunset.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Sunrise-Sunset: 指定地点・日付の日の出・日の入り時刻を取得する。"""
import httpx

params = {"lat": 35.1815, "lng": 136.9066, "date": "today",
          "formatted": 0, "tzid": "Asia/Tokyo"}
r = httpx.get("https://api.sunrise-sunset.org/json", params=params, timeout=15)
r.raise_for_status()
res = r.json()["results"]
for key, label in [("civil_twilight_begin", "薄明開始"), ("sunrise", "日の出"),
                   ("solar_noon", "南中"), ("sunset", "日の入り"),
                   ("civil_twilight_end", "薄明終了")]:
    print(f"{label:<6}: {res[key]}")
h, m = divmod(res["day_length"] // 60, 60)
print(f"昼の長さ: {h}時間{m}分")
```

実行結果　uv run scripts/free/16_sunrise_sunset.py

```
薄明開始  : 2026-10-09T05:27:53+09:00
日の出    : 2026-10-09T05:51:58+09:00
南中      : 2026-10-09T11:39:44+09:00
日の入り  : 2026-10-09T17:27:30+09:00
薄明終了  : 2026-10-09T17:51:34+09:00
昼の長さ: 11時間35分
```

`formatted=0` を指定すると ISO 8601 形式の時刻、`day_length` は秒数で返る。`tzid` を指定しないと UTC で返るので、日本向けには `Asia/Tokyo` を指定する。写真撮影アプリやダークモードの自動切り替えなどに使える。

第 5 章　日本向けデータ

5.1　zipcloud 郵便番号検索 API

公式　https://zipcloud.ibsnet.co.jp/doc/api

認証　不要

料金　無料

郵便番号から住所（都道府県・市区町村・町域）とそのカナを返す。住所入力フォームの「郵便番号から住所を自動入力」機能の実装によく使われる。

scripts/free/17_zipcloud.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""zipcloud 郵便番号検索 API: 郵便番号から住所を取得する。

住所入力フォームで「郵便番号を入れたら住所を自動入力」する機能に使える。
"""
import httpx

for zipcode in ["4670802", "1000001", "0000000"]:
    r = httpx.get("https://zipcloud.ibsnet.co.jp/api/search",
                  params={"zipcode": zipcode}, timeout=15)
    d = r.json()
    if d["results"] is None:
        print(f"〒{zipcode}: 該当なし (message={d['message']})")
        continue
    for a in d["results"]:
        print(f"〒{zipcode}: {a['address1']}{a['address2']}{a['address3']}  "
              f"({a['kana1']}{a['kana2']}{a['kana3']})")
```

実行結果　`uv run scripts/free/17_zipcloud.py`

```
〒4670802: 愛知県名古屋市瑞穂区瑞穂町山の畑  (ｱｲﾁｹﾝﾅｺﾞﾔｼﾐｽﾞﾎｸﾐｽﾞﾎﾁｮｳﾔﾏﾉﾊﾞﾀ)
〒4670802: 愛知県名古屋市瑞穂区瑞穂町  (ｱｲﾁｹﾝﾅｺﾞﾔｼﾐｽﾞﾎｸﾐｽﾞﾎﾁｮｳ)
〒1000001: 東京都千代田区千代田  (ﾄｳｷｮｳﾄﾁﾖﾀﾞｸﾁﾖﾀﾞ)
〒0000000: 該当なし (message=None)
```

ひとつの郵便番号に複数の町域が対応する場合は、結果が複数件返る（467-0802 の例）。該当がない場合は `results` が `null` になるので、そのケースを必ず処理する。カナは半角カタカナで返るため、フォームに入れる前に全角へ変換するとよい（`unicodedata.normalize("NFKC", s)` で変換できる）。JSONP にも対応しているため、ブラウザから直接呼ぶこともできる。

5.2 Holidays JP API

公式 https://holidays-jp.github.io/

認証 不要

料金 無料

内閣府が公表する「国民の祝日」を JSON / CSV で配信する。GitHub Pages 上の静的ファイルなので、高速かつ安定している。

scripts/free/18_holidays_jp.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Holidays JP API: 日本の祝日一覧を取得する。"""
from datetime import date

import httpx

year = 2026
r = httpx.get(f"https://holidays-jp.github.io/api/v1/{year}/date.json", timeout=15)
r.raise_for_status()
holidays = r.json()  # {"2026-01-01": "元日", ...}
print(f"{year}年の祝日: {len(holidays)} 日")
for d, name in holidays.items():
    wd = "月火水木金土日"[date.fromisoformat(d).weekday()]
    print(f"  {d} ({wd}) {name}")


def is_business_day(d: date) -> bool:
    """土日祝を除いた営業日かどうか。"""
    return d.weekday() < 5 and d.isoformat() not in holidays


print("2026-11-23 は営業日?", is_business_day(date(2026, 11, 23)))
```

実行結果 uv run scripts/free/18_holidays_jp.py

```
2026 年の祝日: 18 日
  2026-01-01 (木) 元日
  2026-01-12 (月) 成人の日
  2026-02-11 (水) 建国記念の日
  2026-02-23 (月) 天皇誕生日
  2026-03-20 (金) 春分の日
  2026-04-29 (水) 昭和の日
  2026-05-03 (日) 憲法記念日
  2026-05-04 (月) みどりの日
  2026-05-05 (火) こどもの日
  2026-05-06 (水) 憲法記念日 振替休日
  2026-07-20 (月) 海の日
  2026-08-11 (火) 山の日
  2026-09-21 (月) 敬老の日
  2026-09-22 (火) 国民の休日
  2026-09-23 (水) 秋分の日
  2026-10-12 (月) スポーツの日
  2026-11-03 (火) 文化の日
```

```
  2026-11-23 (月) 勤労感謝の日
2026-11-23 は営業日? False
```

2026 年は振替休日と国民の休日を含めて 18 日あることがわかる。例のように土日と組み合わせれば、「営業日判定」や「n 営業日後の日付」を計算できる。予約システムや配送日の表示などで頻繁に必要になる処理である。

5.3 HeartRails Express

公式 https://express.heartrails.com/api.html

認証 不要

料金 無料

全国の駅・路線の情報を返す API である。座標からの最寄り駅検索、路線ごとの駅一覧、都道府県ごとの路線一覧などを取得できる。

scripts/free/19_heartrails_express.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""HeartRails Express: 駅・路線情報を取得する。"""
import httpx

BASE = "https://express.heartrails.com/api/json"

with httpx.Client(timeout=15) as client:
    # 座標から最寄り駅を検索 (x=経度, y=緯度)
    r = client.get(BASE, params={"method": "getStations", "x": 136.9354, "y": 35.1407})
    print("名古屋市立大学 滝子キャンパス付近の最寄り駅:")
    for s in r.json()["response"]["station"][:4]:
        print(f"  {s['name']}駅 ({s['line']})  距離 {s['distance']}")

    # 路線の駅一覧
    r = client.get(BASE, params={"method": "getStations", "line": "名古屋市桜通線"})
    names = [s["name"] for s in r.json()["response"]["station"]]
    print(f"桜通線の駅 ({len(names)} 駅): {' - '.join(names[:8])} ...")
```

実行結果 uv run scripts/free/19_heartrails_express.py

```
名古屋市立大学 滝子キャンパス付近の最寄り駅:
  桜山駅 (名古屋市桜通線)  距離 220m
  瑞穂区役所駅 (名古屋市桜通線)  距離 1000m
  御器所駅 (名古屋市桜通線)  距離 1000m
  御器所駅 (名古屋市鶴舞線)  距離 1000m
桜通線の駅 (21 駅): 太閤通 - 名古屋 - 国際センター - 丸の内 - 久屋大通 - 高岳 - 車道 - 今池
↪  ...
```

座標は x（経度）と y（緯度）で指定する。最寄り駅は距離の近い順に返り、桜山駅が 220 m の位置にあることがわかる。路線名は「名古屋市桜通線」のように API 独自の表記で指定する必要があり、表記が違うとエラーになる（筆者も「名古屋市営桜通線」と書いて失敗した）。正しい路線名は、method=getLines&prefecture=愛知県 で一覧を取得して確認できる。

5.4 e-Stat API

公式 https://www.e-stat.go.jp/api/

認証 アプリケーション ID（無料登録）

料金 無料

政府統計の総合窓口（e-Stat）に収録された、国勢調査・人口推計・家計調査などの統計データを取得できる。経済・社会データを扱うアプリやダッシュボードには欠かせない。

scripts/free/20_estat.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""e-Stat API: 政府統計の総合窓口から統計表を検索する。

https://www.e-stat.go.jp/api/ で無料のユーザー登録をし、アプリケーション ID を取得する。
"""
import os
import sys

import httpx

app_id = os.environ.get("ESTAT_APP_ID")
if not app_id:
    print("[SKIP] 環境変数 ESTAT_APP_ID が未設定です。")
    sys.exit(0)

BASE = "https://api.e-stat.go.jp/rest/3.0/app/json"
r = httpx.get(f"{BASE}/getStatsList",
              params={"appId": app_id, "searchWord": "人口推計", "limit": 5}, timeout=30)
r.raise_for_status()
tables = r.json()["GET_STATS_LIST"]["DATALIST_INF"]["TABLE_INF"]
for t in tables:
    print(f"{t['@id']}  {t['STAT_NAME']['$']}: {t['TITLE'] if isinstance(t['TITLE'], str) else t['TITLE
        ↪ ']['$']}")

# 統計データ本体は getStatsData?statsDataId=<上の ID> で取得する
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 ESTAT_APP_ID が未設定です。
```

`getStatsList` で統計表を検索し、得られた統計表 ID を `getStatsData` に渡すとデータ本体を取得できる。データは「表章項目」「時間軸」「地域」などの分類コードと値の組で返るため、pandas の `DataFrame` に変換してから集計するのが扱いやすい。

5.5 国立国会図書館サーチ API

公式 https://ndlsearch.ndl.go.jp/help/api

認証 不要

料金 無料（非営利。営利利用は事前の申請が必要）

国立国会図書館サーチの書誌データベースを検索できる。OpenSearch・SRU・OAI-PMH などの方式に対応しており、ここでは最も簡単な OpenSearch を使う。

scripts/free/21_ndl_search.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""国立国会図書館サーチ API (OpenSearch): 書誌情報を検索する。

結果は RSS (XML) で返るので、標準ライブラリの xml.etree で解析する。
"""
import xml.etree.ElementTree as ET

import httpx

r = httpx.get("https://ndlsearch.ndl.go.jp/api/opensearch",
              params={"title": "計量経済学", "cnt": 5, "mediatype": "books"}, timeout=30)
r.raise_for_status()
root = ET.fromstring(r.content)
ns = {"dc": "http://purl.org/dc/elements/1.1/",
      "openSearch": "http://a9.com/-/spec/opensearchrss/1.0/"}
total = root.findtext("channel/openSearch:totalResults", namespaces=ns)
print(f"ヒット件数: {total}")
for item in root.findall("channel/item"):
    title = item.findtext("title")
    author = item.findtext("author") or "-"
    pub = item.findtext("dc:publisher", namespaces=ns) or "-"
    print(f"  『{title}』 {author} / {pub}")
```

実行結果　`uv run scripts/free/21_ndl_search.py`

```
ヒット件数: 574
  『Rで学ぶ空間計量経済学入門』 Arbia, Giuseppe,堤, 盛人, 1968-,ジュセッペ・アルビア 著,堤盛人
  ↪ 監訳 / 勁草書房
  『R による計量経済学 : Econometrics』 秋山, 裕, 1961-,秋山裕 著 / オーム社
  『新しい経済学 : 理論と政策にたいするケインズの影響』 Harris, Seymour Edwin, 1897-1974,日本
  ↪ 銀行,セイモア・E.ハリス 編,日本銀行調査局 訳 / 東洋経済新報社
  『新しい計量経済学 : データで因果関係に迫る』 鹿野, 繁樹, 1975-,鹿野繁樹 著 / 日本評論社
  『アムステルダム市における不動産評価のための計量経済学的モデルの活用方法』 Needham,
  ↪ Barrie,Francke, Marc,Bosma, Pieter,資産評価システム研究センター,Barrie Needham, Mark
  ↪ Francke, Peter Bosma [共著] / 資産評価システム研究センター
```

結果は JSON ではなく RSS（XML）で返るため、標準ライブラリの `xml.etree.ElementTree` で解析している。`dc:` などの名前空間付きの要素は、名前空間の辞書を渡して取得する。タイトルに「計量経済学」を含む図書が 574 件ヒットした。図書館資料の検索アプリや、ISBN からの書誌情報の自動入力に使える。

5.6　楽天ウェブサービス

公式　`https://webservice.rakuten.co.jp/`

認証　アプリ ID（無料登録）

料金　無料

楽天市場の商品検索、楽天ブックスの書籍検索、楽天トラベルの施設検索などの API

を提供している。アフィリエイト ID を付けると、成果報酬の対象リンクを生成できる。

scripts/free/22_rakuten.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""楽天ウェブサービス: 楽天市場の商品を検索する。

https://webservice.rakuten.co.jp/ でアプリ ID を無料発行する。
"""
import os
import sys

import httpx

app_id = os.environ.get("RAKUTEN_APP_ID")
if not app_id:
    print("[SKIP] 環境変数 RAKUTEN_APP_ID が未設定です。")
    sys.exit(0)

r = httpx.get("https://app.rakuten.co.jp/services/api/IchibaItem/Search/20220601",
              params={"applicationId": app_id, "keyword": "名古屋 ういろう", "hits": 5,
                      "sort": "-reviewAverage", "formatVersion": 2}, timeout=20)
r.raise_for_status()
for item in r.json()["Items"]:
    print(f"{item['itemPrice']:>7,}円 ★{item['reviewAverage']} {item['itemName'][:40]}")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 RAKUTEN_APP_ID が未設定です。
```

`formatVersion=2` を指定すると、レスポンスの入れ子が浅くなって扱いやすくなる。`sort=-reviewAverage` のように、先頭に `-` を付けると降順になる。

5.7 Yahoo!デベロッパーネットワーク

公式 `https://developer.yahoo.co.jp/`

認証 Client ID（無料登録）

料金 無料（API ごとに 1 日あたりの上限あり）

Yahoo! JAPAN は、ショッピング商品検索のほか、日本語形態素解析・ルビ振り・校正支援などのテキスト解析 API を公開している。ここではショッピング商品検索（v3）を使う。

scripts/free/23_yahoo_shopping.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Yahoo!デベロッパーネットワーク: Yahoo!ショッピングの商品検索 API (v3)。

https://developer.yahoo.co.jp/ でアプリケーションを登録し Client ID を取得する。
"""
```

```python
import os
import sys

import httpx

appid = os.environ.get("YAHOO_CLIENT_ID")
if not appid:
    print("[SKIP] 環境変数 YAHOO_CLIENT_ID が未設定です。")
    sys.exit(0)

r = httpx.get("https://shopping.yahooapis.jp/ShoppingWebService/V3/itemSearch",
              params={"appid": appid, "query": "味噌煮込みうどん", "results": 5,
                      "sort": "-score"}, timeout=20)
r.raise_for_status()
for hit in r.json()["hits"]:
    print(f"{hit['price']:>7,}円  {hit['name'][:40]}  ({hit['seller']['name']})")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 YAHOO_CLIENT_ID が未設定です。
```

第 6 章　画像・メディア

6.1　Unsplash API

公式　https://unsplash.com/developers

認証　Access Key（無料登録）

料金　無料（デモ段階は 1 時間 50 リクエスト。審査後に上限が上がる）

高品質なフリー写真を検索・取得できる。利用規約により、写真を表示するときは撮影者名と Unsplash へのリンクを併記し、ダウンロード時には専用のエンドポイントを呼んでダウンロード数を記録する必要がある。

scripts/free/24_unsplash.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Unsplash API: フリー写真を検索する。

https://unsplash.com/developers でアプリを登録し Access Key を取得する。
利用規約により、表示時は撮影者名と Unsplash へのリンクを併記する。
"""
import os
import sys

import httpx

key = os.environ.get("UNSPLASH_ACCESS_KEY")
if not key:
    print("[SKIP] 環境変数 UNSPLASH_ACCESS_KEY が未設定です。")
    sys.exit(0)

r = httpx.get("https://api.unsplash.com/search/photos",
              params={"query": "nagoya castle", "per_page": 5, "orientation": "landscape"},
              headers={"Authorization": f"Client-ID {key}", "Accept-Version": "v1"}, timeout=20)
r.raise_for_status()
for p in r.json()["results"]:
    print(f"{p['urls']['small']}\n  Photo by {p['user']['name']} on Unsplash")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 UNSPLASH_ACCESS_KEY が未設定です。
```

6.2 Pexels API

公式 https://www.pexels.com/api/

認証 API キー（無料登録）

料金 無料（1 時間 200 / 月 2 万リクエスト）

フリーの写真と動画を検索できる。ヘッダー `Authorization` に API キーをそのまま入れる（`Bearer` は付けない）点が他と異なる。

scripts/free/25_pexels.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Pexels API: フリー写真・動画を検索する。

https://www.pexels.com/api/ で API キーを無料取得する。
"""
import os
import sys

import httpx

key = os.environ.get("PEXELS_API_KEY")
if not key:
    print("[SKIP] 環境変数 PEXELS_API_KEY が未設定です。")
    sys.exit(0)

r = httpx.get("https://api.pexels.com/v1/search",
              params={"query": "tokyo night", "per_page": 5, "locale": "ja-JP"},
              headers={"Authorization": key}, timeout=20)
r.raise_for_status()
for p in r.json()["photos"]:
    print(f"{p['src']['medium']}  by {p['photographer']}")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 PEXELS_API_KEY が未設定です。
```

6.3 Pixabay API

公式 https://pixabay.com/api/docs/

認証 API キー（無料登録）

料金 無料（1 分 100 リクエスト）

写真・イラスト・ベクター画像・動画を検索できる。`lang=ja` で日本語のキーワード検索ができる。画像 URL は 24 時間で無効になるため、表示する画像は自前のサーバーにコピーするよう求められている。

scripts/free/26_pixabay.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Pixabay API: フリー画像・イラストを検索する。

https://pixabay.com/api/docs/ でログインすると API キーが表示される。
"""
import os
import sys

import httpx

key = os.environ.get("PIXABAY_API_KEY")
if not key:
    print("[SKIP] 環境変数 PIXABAY_API_KEY が未設定です。")
    sys.exit(0)

r = httpx.get("https://pixabay.com/api/",
              params={"key": key, "q": "桜", "lang": "ja", "image_type": "illustration",
                      "per_page": 5}, timeout=20)
r.raise_for_status()
d = r.json()
print("ヒット数:", d["totalHits"])
for h in d["hits"]:
    print(f"{h['webformatURL']}  tags={h['tags']}")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 PIXABAY_API_KEY が未設定です。
```

6.4 DiceBear

公式　https://www.dicebear.com/

認証　不要

料金　無料（OSS。npm パッケージでローカル生成も可能）

文字列（seed）からアバター画像を生成する。同じ seed からは常に同じ画像が得られる。

scripts/free/27_dicebear.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""DiceBear: 文字列 (seed) からアバター画像を自動生成する。

同じ seed なら常に同じ画像になるので、ユーザー ID を seed にすると
「プロフィール画像未設定ユーザーのデフォルトアイコン」として使える。
"""
from pathlib import Path

import httpx
```

```python

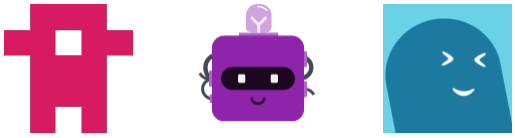
OUT = Path(__file__).resolve().parents[2] / "outputs" / "img"
OUT.mkdir(parents=True, exist_ok=True)

for style in ["identicon", "bottts", "thumbs"]:
    url = f"https://api.dicebear.com/9.x/{style}/png"
    r = httpx.get(url, params={"seed": "kklab", "size": 128}, timeout=20)
    r.raise_for_status()
    (OUT / f"dicebear_{style}.png").write_bytes(r.content)
    print(f"{style:<10} -> dicebear_{style}.png ({len(r.content):,} bytes)")

# SVG なら <img src="..."> にそのまま書ける
print('HTML 例: <img src="https://api.dicebear.com/9.x/identicon/svg?seed=user123">')
```

実行結果　`uv run scripts/free/27_dicebear.py`

```
identicon  -> dicebear_identicon.png (2,529 bytes)
bottts     -> dicebear_bottts.png (5,939 bytes)
thumbs     -> dicebear_thumbs.png (4,142 bytes)
HTML 例: <img src="https://api.dicebear.com/9.x/identicon/svg?seed=user123">
```



左から `identicon`、`bottts`、`thumbs` スタイルで、seed はいずれも `kklab` である。ユーザー ID を seed にすれば、プロフィール画像が未設定のユーザーにも個別のアイコンを表示できる。SVG 形式なら `<img>` タグの `src` に URL を書くだけで済む。

6.5　Iconify API

公式　`https://iconify.design/docs/api/`

認証　不要

料金　無料（各アイコンセットのライセンスに従う）

Material Design Icons、Tabler、Font Awesome など 200 以上のアイコンセット（計 20 万以上のアイコン）を、統一された URL 形式で配信する。

`scripts/free/28_iconify.py`

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Iconify API: 多数のアイコンセットからアイコンを検索・取得する。"""
import httpx

with httpx.Client(base_url="https://api.iconify.design", timeout=20) as client:
    # キーワード検索
    r = client.get("/search", params={"query": "train", "limit": 32})  # limit の最小値は 32
    icons = r.json()["icons"]
    print(f"検索結果 {len(icons)} 件:", ", ".join(icons[:8]), "...")

    # SVG を直接取得 (色・サイズも指定可能)
    svg = client.get("/mdi/home.svg", params={"color": "#1e66f5", "height": 32}).text
```

```
    print("SVG:", svg[:110], "...")

# HTML ではタグを書くだけで使える
print('HTML 例: <img src="https://api.iconify.design/mdi/home.svg?color=%231e66f5">')
```

実行結果　`uv run scripts/free/28_iconify.py`

```
検索結果 32 件: material-symbols:train, material-symbols:train-outline,
↪  material-symbols:train-outline-rounded, material-symbols:train-rounded,
↪  material-symbols-light:train, material-symbols-light:train-outline,
↪  material-symbols-light:train-outline-rounded, material-symbols-light:train-rounded ...
SVG: <svg xmlns="http://www.w3.org/2000/svg" width="32" height="32" viewBox="0 0 24
↪  24"><path fill="#1e66f5" d="M10 ...
HTML 例: <img src="https://api.iconify.design/mdi/home.svg?color=%231e66f5">
```

検索 API の `limit` は最小値が 32 で、それより小さい値を指定しても 32 件返る。SVG の URL では `color` や `height` を指定でき、色の `#` は URL エンコードして `%23` と書く。React や Vue 向けの公式コンポーネントもあり、使うアイコンだけを読み込める。

第 7 章　知識・コンテンツ

7.1　Wikipedia / MediaWiki API

公式　https://www.mediawiki.org/wiki/API:Main_page

認証　不要（連絡先入りの User-Agent を推奨）

料金　無料

Wikipedia の記事の検索・取得には、汎用の Action API（/w/api.php）と、要約などを手軽に取得できる REST API がある。

scripts/free/29_wikipedia.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Wikipedia / MediaWiki API: 記事の検索と要約の取得。

Wikimedia は連絡先を含む User-Agent の指定を求めている。
"""
import httpx

UA = "kklab-api-guide/1.0 (+https://github.com/katzkawai/kklab-api-list)"

with httpx.Client(headers={"User-Agent": UA}, timeout=20) as client:
    # 全文検索 (Action API)
    r = client.get("https://ja.wikipedia.org/w/api.php",
                   params={"action": "query", "list": "search", "srsearch": "行動経済学",
                           "srlimit": 5, "format": "json"})
    print("検索結果:")
    for s in r.json()["query"]["search"]:
        print(f"  {s['title']} ({s['wordcount']:,} 語)")

    # 記事の要約 (REST API)
    r = client.get("https://ja.wikipedia.org/api/rest_v1/page/summary/名古屋市立大学")
    d = r.json()
    print(f"\n{d['title']}: {d['extract'][:150]}...")
    print("URL:", d["content_urls"]["desktop"]["page"])
```

実行結果　uv run scripts/free/29_wikipedia.py

```
検索結果:
  行動経済学 (5,311 語)
  経済学 (6,817 語)
  開発経済学 (1,914 語)
  経済学者 (2,419 語)
  環境経済学 (2,151 語)

名古屋市立大学: 名古屋市立大学 は、愛知県名古屋市瑞穂区瑞穂町字川澄1番地に本部を置く日本の公立大
↪  学である。...
```

```
URL: https://ja.wikipedia.org/wiki/%E5%90%8D%E5%8F%A4%E5%B1%8B%E5%B8%82%E7%AB%8B%E5%A4%A7」
↪  %E5%AD%A6
```

Action API の全文検索で「行動経済学」に関連する記事が、REST API の `page/summary` で記事冒頭の要約が得られた。要約にはサムネイル画像の URL も含まれるため、用語のツールチップ表示などに使える。記事の本文はクリエイティブ・コモンズ表示–継承ライセンス（CC BY-SA）なので、転載する際は出典とライセンスを明記する。

7.2 Open Library API

公式 `https://openlibrary.org/developers/api`
認証 不要
料金 無料

インターネットアーカイブが運営する書誌データベースで、書籍検索、ISBN からの書誌取得、表紙画像の取得ができる。

`scripts/free/30_open_library.py`

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Open Library API: 書籍の検索と表紙画像の URL 取得。"""
import httpx

UA = "kklab-api-guide/1.0 (+https://github.com/katzkawai/kklab-api-list)"
r = httpx.get("https://openlibrary.org/search.json",
              params={"q": "the wealth of nations", "limit": 3,
                      "fields": "title,author_name,first_publish_year,cover_i,edition_count"},
              headers={"User-Agent": UA}, timeout=30)
r.raise_for_status()
d = r.json()
print("ヒット件数:", d["numFound"])
for b in d["docs"]:
    print(f"{b['title']} / {', '.join(b.get('author_name', []))} "
          f"({b.get('first_publish_year')}, {b.get('edition_count')} 版)")
    if b.get("cover_i"):
        print(f"  表紙: https://covers.openlibrary.org/b/id/{b['cover_i']}-M.jpg")
```

実行結果 `uv run scripts/free/30_open_library.py`

```
ヒット件数: 1961
The Wealth of Nations / Adam Smith (1776, 123 版)
  表紙: https://covers.openlibrary.org/b/id/12816911-M.jpg
Inquiry into the nature and causes of the wealth of nations / Adam Smith, Andrew Skinner
↪  (1788, 41 版)
  表紙: https://covers.openlibrary.org/b/id/8243302-M.jpg
The Wealth of Nations / Adam Smith (2011, 7 版)
  表紙: https://covers.openlibrary.org/b/id/12105860-M.jpg
```

`fields` で必要な項目だけを指定すると、レスポンスが大幅に小さくなる。`cover_i`（表紙 ID）から、`-S`・`-M`・`-L` の 3 サイズの表紙画像の URL を作れる。アダム・スミス『国富論』は 123 版が登録されていた。洋書に強い一方、和書の収録は少ない。

7.3 Google Books API

公式 `https://developers.google.com/books`

認証 API キー（少量なら不要）

料金 無料（1 日 1,000 リクエストが既定の上限）

Google ブックスの書誌情報を検索できる。和書も多く収録されており、ISBN 検索（`q=isbn:978...`）もできる。

scripts/free/31_google_books.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Google Books API: 書籍を検索する (少量なら API キー不要)。"""
import os

import httpx

params = {"q": "intitle:ミクロ経済学", "maxResults": 5, "langRestrict": "ja",
          "printType": "books"}
if key := os.environ.get("GOOGLE_BOOKS_API_KEY"):
    params["key"] = key  # キーを付けると利用上限が上がる

r = httpx.get("https://www.googleapis.com/books/v1/volumes", params=params, timeout=20)
if r.status_code == 429:
    # キーなしの場合は全利用者で共有の上限に達すると 429 になる
    print("429 Too Many Requests: GOOGLE_BOOKS_API_KEY を設定して再実行してください。")
    raise SystemExit(1)
r.raise_for_status()
d = r.json()
print("ヒット件数:", d.get("totalItems"))
for v in d.get("items", []):
    info = v["volumeInfo"]
    isbn = next((i["identifier"] for i in info.get("industryIdentifiers", [])
                 if i["type"] == "ISBN_13"), "-")
    print(f"『{info['title']}』 {', '.join(info.get('authors', ['-']))} "
          f"({info.get('publishedDate', '-')}) ISBN:{isbn}")
```

実行結果（エラー終了） `uv run scripts/free/31_google_books.py`

```
429 Too Many Requests: GOOGLE_BOOKS_API_KEY を設定して再実行してください。
```

キーなしで実行したところ、429 Too Many Requests が返った。API キーを付けないリクエストは、全利用者で共有される 1 日あたりの上限にカウントされるため、その上限に達していたのである。本番で使う場合は、Google Cloud でプロジェクトを作って Books API を有効にし、API キーを付けて呼ぶ。このように「キー不要」の API でも、安定して使うにはキーの登録が事実上必要なことがある。

7.4 Hacker News API

公式 https://github.com/HackerNews/API

認証 不要

料金 無料

技術系ニュースサイト Hacker News のデータを、Firebase 上の JSON として公開している。

scripts/free/32_hacker_news.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Hacker News API (Firebase): トップ記事を取得する。

一覧 API は ID の配列だけを返すので、各記事は個別に取得する。
並列に取得すると速い。
"""
import asyncio

import httpx

BASE = "https://hacker-news.firebaseio.com/v0"


async def main() -> None:
    async with httpx.AsyncClient(base_url=BASE, timeout=20) as client:
        ids = (await client.get("/topstories.json")).json()[:5]
        items = await asyncio.gather(*(client.get(f"/item/{i}.json") for i in ids))
        for r in items:
            it = r.json()
            print(f"▲{it.get('score', 0):>4}  {it['title']}")
            print(f"       {it.get('url', 'https://news.ycombinator.com/item?id=' + str(it['id']))}")


asyncio.run(main())
```

実行結果 uv run scripts/free/32_hacker_news.py

```
▲ 451  Whistle: Speech to Text in 16.9 MB
       https://cactuscompute.com/blog/whistle
▲ 213  Theranos.world
       https://www.theranos.world/
▲ 323  Why isn't the industry freaking out about DeepSeek 4.1 Flash?
       https://www.dgt.is/blog/2026-10-07-deepseek-freek-out/
▲  94  The value of not getting to the point (2015)
       https://ken.arneson.name/2015/11/the-value-of-not-getting-to-the-point/
▲  43  AI-ready biological data: $1.8B global commitment
       https://biohub.org/news/virtual-biology-initiative-expansion/
```

`/topstories.json` は記事 ID の配列だけを返すので、各記事の詳細は ID ごとに個別に取得する必要がある。このような「N+1 回のリクエスト」が必要な場合は、`httpx.AsyncClient` と `asyncio.gather` で並行に取得すると、1 件ずつ順に取得するよ

りも大幅に速くなる。

7.5 PokéAPI

公式 https://pokeapi.co/
認証 不要
料金 無料

ポケモンの種族・技・タイプなどを網羅した API である。データ構造が豊かで、リソース間のリンク（URL）を辿る設計になっているため、REST API の学習やポートフォリオ作成の題材として定番になっている。

scripts/free/33_pokeapi.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""PokéAPI: ポケモンのデータを取得する (学習・ポートフォリオ向けの定番)。"""
import httpx

with httpx.Client(base_url="https://pokeapi.co/api/v2", timeout=20) as client:
    for name in ["pikachu", "bulbasaur"]:
        p = client.get(f"/pokemon/{name}").json()
        sp = client.get(f"/pokemon-species/{p['id']}").json()
        ja = next(n["name"] for n in sp["names"] if n["language"]["name"] == "ja")
        types = "/".join(t["type"]["name"] for t in p["types"])
        stats = {s["stat"]["name"]: s["base_stat"] for s in p["stats"]}
        print(f"No.{p['id']:03d} {ja} ({p['name']})  タイプ: {types}  "
              f"HP {stats['hp']} / 攻撃 {stats['attack']} / 素早さ {stats['speed']}")
        print(f"  画像: {p['sprites']['front_default']}")
```

実行結果 uv run scripts/free/33_pokeapi.py

```
No.025 ピカチュウ (pikachu)  タイプ: electric  HP 35 / 攻撃 55 / 素早さ 90
  画像: https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/25.png
No.001 フシギダネ (bulbasaur)  タイプ: grass/poison  HP 45 / 攻撃 49 / 素早さ 45
  画像: https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/1.png
```

`/pokemon/{name}` にはステータスや画像、`/pokemon-species/{id}` には多言語の名前や図鑑説明が入っている。日本語名を得るには、`names` の中から `language.name == "ja"` の要素を探す。

7.6 GitHub REST API

公式 https://docs.github.com/rest
認証 任意（トークン / OAuth / GitHub App）
料金 無料（未認証は 1 時間 60 リクエスト、認証済みは 5,000）

リポジトリ、Issue、Pull Request、ユーザー、リリースなど、GitHub のほぼすべての情報を扱える。

scripts/free/34_github.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""GitHub REST API: リポジトリ情報とリリースを取得する。

未認証は 60 リクエスト/時。GITHUB_TOKEN を設定すると 5,000 リクエスト/時になる。
"""
import os

import httpx

headers = {"Accept": "application/vnd.github+json", "X-GitHub-Api-Version": "2022-11-28"}
if token := os.environ.get("GITHUB_TOKEN"):
    headers["Authorization"] = f"Bearer {token}"

with httpx.Client(base_url="https://api.github.com", headers=headers, timeout=20) as client:
    r = client.get("/repos/astral-sh/uv")
    repo = r.json()
    print(f"{repo['full_name']}: ★{repo['stargazers_count']:,}  fork {repo['forks_count']:,}")
    print(f"  {repo['description']}")

    rel = client.get("/repos/astral-sh/uv/releases", params={"per_page": 3}).json()
    print("最新リリース:", ", ".join(f"{x['tag_name']} ({x['published_at'][:10]})" for x in rel))

    print(f"残りリクエスト数: {r.headers['x-ratelimit-remaining']}/{r.headers['x-ratelimit-limit']}")
```

実行結果　`uv run scripts/free/34_github.py`

```
astral-sh/uv: ★90,509  fork 3,637
  An extremely fast Python package and project manager, written in Rust.
最新リリース: 0.12.24 (2026-10-08), 0.12.23 (2026-10-03), 0.12.22 (2026-10-02)
残りリクエスト数: 59/60
```

レスポンスヘッダー `x-ratelimit-remaining` で、残りのリクエスト数を確認できる。未認証では 1 時間に 60 回しか呼べないため、アプリに組み込む場合は個人アクセストークンや GitHub App を使う。`X-GitHub-Api-Version` ヘッダーで API のバージョンを固定しておくと、将来の仕様変更の影響を受けにくい。

第 8 章　金融・為替

8.1　Frankfurter

公式　https://frankfurter.dev/

認証　不要

料金　無料（OSS）

欧州中央銀行（ECB）が公表する参照レートを返す為替 API である。最新レート、過去の特定日、期間の時系列を取得できる。

scripts/free/35_frankfurter.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Frankfurter: 欧州中央銀行の参照レートで為替を取得・換算する。"""
import httpx

with httpx.Client(base_url="https://api.frankfurter.dev/v1", timeout=20) as client:
    latest = client.get("/latest", params={"base": "USD", "symbols": "JPY,EUR,CNY"}).json()
    print(f"{latest['date']} の 1 USD:", latest["rates"])

    # 金額換算
    conv = client.get("/latest", params={"amount": 10000, "base": "JPY", "symbols": "USD"}).json()
    print(f"10,000 JPY = {conv['rates']['USD']} USD")

    # 期間指定の時系列
    ts = client.get("/2026-09-01..2026-09-07", params={"base": "USD", "symbols": "JPY"}).json()
    for d, rate in ts["rates"].items():
        print(f"  {d}: {rate['JPY']} 円/ドル")
```

実行結果　`uv run scripts/free/35_frankfurter.py`

```
2026-10-08 の 1 USD: {'CNY': 6.7023, 'EUR': 0.89397, 'JPY': 158.28}
10,000 JPY = 63.18 USD
  2026-09-01: 160.16 円/ドル
  2026-09-02: 159.6 円/ドル
  2026-09-03: 156.01 円/ドル
  2026-09-04: 156.25 円/ドル
  2026-09-07: 154.75 円/ドル
```

`amount` を指定すると金額の換算もできる。時系列は `/開始日..終了日` の形式で指定する。ECB のレートは営業日に 1 回（中央ヨーロッパ時間 16 時頃）更新されるため、土日の値は出力に含まれない（9 月 5・6 日が抜けている）。リアルタイムの値ではないので、実際の決済に使うレートとしては適さない。

8.2 CoinGecko API

公式 https://www.coingecko.com/en/api

認証 Demo キー（無料登録、少量ならキーなしでも可）

料金 Demo プランは無料（月 1 万リクエスト）。有料プランあり

暗号資産の価格・時価総額・取引量などを取得できる。

scripts/free/36_coingecko.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""CoinGecko API: 暗号資産の価格を取得する。

Demo プランの API キー (無料) を COINGECKO_API_KEY に設定すると安定して使える。
"""
import os

import httpx

headers = {}
if key := os.environ.get("COINGECKO_API_KEY"):
    headers["x-cg-demo-api-key"] = key

r = httpx.get("https://api.coingecko.com/api/v3/simple/price",
              params={"ids": "bitcoin,ethereum,solana", "vs_currencies": "jpy,usd",
                      "include_24hr_change": "true"}, headers=headers, timeout=20)
r.raise_for_status()
for coin, v in r.json().items():
    print(f"{coin:<9} {v['jpy']:>14,.0f} 円  ${v['usd']:>10,.2f}  24h {v['jpy_24h_change']:+.2f}%")
```

実行結果 uv run scripts/free/36_coingecko.py

```
bitcoin       12,930,159 円  $ 81,842.00  24h -1.73%
ethereum         391,340 円  $  2,477.00  24h -3.75%
solana            17,429 円  $    110.32  24h -5.13%
```

simple/price に通貨 ID と法定通貨をカンマ区切りで渡すと、複数の組み合わせをまとめて取得できる。include_24hr_change で 24 時間の変化率も得られる。

第 9 章　ユーティリティ

9.1　goqr.me QR Code API

公式　https://goqr.me/api/

認証　不要

料金　無料

URL や文字列から QR コード画像を生成する。

scripts/free/37_qrserver.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""goqr.me QR Code API: URL などの文字列から QR コード画像を生成する。"""
from pathlib import Path

import httpx

OUT = Path(__file__).resolve().parents[2] / "outputs" / "img"
OUT.mkdir(parents=True, exist_ok=True)

params = {"data": "https://www.nagoya-cu.ac.jp/", "size": "240x240", "margin": 8,
          "format": "png", "ecc": "M"}
r = httpx.get("https://api.qrserver.com/v1/create-qr-code/", params=params, timeout=20)
r.raise_for_status()
(OUT / "qrcode.png").write_bytes(r.content)
print(f"保存: qrcode.png ({len(r.content):,} bytes)")
print("HTML 例: <img src=\"https://api.qrserver.com/v1/create-qr-code/?size=150x150&data=...\">")
```

実行結果　`uv run scripts/free/37_qrserver.py`

```
保存: qrcode.png (520 bytes)
HTML 例: <img src="https://api.qrserver.com/v1/create-qr-code/?size=150x150&data=...">
```



`ecc` は誤り訂正レベル（L / M / Q / H）で、高いほど汚れや欠けに強くなる代わりに模様が細かくなる。サーバーで生成する必要がなければ、`qrcode`（Python）や `qrcode.js`（JavaScript）などのライブラリを使ってローカルで生成するのも簡単である。

9.2 Microlink

公式 https://microlink.io/

認証 不要（無料枠を超えると有料キー）

料金 無料枠は 1 日 50 リクエスト

URL を渡すと、ページのタイトル・説明・OGP 画像・ロゴなどを抽出して返す。チャットや SNS でよく見る「リンクカード（リンクプレビュー）」を作るのに使える。

scripts/free/38_microlink.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Microlink: URL からタイトル・説明・OGP 画像などのメタデータを取得する。

SNS 風の「リンクカード」表示に使える。無料枠は 1 日 50 リクエスト。
"""
import httpx

for url in ["https://www.python.org/", "https://github.com/astral-sh/uv"]:
    r = httpx.get("https://api.microlink.io/", params={"url": url}, timeout=30)
    d = r.json()
    if d["status"] != "success":
        print(url, "->", d.get("message"))
        continue
    m = d["data"]
    print(f"[{m.get('publisher')}] {m['title']}")
    print(f"  {(m.get('description') or '')[:80]}")
    print(f"  画像: {(m.get('image') or {}).get('url')}")
```

実行結果 uv run scripts/free/38_microlink.py

```
[Python.org] Welcome to Python.org
  The official home of the Python Programming Language
  画像: https://www.python.org/static/opengraph-icon-200x200.png
[GitHub] GitHub - astral-sh/uv: An extremely fast Python package and project manager,
↪  written in Rust.
  An extremely fast Python package and project manager, written in Rust. - astral-
  画像: https://opengraph.githubassets.com/a6e8ffc02446a1683877e1d4de3fa54b253dcd89bad1f1b」
  ↪  fed639ae57282423b/astral-sh/uv
```

自分でページを取得して OGP タグを解析することもできるが、JavaScript で描画されるページへの対応や、悪意ある URL からサーバーを守る対策（SSRF 対策）などを考えると、専用 API に任せるほうが手軽で安全である。

9.3 Google Fonts

公式 https://developers.google.com/fonts

認証 不要（フォント一覧の Developer API はキーが必要）

料金 無料

Web フォントを配信する。通常は HTML の <link> タグで CSS を読み込むだけで使

える。

scripts/free/39_google_fonts.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Google Fonts (CSS API): Web フォントを読み込むための CSS を取得する。

通常は HTML の <link> で読み込むだけでよい。ここでは返ってくる CSS の中身を確認する。
(フォント一覧を取得する Developer API は API キーが必要)
"""
import httpx

params = {"family": "Noto Sans JP:wght@400;700", "display": "swap"}
# User-Agent によって返るフォント形式 (woff2 など) が変わる
ua = "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 Chrome/130 Safari/537.36"
r = httpx.get("https://fonts.googleapis.com/css2", params=params,
              headers={"User-Agent": ua}, timeout=20)
r.raise_for_status()
css = r.text
print(f"CSS {len(css):,} 文字, @font-face ブロック数: {css.count('@font-face')}")
for line in css.splitlines()[:9]:
    print(line if len(line) < 90 else line[:90] + " ...")
print("\nHTML 例:")
print('<link href="https://fonts.googleapis.com/css2?family=Noto+Sans+JP:wght@400;700'
      '&display=swap" rel="stylesheet">')
```

実行結果　`uv run scripts/free/39_google_fonts.py`

```
CSS 228,130 文字, @font-face ブロック数: 248
@font-face {
  font-family: 'Noto Sans JP';
  font-style: normal;
  font-weight: 400;
  font-display: swap;
  src:
  ↪  url(https://fonts.gstatic.com/s/notosansjp/v57/-F62fjtqLzI2JPCgQBnw7HFow2oe2EcP5pp0
  ↪  ...
  unicode-range: U+25ee8, U+25f23, U+25f5c, U+25fd4, U+25fe0, U+25ffb, U+2600c, U+26017,
  ↪  U ...
}
@font-face {

HTML 例:
<link href="https://fonts.googleapis.com/css2?family=Noto+Sans+JP:wght@400;700&display=sw」
↪  ap"
↪  rel="stylesheet">
```

Noto Sans JP の CSS には 248 個の `@font-face` ブロックが含まれていた。日本語フォントはファイルが大きいため、文字を Unicode の範囲（`unicode-range`）ごとに約 120 のファイルに分割し、ページで実際に使われている文字を含むファイルだけをブラウザがダウンロードする仕組みになっている（2 つのウェイト × 約 124 分割）。`display=swap` は、フォントの読み込み中も代替フォントで文字を表示させる指定である。

9.4 World Time API / Time API

公式 https://worldtimeapi.org/

認証 不要

料金 無料（稼働が不安定）

タイムゾーンごとの現在時刻と UTC オフセット、夏時間かどうかを返す。

scripts/free/40_worldtimeapi.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""World Time API / Time API: タイムゾーンごとの現在時刻を取得する。

World Time API は稼働が不安定なことがあるため、失敗したら代替の timeapi.io を使う。
無償 API を使うときは、このように「代替先へのフォールバック」を用意しておくと安心。
"""
import httpx


def now_in(tz: str) -> str:
    try:
        r = httpx.get(f"https://worldtimeapi.org/api/timezone/{tz}", timeout=5)
        r.raise_for_status()
        d = r.json()
        return f"{d['datetime'][:19]}  UTC{d['utc_offset']}  (worldtimeapi.org)"
    except httpx.HTTPError as e:
        fallback_reason = type(e).__name__
    r = httpx.get("https://timeapi.io/api/time/current/zone", params={"timeZone": tz},
                  timeout=10)
    r.raise_for_status()
    d = r.json()
    return f"{d['dateTime'][:19]}  夏時間: {d['dstActive']}  (timeapi.io, 代替理由: {fallback_reason})"


for tz in ["Asia/Tokyo", "Europe/London", "America/New_York"]:
    print(f"{tz:<17} {now_in(tz)}")
```

実行結果 uv run scripts/free/40_worldtimeapi.py

```
Asia/Tokyo        2026-10-09T08:27:45  夏時間: False  (timeapi.io, 代替理由: ConnectError)
Europe/London     2026-10-09T00:27:47  夏時間: True  (timeapi.io, 代替理由: ConnectError)
America/New_York  2026-10-08T19:27:48  夏時間: True  (timeapi.io, 代替理由: ConnectError)
```

執筆時点では World Time API のサーバーに接続できなかった（接続がリセットされる）。そこでスクリプトでは、失敗したら代替の timeapi.io に切り替えるフォールバックを実装している。出力の「代替理由: ConnectError」は、すべての問い合わせが代替先で処理されたことを示す。無償 API に依存する機能では、このように代替手段を用意しておくか、Python の `zoneinfo` のように API を使わずに済む方法を検討するとよい。現在時刻の取得であれば、実際には `datetime.now(ZoneInfo("Asia/Tokyo"))` で十分なことが多い。

第II部　有償 API

第 II 部では、従量課金やサブスクリプションで提供される API を扱う。多くは無料枠・トライアル・テストモードを備えているため、開発段階では費用をかけずに試せる。一方で、本番運用では想定外のアクセスによって高額の請求が発生しうるので、各サービスの管理画面で利用上限や予算アラートを必ず設定しておく。

第 10 章　生成 AI・機械学習

生成 AI の API は、トークン（文章の断片）の数に応じて課金される。入力（プロンプト）と出力の両方が課金対象で、出力のほうが単価が高い。どのサービスも公式の Python SDK を提供しており、環境変数から API キーを自動で読み込む。

10.1　Claude API (Anthropic)

公式　`https://platform.claude.com/`

認証　API キー（`ANTHROPIC_API_KEY`）

料金　トークン従量課金。例：Claude Opus 5.5 は入力 $4 / 出力 $20（100 万トークンあたり）

Anthropic の Claude を使うための API である。文章生成・要約・翻訳・コード生成・画像や PDF の理解、ツール（関数）呼び出しによるエージェントの構築などができる。

scripts/paid/41_anthropic_claude.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["anthropic"]
# ///
"""Claude API (Anthropic): テキスト生成の基本。

環境変数 ANTHROPIC_API_KEY に API キーを設定する (https://platform.claude.com/)。
"""
import os
import sys

import anthropic

if not os.environ.get("ANTHROPIC_API_KEY"):
    print("[SKIP] 環境変数 ANTHROPIC_API_KEY が未設定です。")
    sys.exit(0)

client = anthropic.Anthropic()  # ANTHROPIC_API_KEY を自動で読み込む

response = client.beta.messages.create(
    model="claude-opus-5-5",
    max_tokens=16000,
    output_config={"effort": "low"},          # 簡単なタスクなので思考量を抑える
    betas=["server-side-fallback-2026-07-01"],
    fallbacks="default",                      # 安全分類器で断られた場合に別モデルで再実行
    system="あなたは Web 開発の講師です。日本語で簡潔に答えてください。",
    messages=[{"role": "user",
               "content": "REST API と GraphQL の違いを、初学者向けに3行で説明してください。"}],
)

if response.stop_reason == "refusal":
    print("リクエストが拒否されました:", response.stop_details)
    sys.exit(1)
```

```
34
35 text = "".join(b.text for b in response.content if b.type == "text")
36 print(text)
37 print(f"\n[model={response.model}, 入力 {response.usage.input_tokens} tokens, "
38       f"出力 {response.usage.output_tokens} tokens]")
```

実行結果　`uv run scripts/paid/41_anthropic_claude.py`

```
- **REST API**：「ユーザー情報は `/users`、投稿は `/posts`」のように、データの種類ごとに用意さ
↪  れた URL へアクセスして取得します。
- **GraphQL**：窓口（URL）は基本的に1つで、「名前とメールだけほしい」のように必要な項目をクエリ
↪  で指定して取得します。
- そのため REST はシンプルで始めやすく、GraphQL は余分なデータの取得や何度ものリクエストを減ら
↪  しやすいのが特徴です。

[model=claude-opus-5-5, 入力 65 tokens, 出力 198 tokens]
```

`anthropic.Anthropic()` は環境変数 `ANTHROPIC_API_KEY` を自動で読み込む。`system` で役割や口調を指定し、`messages` にユーザーの発話を渡す。Claude Opus 5.5 は応答前に必要に応じて思考（thinking）を行うモデルで、`output_config={"effort": "low"}` は、簡単なタスクなので思考の量を抑えて速く安く答えさせる指定である。`fallbacks="default"` は、安全上の分類器がリクエストを拒否した場合に、サーバー側で適切な別モデルに切り替えて再実行させるオプションである。それでも拒否された場合は `stop_reason` が `"refusal"` になるので、本文を読む前に確認している。`usage` から入力・出力のトークン数がわかり、この呼び出しの費用は 1 円に満たない。

10.2　OpenAI API

公式　`https://platform.openai.com/`

認証　API キー（`OPENAI_API_KEY`）

料金　トークン従量課金（プリペイド方式）

GPT シリーズのモデルによるテキスト生成、画像生成、音声認識・合成、埋め込みなどを提供する。

scripts/paid/42_openai.py

```
1  # /// script
2  # requires-python = ">=3.11"
3  # dependencies = ["openai"]
4  # ///
5  """OpenAI API: Responses API によるテキスト生成。
6
7  環境変数 OPENAI_API_KEY に API キーを設定する。
8  """
9  import os
10 import sys
11
12 from openai import OpenAI
13
14 if not os.environ.get("OPENAI_API_KEY"):
15     print("[SKIP] 環境変数 OPENAI_API_KEY が未設定です。")
16     sys.exit(0)
```

```python

client = OpenAI()
model = os.environ.get("OPENAI_MODEL", "gpt-5-mini")
response = client.responses.create(
    model=model,
    instructions="あなたは Web 開発の講師です。日本語で簡潔に答えてください。",
    input="CORS とは何か、初学者向けに3行で説明してください。",
)
print(response.output_text)
print(f"\n[model={response.model}, 入力 {response.usage.input_tokens} tokens, "
      f"出力 {response.usage.output_tokens} tokens]")
```

実行結果　`uv run scripts/paid/42_openai.py`

```
CORS（Cross-Origin Resource Sharing）は、ブラウザが別オリジン（ドメイン・プロトコル・ポートが異
↪ なる）のリソースへアクセスする際のセキュリティ仕様です。
ブラウザは同一生成元ポリシーで制限するため、別オリジンへのリクエストは原則ブロックされます。
サーバーが Access-Control-Allow-Origin などのヘッダーで明示的に許可すると、そのアクセスが可能
↪ になります。

[model=gpt-5-mini-2025-08-07, 入力 49 tokens, 出力 350 tokens]
```

現在の推奨は Responses API（`client.responses.create`）で、`instructions` がシステムプロンプト、`input` がユーザー入力にあたる。`response.output_text` で生成されたテキストをまとめて取り出せる。出力トークンが 350 と本文の長さの割に多いのは、推論モデルが内部で行った思考のトークンも出力として課金されるためである。

10.3　Google Gemini API

公式　`https://ai.google.dev/`

認証　API キー（`GEMINI_API_KEY`、Google AI Studio で発行）

料金　無料枠あり（レート制限付き）。超過分はトークン従量課金

Google の Gemini モデルを使う API である。無料枠があるため、学習や試作に使いやすい。

`scripts/paid/43_gemini.py`

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["google-genai"]
# ///
"""Google Gemini API: google-genai SDK によるテキスト生成。

環境変数 GEMINI_API_KEY に API キーを設定する (Google AI Studio で無料発行)。
"""
import os
import sys

from google import genai

if not os.environ.get("GEMINI_API_KEY"):
    print("[SKIP] 環境変数 GEMINI_API_KEY が未設定です。")
    sys.exit(0)

```

```python
client = genai.Client()  # GEMINI_API_KEY を自動で読み込む
model = os.environ.get("GEMINI_MODEL", "gemini-2.5-flash")
response = client.models.generate_content(
    model=model,
    contents="Web アプリで API キーをフロントエンドに置いてはいけない理由を3行で説明してください。",
)
print(response.text)
u = response.usage_metadata
print(f"\n[model={model}, 入力 {u.prompt_token_count} tokens, 出力 {u.candidates_token_count} tokens]")
```

実行結果　`uv run scripts/paid/43_gemini.py`

```
1. フロントエンドのコードは誰でも閲覧・解析できるため、API キーが公開状態になります。
2. 公開されたキーは悪意のある第三者に不正利用され、サービスへの高額請求やデータ漏洩に繋がる危険があります。
3. そのため、重要なリソースへの不正アクセスや金銭的被害を防ぐため、サーバー側で厳重に管理する必要があります。

[model=gemini-2.5-flash, 入力 22 tokens, 出力 85 tokens]
```

SDK は `google-genai`（import 名は `from google import genai`）を使う。旧 SDK の `google-generativeai` とはパッケージ名も書き方も異なるので、古い記事を参考にするときは注意する。無料枠で送信した内容は、Google のサービス改善に利用されることがある点にも留意する。

10.4　Replicate

公式　`https://replicate.com/`

認証　API トークン（`REPLICATE_API_TOKEN`）

料金　実行時間またはモデルごとの従量課金

画像生成・音声・動画・言語モデルなど、公開されている多数の OSS モデルを API として実行できる。GPU サーバーを自前で用意せずに最新のモデルを試せる。

`scripts/paid/44_replicate.py`

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["replicate"]
# ///
"""Replicate: 公開されている OSS モデル（ここでは画像生成）を API で実行する。

環境変数 REPLICATE_API_TOKEN を設定する。課金は実行時間ベース。
"""
import os
import sys
from pathlib import Path

import replicate

if not os.environ.get("REPLICATE_API_TOKEN"):
    print("[SKIP] 環境変数 REPLICATE_API_TOKEN が未設定です。")
    sys.exit(0)

output = replicate.run(
```

```
    "black-forest-labs/flux-schnell",
    input={"prompt": "a watercolor painting of Nagoya castle in autumn", "num_outputs": 1,
           "aspect_ratio": "3:2", "output_format": "png"},
)
out = Path(__file__).resolve().parents[2] / "outputs" / "img" / "replicate.png"
out.write_bytes(output[0].read())
print("生成画像:", output[0].url, "->", out.name)
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 REPLICATE_API_TOKEN が未設定です。
```

`replicate.run("所有者/モデル名", input={...})` の形で、どのモデルも同じように呼べる。モデルごとに入力パラメータが異なるので、各モデルのページにある API の説明を確認する。

10.5 Hugging Face Inference Providers

公式 `https://huggingface.co/docs/inference-providers`

認証 アクセストークン（`HF_TOKEN`）

料金 無料アカウントにも少額の月間クレジット。超過分は従量課金

Hugging Face Hub 上のモデルを、提携する推論事業者を通じてサーバーレスで実行できる。OpenAI 互換のチャット API と、埋め込み・画像分類などのタスク別 API がある。

scripts/paid/45_huggingface.py

```
# /// script
# requires-python = ">=3.11"
# dependencies = ["huggingface_hub", "numpy"]
# ///
"""Hugging Face Inference Providers: OSS モデルをサーバーレスで推論する。

環境変数 HF_TOKEN にアクセストークンを設定する。無料アカウントにも少額のクレジットがある。
"""
import os
import sys

from huggingface_hub import InferenceClient

if not os.environ.get("HF_TOKEN"):
    print("[SKIP] 環境変数 HF_TOKEN が未設定です。")
    sys.exit(0)

client = InferenceClient()  # HF_TOKEN を自動で読み込む
model = os.environ.get("HF_MODEL", "openai/gpt-oss-20b")
completion = client.chat.completions.create(
    model=model,
    messages=[{"role": "user", "content": "HTTP ステータスコード 404 と 500 の違いを1行で。日本語で。"}],
    max_tokens=512,
)
print(completion.choices[0].message.content)
print(f"\n[model={model}]")

# 埋め込みベクトル (類似検索などに使う)
```

```python
vec = client.feature_extraction("名古屋市立大学", model="sentence-transformers/all-MiniLM-L6-v2")
print(f"埋め込みベクトルの次元数: {len(vec)}  先頭3要素: {[round(float(x), 4) for x in vec[:3]]}")
```

実行結果　`uv run scripts/paid/45_huggingface.py`

```
404 は「ページが見つからない」を示し、500 は「サーバ内部で予期せぬエラーが発生した」を示す。

[model=openai/gpt-oss-20b]
埋め込みベクトルの次元数: 384  先頭 3 要素: [-0.0142, 0.0944, 0.0182]
```

`client.chat.completions.create` は OpenAI の SDK と同じ書き方で、ここでは OSS の `openai/gpt-oss-20b` を使った。`feature_extraction` は文章を数値ベクトル（埋め込み）に変換する。`all-MiniLM-L6-v2` は 384 次元のベクトルを返し、ベクトル間のコサイン類似度を計算すれば、意味の近い文章を検索できる。埋め込みの結果は NumPy 配列で返るため、依存関係に `numpy` を加えている（加えないと ImportError になる）。

第 11 章　決済

決済サービスは、初期費用・月額費用が無料で、決済ごとに手数料がかかるものが多い。いずれもテストモード（サンドボックス）があり、実際の請求を発生させずに開発できる。カード番号などの機密情報は自分のサーバーを通さず、決済サービスが提供する画面や JavaScript でトークン化するのが原則である。

11.1　Stripe

公式　https://stripe.com/jp

認証　シークレットキー（STRIPE_SECRET_KEY、テストは sk_test_...）

料金　日本の国内カードは 3.6% が目安

カード決済、サブスクリプション、請求書、マーケットプレイス向けの送金などを提供する、開発者に人気の決済サービスである。

scripts/paid/46_stripe.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["stripe"]
# ///
"""Stripe: Checkout Session (Stripe がホストする決済ページ) を作成する。

環境変数 STRIPE_SECRET_KEY にテストモードのシークレットキー (sk_test_...) を設定する。
テストモードでは実際の請求は発生しない。カード番号 4242 4242 4242 4242 でテスト可能。
"""
import os
import sys

import stripe

key = os.environ.get("STRIPE_SECRET_KEY")
if not key:
    print("[SKIP] 環境変数 STRIPE_SECRET_KEY が未設定です。")
    sys.exit(0)
if not key.startswith("sk_test_"):
    print("安全のため、このサンプルはテストモードのキー (sk_test_) でのみ動作します。")
    sys.exit(1)

client = stripe.StripeClient(key)
session = client.v1.checkout.sessions.create(params={
    "mode": "payment",
    "line_items": [{
        "price_data": {"currency": "jpy", "unit_amount": 1500,
                       "product_data": {"name": "API 利用ガイド (PDF)"}},
        "quantity": 1,
    }],
    "success_url": "https://example.com/success?session_id={CHECKOUT_SESSION_ID}",
    "cancel_url": "https://example.com/cancel",
})
```

```
print("Checkout Session ID:", session.id)
print("決済ページ URL:", session.url)
# 決済完了の確認は Webhook (checkout.session.completed イベント) で受け取るのが基本
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 STRIPE_SECRET_KEY が未設定です。
<claude-code-hint v="1" type="plugin" value="stripe@claude-plugins-official" />
```

Checkout Session を作ると、Stripe がホストする決済ページの URL が返る。ユーザーをその URL にリダイレクトするだけで、カード入力・3D セキュア認証・領収書メールまで Stripe が処理する。決済が完了したかどうかは `success_url` へのリダイレクトではなく、Webhook（`checkout.session.completed`）で確認する。リダイレクトはユーザーがブラウザを閉じると届かないためである。スクリプトでは、誤って本番のキーで実行しないように、`sk_test_` で始まるキーでのみ動くようにしている。

11.2 PAY.JP

公式 `https://pay.jp/`

認証 秘密鍵（`PAYJP_SECRET_KEY`）

料金 決済手数料のみ

日本の事業者が運営する、国内向けのオンライン決済サービスである。API の設計は Stripe に近い。

`scripts/paid/47_payjp.py`

```
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""PAY.JP: REST API で顧客を作成し、課金一覧を取得する。

環境変数 PAYJP_SECRET_KEY にテスト用秘密鍵 (sk_test_...) を設定する。
認証は秘密鍵をユーザー名にした Basic 認証。カード情報はフロント側の payjp.js で
トークン化し、サーバーにはトークンだけを送るのが原則。
"""
import os
import sys

import httpx

key = os.environ.get("PAYJP_SECRET_KEY")
if not key:
    print("[SKIP] 環境変数 PAYJP_SECRET_KEY が未設定です。")
    sys.exit(0)

with httpx.Client(base_url="https://api.pay.jp/v1", auth=(key, ""), timeout=20) as client:
    cus = client.post("/customers", data={"email": "test@example.com",
                                          "description": "ガイド用テスト顧客"}).json()
    print("顧客作成:", cus["id"], cus["email"])
    charges = client.get("/charges", params={"limit": 3}).json()
    print("直近の課金:", [(c["id"], c["amount"], c["currency"]) for c in charges["data"]])
    # 課金の作成例: client.post("/charges", data={"amount": 1000, "currency": "jpy",
```

```
    #                                       "card": "<payjp.js で作ったトークン>"})
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 PAYJP_SECRET_KEY が未設定です。
```

認証は、秘密鍵をユーザー名、パスワードを空にした Basic 認証で行う（httpx では `auth=(key, "")`）。課金を作成するには、フロントエンドの payjp.js でカード情報をトークン化し、そのトークンをサーバーに送って `/charges` に渡す。

11.3 Square

公式 `https://developer.squareup.com/`

認証 アクセストークン（`SQUARE_ACCESS_TOKEN`）

料金 決済手数料のみ

店舗の POS レジとオンライン決済を一元管理できる。Sandbox 環境ではテスト用の店舗やカードが使える。

scripts/paid/48_square.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Square: Sandbox 環境で店舗 (Location) 一覧を取得し、決済リンクを作成する。

環境変数 SQUARE_ACCESS_TOKEN に Sandbox のアクセストークンを設定する。
"""
import os
import sys
import uuid

import httpx

token = os.environ.get("SQUARE_ACCESS_TOKEN")
if not token:
    print("[SKIP] 環境変数 SQUARE_ACCESS_TOKEN が未設定です。")
    sys.exit(0)

headers = {"Authorization": f"Bearer {token}", "Square-Version": "2025-01-23"}
with httpx.Client(base_url="https://connect.squareupsandbox.com/v2", headers=headers,
                  timeout=20) as client:
    locations = client.get("/locations").json()["locations"]
    loc = locations[0]
    print("Location:", loc["id"], loc["name"], loc["currency"])

    link = client.post("/online-checkout/payment-links", json={
        "idempotency_key": str(uuid.uuid4()),  # 二重実行を防ぐためのキー
        "quick_pay": {"name": "コーヒー豆 200g", "location_id": loc["id"],
                      "price_money": {"amount": 1200, "currency": loc["currency"]}},
    }).json()
    print("決済リンク:", link["payment_link"]["url"])
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 SQUARE_ACCESS_TOKEN が未設定です。
```

Square-Version ヘッダーで API のバージョンを指定する。idempotency_key（冪等キー）は、ネットワークエラーで同じリクエストを再送しても二重に処理されないようにするための一意な値である。決済のように「二度実行されると困る」操作では、Stripe なども含めて冪等キーを付けるのが定石である。

11.4 PayPal

公式 https://developer.paypal.com/

認証 Client ID と Secret（PAYPAL_CLIENT_ID / PAYPAL_CLIENT_SECRET）

料金 決済手数料のみ

PayPal アカウントによる決済を受け付ける。海外の利用者が多いサービスでよく使われる。

scripts/paid/49_paypal.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""PayPal: Sandbox で OAuth トークンを取得し、注文 (Order) を作成する。

環境変数 PAYPAL_CLIENT_ID / PAYPAL_CLIENT_SECRET に Sandbox アプリの認証情報を設定する。
"""
import os
import sys

import httpx

cid, secret = os.environ.get("PAYPAL_CLIENT_ID"), os.environ.get("PAYPAL_CLIENT_SECRET")
if not (cid and secret):
    print("[SKIP] 環境変数 PAYPAL_CLIENT_ID / PAYPAL_CLIENT_SECRET が未設定です。")
    sys.exit(0)

BASE = "https://api-m.sandbox.paypal.com"
with httpx.Client(base_url=BASE, timeout=20) as client:
    # 1) Client Credentials でアクセストークンを取得
    tok = client.post("/v1/oauth2/token", auth=(cid, secret),
                      data={"grant_type": "client_credentials"}).json()["access_token"]
    # 2) 注文を作成
    order = client.post("/v2/checkout/orders", headers={"Authorization": f"Bearer {tok}"}, json={
        "intent": "CAPTURE",
        "purchase_units": [{"amount": {"currency_code": "JPY", "value": "1500"}}],
    }).json()
    print("Order ID:", order["id"], "status:", order["status"])
    approve = next(l["href"] for l in order["links"] if l["rel"] == "approve")
    print("購入者の承認 URL:", approve)
    # 購入者が承認した後、POST /v2/checkout/orders/{id}/capture で確定する
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 PAYPAL_CLIENT_ID / PAYPAL_CLIENT_SECRET が未設定です。
```

OAuth 2.0 の Client Credentials フローでアクセストークンを取得してから API を呼ぶ、という 2 段階の手順は、多くの業務系 API に共通する。注文を作成すると購入者の承認 URL が返り、購入者が承認した後に capture を呼んで決済を確定する。

11.5 KOMOJU

公式 https://ja.komoju.com/

認証 秘密鍵（KOMOJU_SECRET_KEY）

料金 決済手段ごとの手数料

コンビニ払い、銀行振込、PayPay、楽天ペイなど、日本特有の決済手段に対応した決済サービスである。

scripts/paid/50_komoju.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""KOMOJU: コンビニ決済・PayPay などに対応した決済セッションを作成する。

環境変数 KOMOJU_SECRET_KEY にテストモードの秘密鍵を設定する。
"""
import os
import sys

import httpx

key = os.environ.get("KOMOJU_SECRET_KEY")
if not key:
    print("[SKIP] 環境変数 KOMOJU_SECRET_KEY が未設定です。")
    sys.exit(0)

r = httpx.post("https://komoju.com/api/v1/sessions", auth=(key, ""), timeout=20, json={
    "amount": 2000,
    "currency": "JPY",
    "default_locale": "ja",
    "payment_types": ["konbini", "paypay", "credit_card"],
    "return_url": "https://example.com/thanks",
})
r.raise_for_status()
s = r.json()
print("Session ID:", s["id"])
print("決済ページ URL:", s["session_url"])
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 KOMOJU_SECRET_KEY が未設定です。
```

`payment_types` に受け付ける決済手段を並べてセッションを作ると、利用者が手段を選べる決済ページの URL が返る。コンビニ払いのように支払いが後日になる手段では、

Webhook で入金を確認してから商品を発送する。

第 12 章　認証・ユーザー管理

ログイン機能を自前で実装すると、パスワードの安全な保存、メール確認、パスワード再設定、多要素認証、ソーシャルログインなど、考慮すべきことが非常に多い。認証サービスを使えばこれらを任せられる。

12.1　Firebase Authentication

公式　`https://firebase.google.com/docs/auth`

認証　Web API キー（`FIREBASE_API_KEY`）

料金　大部分の機能は無料。電話認証や大規模利用は従量課金

メール・パスワード、Google・Apple・GitHub などのソーシャルログイン、電話番号、匿名ログインに対応する。

scripts/paid/51_firebase_auth.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Firebase Authentication: REST API で匿名ユーザーを作成し、ID トークンを検証する。

環境変数 FIREBASE_API_KEY に Web API キー（プロジェクト設定に表示される）を設定する。
Firebase コンソールで「匿名」ログインを有効にしておく必要がある。
実際の Web アプリでは JavaScript SDK (firebase/auth) を使うのが一般的。
"""
import os
import sys

import httpx

key = os.environ.get("FIREBASE_API_KEY")
if not key:
    print("[SKIP] 環境変数 FIREBASE_API_KEY が未設定です。")
    sys.exit(0)

BASE = "https://identitytoolkit.googleapis.com/v1"
with httpx.Client(timeout=20) as client:
    # 匿名サインアップ（メール / パスワードなら email, password を追加する）
    r = client.post(f"{BASE}/accounts:signUp", params={"key": key},
                    json={"returnSecureToken": True})
    r.raise_for_status()
    d = r.json()
    print("localId:", d["localId"], " 有効期限(秒):", d["expiresIn"])

    # ID トークンからユーザー情報を取得
    info = client.post(f"{BASE}/accounts:lookup", params={"key": key},
                       json={"idToken": d["idToken"]}).json()
    print("lookup:", info["users"][0]["localId"], "作成日時:", info["users"][0]["createdAt"])
    # サーバー側でのトークン検証には firebase-admin の auth.verify_id_token() を使う
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 FIREBASE_API_KEY が未設定です。
```

ここではサーバーから REST API を呼んで匿名ユーザーを作成しているが、Web アプリでは JavaScript SDK（`firebase/auth`）の `signInWithPopup` などを使うのが一般的である。Firebase の Web API キーはフロントエンドに置く前提のもので、秘密情報ではない（アクセス制御はセキュリティルールで行う）。フロントエンドから送られてきた ID トークンは、サーバー側で firebase-admin の `verify_id_token()` を使って検証する。

12.2 Auth0

公式 `https://auth0.com/`

認証 Client ID / Secret（`AUTH0_DOMAIN` ほか）

料金 無料プランあり（月間アクティブユーザー数の上限付き）

企業向けの機能（SSO、多要素認証、組織管理、細かな権限管理）が充実した認証基盤である。

scripts/paid/52_auth0.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Auth0: Client Credentials でトークンを取得し、Management API でユーザー一覧を取得する。

環境変数 AUTH0_DOMAIN (例: your-tenant.jp.auth0.com), AUTH0_CLIENT_ID, AUTH0_CLIENT_SECRET
を設定する (Machine to Machine アプリケーションの認証情報)。
"""
import os
import sys

import httpx

domain = os.environ.get("AUTH0_DOMAIN")
cid, secret = os.environ.get("AUTH0_CLIENT_ID"), os.environ.get("AUTH0_CLIENT_SECRET")
if not (domain and cid and secret):
    print("[SKIP] 環境変数 AUTH0_DOMAIN / AUTH0_CLIENT_ID / AUTH0_CLIENT_SECRET が未設定です。")
    sys.exit(0)

with httpx.Client(base_url=f"https://{domain}", timeout=20) as client:
    tok = client.post("/oauth/token", json={
        "grant_type": "client_credentials", "client_id": cid, "client_secret": secret,
        "audience": f"https://{domain}/api/v2/",
    }).json()["access_token"]
    users = client.get("/api/v2/users", params={"per_page": 5, "fields": "email,last_login"},
                       headers={"Authorization": f"Bearer {tok}"}).json()
    for u in users:
        print(u.get("email"), u.get("last_login"))
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 AUTH0_DOMAIN / AUTH0_CLIENT_ID / AUTH0_CLIENT_SECRET が未設定です。
```

ユーザーのログインは Auth0 のホスト画面（Universal Login）で行い、アプリは OpenID Connect で結果を受け取る。スクリプトは、管理用の Management API でユーザー一覧を取得する例である。

12.3 Clerk

公式 https://clerk.com/

認証 シークレットキー（CLERK_SECRET_KEY）

料金 無料プランあり（月間アクティブユーザー数の上限付き）

ログイン・サインアップ・プロフィール管理の UI コンポーネントまで提供する認証サービスで、Next.js などとの統合が容易である。

scripts/paid/53_clerk.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Clerk: Backend API でユーザー一覧を取得する。

環境変数 CLERK_SECRET_KEY (sk_test_... / sk_live_...) を設定する。
ログイン画面はフロントエンドの Clerk コンポーネント (<SignIn /> など) が提供する。
"""
import os
import sys

import httpx

key = os.environ.get("CLERK_SECRET_KEY")
if not key:
    print("[SKIP] 環境変数 CLERK_SECRET_KEY が未設定です。")
    sys.exit(0)

r = httpx.get("https://api.clerk.com/v1/users", params={"limit": 5, "order_by": "-created_at"},
              headers={"Authorization": f"Bearer {key}"}, timeout=20)
r.raise_for_status()
for u in r.json():
    emails = [e["email_address"] for e in u["email_addresses"]]
    print(u["id"], emails, u["created_at"])
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 CLERK_SECRET_KEY が未設定です。
```

12.4 Supabase Auth

公式 https://supabase.com/auth

認証 SUPABASE_URL と anon キー

料金 Supabase のプランに含まれる

Supabase の認証機能である。認証したユーザーの ID を、PostgreSQL の行レベルセキュリティ（RLS）のポリシーで参照できるのが強みである。

scripts/paid/54_supabase_auth.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["supabase"]
# ///
"""Supabase Auth: メールアドレスとパスワードでサインインする。

環境変数 SUPABASE_URL, SUPABASE_ANON_KEY と、テスト用ユーザーの
SUPABASE_TEST_EMAIL, SUPABASE_TEST_PASSWORD を設定する。
"""
import os
import sys

from supabase import create_client

need = ["SUPABASE_URL", "SUPABASE_ANON_KEY", "SUPABASE_TEST_EMAIL", "SUPABASE_TEST_PASSWORD"]
if not all(os.environ.get(k) for k in need):
    print(f"[SKIP] 環境変数 {' / '.join(need)} が未設定です。")
    sys.exit(0)

supabase = create_client(os.environ["SUPABASE_URL"], os.environ["SUPABASE_ANON_KEY"])
res = supabase.auth.sign_in_with_password({"email": os.environ["SUPABASE_TEST_EMAIL"],
                                           "password": os.environ["SUPABASE_TEST_PASSWORD"]})
print("ユーザー ID:", res.user.id)
print("アクセストークン:", res.session.access_token[:20], "...")
supabase.auth.sign_out()
# 新規登録は supabase.auth.sign_up({"email": ..., "password": ...})
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 SUPABASE_URL / SUPABASE_ANON_KEY / SUPABASE_TEST_EMAIL /
↪  SUPABASE_TEST_PASSWORD が未設定です。
```

第 13 章　データベース・BaaS

13.1　Cloud Firestore (Firebase)

公式　https://firebase.google.com/

認証　サービスアカウント鍵（GOOGLE_APPLICATION_CREDENTIALS）

料金　無料枠（Spark プラン）あり。超過分は読み書き回数の従量課金

Google のサーバーレス NoSQL データベースで、ドキュメントをコレクションにまとめて保存する。データの変更をクライアントへリアルタイムに配信できる。

scripts/paid/55_firestore.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["google-cloud-firestore"]
# ///
"""Cloud Firestore: ドキュメントの書き込み・読み込み・クエリ。

GOOGLE_APPLICATION_CREDENTIALS にサービスアカウント鍵 (JSON) のパスを設定する。
"""
import os
import sys

from google.cloud import firestore

if not os.environ.get("GOOGLE_APPLICATION_CREDENTIALS"):
    print("[SKIP] 環境変数 GOOGLE_APPLICATION_CREDENTIALS が未設定です。")
    sys.exit(0)

db = firestore.Client()
todos = db.collection("guide_todos")
todos.document("t1").set({"title": "API ガイドを読む", "done": False,
                          "created": firestore.SERVER_TIMESTAMP})
todos.document("t2").set({"title": "サンプルを動かす", "done": True,
                          "created": firestore.SERVER_TIMESTAMP})

print("t1:", todos.document("t1").get().to_dict())
for doc in todos.where(filter=firestore.FieldFilter("done", "==", False)).stream():
    print("未完了:", doc.id, doc.to_dict()["title"])
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 GOOGLE_APPLICATION_CREDENTIALS が未設定です。
```

SERVER_TIMESTAMP は、クライアントの時計ではなくサーバーの時刻を記録する特殊な値である。where によるクエリは、複数の項目を組み合わせると複合インデックスの作成が必要になることがある。課金は読み書きの回数で決まるため、一覧画面で大量のドキュメントを毎回読むような設計は避ける。

13.2 Supabase

公式 https://supabase.com/

認証 SUPABASE_URL と anon キー

料金 無料プランあり（2 プロジェクトまで、1 週間アクセスがないと一時停止）

PostgreSQL を中心に、認証・ストレージ・リアルタイム配信・Edge Functions をまとめた「オープンソースの Firebase 代替」である。

scripts/paid/56_supabase_db.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["supabase"]
# ///
"""Supabase (PostgreSQL): テーブルへの挿入と検索。

環境変数 SUPABASE_URL, SUPABASE_ANON_KEY を設定する。
事前に SQL エディタで次のテーブルを作成しておく:
    create table todos (id bigint generated always as identity primary key,
                        title text not null, done boolean default false);
"""
import os
import sys

from supabase import create_client

url, key = os.environ.get("SUPABASE_URL"), os.environ.get("SUPABASE_ANON_KEY")
if not (url and key):
    print("[SKIP] 環境変数 SUPABASE_URL / SUPABASE_ANON_KEY が未設定です。")
    sys.exit(0)

supabase = create_client(url, key)
supabase.table("todos").insert({"title": "Supabase を試す"}).execute()
res = (supabase.table("todos").select("id, title, done")
       .eq("done", False).order("id", desc=True).limit(5).execute())
for row in res.data:
    print(row)
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 SUPABASE_URL / SUPABASE_ANON_KEY が未設定です。
```

`select().eq().order().limit()` のメソッドチェーンが、そのまま SQL のクエリに変換される。anon キーはフロントエンドに置く前提のキーなので、テーブルに RLS ポリシーを必ず設定し、誰がどの行を読み書きできるかをデータベース側で制御する。

13.3 Neon

公式 https://neon.com/

認証 接続文字列（DATABASE_URL）

料金 無料プランあり。超過分は計算時間・容量の従量課金

サーバーレスの PostgreSQL で、使っていないときは計算資源を自動で停止して費用を

抑える。データベースを Git のようにブランチできる機能があり、テスト環境の作成が容易である。

scripts/paid/57_neon.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["psycopg[binary]"]
# ///
"""Neon: サーバーレス PostgreSQL に接続して SQL を実行する。

環境変数 DATABASE_URL に Neon の接続文字列 (postgresql://...?sslmode=require) を設定する。
"""
import os
import sys

import psycopg

dsn = os.environ.get("DATABASE_URL")
if not dsn:
    print("[SKIP] 環境変数 DATABASE_URL が未設定です。")
    sys.exit(0)

with psycopg.connect(dsn) as conn, conn.cursor() as cur:
    cur.execute("select version()")
    print(cur.fetchone()[0])
    cur.execute("create table if not exists visits (id serial primary key, at timestamptz default now())")
    cur.execute("insert into visits default values returning id, at")
    print("insert:", cur.fetchone())
    cur.execute("select count(*) from visits")
    print("訪問数:", cur.fetchone()[0])
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 DATABASE_URL が未設定です。
```

通常の PostgreSQL と同じく psycopg などのドライバで接続できる。`psycopg[binary]` はビルド済みのバイナリを含むパッケージで、PostgreSQL のクライアントライブラリを別途インストールしなくてよい。

13.4 Upstash

公式 `https://upstash.com/`

認証 REST URL とトークン

料金 無料枠あり。超過分はリクエスト数の従量課金

HTTP 経由で使える Redis（キー・バリューストア）で、サーバーレス関数やエッジ環境から接続を維持せずに使える。

scripts/paid/58_upstash.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["upstash-redis"]
# ///
```

```python
"""Upstash Redis: HTTP 経由で Redis を使う (サーバーレス環境と相性がよい)。

環境変数 UPSTASH_REDIS_REST_URL, UPSTASH_REDIS_REST_TOKEN を設定する。
"""
import os
import sys

from upstash_redis import Redis

if not os.environ.get("UPSTASH_REDIS_REST_URL"):
    print("[SKIP] 環境変数 UPSTASH_REDIS_REST_URL / UPSTASH_REDIS_REST_TOKEN が未設定です。")
    sys.exit(0)

redis = Redis.from_env()
redis.set("guide:greeting", "こんにちは", ex=60)  # 60 秒で自動削除 (キャッシュ用途)
print("get:", redis.get("guide:greeting"))
print("ページビュー:", redis.incr("guide:pageviews"))

# 簡易レート制限: 1 分あたり 10 回まで
key = "ratelimit:user123"
count = redis.incr(key)
if count == 1:
    redis.expire(key, 60)
print("このユーザーの今分のリクエスト数:", count, "許可" if count <= 10 else "拒否")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 UPSTASH_REDIS_REST_URL / UPSTASH_REDIS_REST_TOKEN が未設定です。
```

`ex=60` は 60 秒後に自動で消えるキーで、API の応答のキャッシュに使う。`incr` と `expire` を組み合わせた例は、固定時間枠方式の簡易レート制限である。Upstash は専用の `upstash-ratelimit` ライブラリも提供している。

13.5 MongoDB Atlas

公式 `https://www.mongodb.com/atlas`

認証 接続文字列（`MONGODB_URI`）

料金 無料の共有クラスタあり。専用クラスタは時間課金

ドキュメント型データベース MongoDB のマネージドサービスである。スキーマを事前に定義せずに JSON 風のデータを保存できる。

`scripts/paid/59_mongodb_atlas.py`

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["pymongo"]
# ///
"""MongoDB Atlas: ドキュメントの挿入と検索。

環境変数 MONGODB_URI に接続文字列 (mongodb+srv://...) を設定する。
"""
import os
import sys
from datetime import datetime, timezone
```

```

from pymongo import MongoClient

uri = os.environ.get("MONGODB_URI")
if not uri:
    print("[SKIP] 環境変数 MONGODB_URI が未設定です。")
    sys.exit(0)

client = MongoClient(uri)
col = client["guide"]["articles"]
col.insert_one({"title": "API 入門", "tags": ["api", "python"],
                "created": datetime.now(timezone.utc)})
for doc in col.find({"tags": "api"}, {"_id": 0}).sort("created", -1).limit(3):
    print(doc)
print("件数:", col.count_documents({}))
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 MONGODB_URI が未設定です。
```

`find({"tags": "api"})` は、配列 `tags` に `"api"` を含むドキュメントを検索する。第 2 引数の `{"_id": 0}` は結果から `_id` を除く指定（射影）である。

第 14 章　メール配信

会員登録の確認、パスワード再設定、注文確認などのトランザクションメールは、自前の SMTP サーバーから送ると迷惑メールに分類されやすい。メール配信サービスを使い、送信ドメインに SPF・DKIM・DMARC を設定するのが現在の標準である。

14.1　Resend

公式　`https://resend.com/`

認証　API キー（`RESEND_API_KEY`）

料金　無料枠あり（月 3,000 通程度）

開発者向けに設計されたシンプルなメール API で、React でメールのテンプレートを書ける React Email と組み合わせて使われることが多い。

scripts/paid/60_resend.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["resend"]
# ///
"""Resend: トランザクションメールを送信する。

環境変数 RESEND_API_KEY と送信先 RESEND_TO を設定する。
独自ドメインを検証するまでは送信元に onboarding@resend.dev を使い、自分宛てにのみ送れる。
"""
import os
import sys

import resend

if not (os.environ.get("RESEND_API_KEY") and os.environ.get("RESEND_TO")):
    print("[SKIP] 環境変数 RESEND_API_KEY / RESEND_TO が未設定です。")
    sys.exit(0)

resend.api_key = os.environ["RESEND_API_KEY"]
email = resend.Emails.send({
    "from": "API Guide <onboarding@resend.dev>",
    "to": [os.environ["RESEND_TO"]],
    "subject": "Resend からのテストメール",
    "html": "<p>これは <strong>Resend API</strong> のテストです。</p>",
})
print("送信 ID:", email["id"])
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 RESEND_API_KEY / RESEND_TO が未設定です。
```

独自ドメインの検証が済むまでは、送信元として `onboarding@resend.dev` を使い、自分のアドレス宛てにだけ送信できる。

14.2 Amazon SES

公式 https://aws.amazon.com/ses/

認証 AWS 認証情報

料金 送信 1,000 通あたり 0.1 ドル程度と非常に安価

AWS のメール送信サービスで、大量配信のコストが最も低い部類に入る。

scripts/paid/61_amazon_ses.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["boto3"]
# ///
"""Amazon SES (v2 API): メールを送信する。

AWS 認証情報 (AWS_ACCESS_KEY_ID など、または AWS_PROFILE) と、検証済みの送信元
SES_FROM・送信先 SES_TO を設定する。サンドボックス中は検証済みアドレスにのみ送れる。
"""
import os
import sys

import boto3

if not (os.environ.get("SES_FROM") and os.environ.get("SES_TO")):
    print("[SKIP] 環境変数 SES_FROM / SES_TO (と AWS 認証情報) が未設定です。")
    sys.exit(0)

ses = boto3.client("sesv2", region_name=os.environ.get("AWS_REGION", "ap-northeast-1"))
res = ses.send_email(
    FromEmailAddress=os.environ["SES_FROM"],
    Destination={"ToAddresses": [os.environ["SES_TO"]]},
    Content={"Simple": {
        "Subject": {"Data": "SES からのテストメール", "Charset": "UTF-8"},
        "Body": {"Text": {"Data": "Amazon SES のテストです。", "Charset": "UTF-8"}},
    }},
)
print("MessageId:", res["MessageId"])
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 SES_FROM / SES_TO (と AWS 認証情報) が未設定です。
```

新規アカウントはサンドボックス状態で、検証済みのアドレスにしか送信できない。本番で使うには、AWS に用途を説明して本番アクセスを申請する。

14.3 SendGrid

公式 https://sendgrid.com/

認証 API キー（SENDGRID_API_KEY）

料金 月額プラン（無料トライアルの条件は要確認）

古くから広く使われているメール配信サービスで、トランザクションメールとマーケティングメールの両方に対応する。

scripts/paid/62_sendgrid.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["sendgrid"]
# ///
"""SendGrid: メールを送信する。

環境変数 SENDGRID_API_KEY, 認証済み送信元 SENDGRID_FROM, 送信先 SENDGRID_TO を設定する。
"""
import os
import sys

from sendgrid import SendGridAPIClient
from sendgrid.helpers.mail import Mail

need = ["SENDGRID_API_KEY", "SENDGRID_FROM", "SENDGRID_TO"]
if not all(os.environ.get(k) for k in need):
    print(f"[SKIP] 環境変数 {' / '.join(need)} が未設定です。")
    sys.exit(0)

message = Mail(from_email=os.environ["SENDGRID_FROM"], to_emails=os.environ["SENDGRID_TO"],
               subject="SendGrid からのテストメール",
               html_content="<p>これは <strong>SendGrid</strong> のテストです。</p>")
res = SendGridAPIClient(os.environ["SENDGRID_API_KEY"]).send(message)
print("status:", res.status_code)  # 202 なら受付成功
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 SENDGRID_API_KEY / SENDGRID_FROM / SENDGRID_TO が未設定です。
```

ステータス 202 は「受け付けた」ことを意味し、相手に届いたことまでは保証しない。到達・開封・バウンスなどの結果は、Event Webhook で受け取る。

14.4 Postmark

公式 https://postmarkapp.com/

認証 サーバートークン

料金 月額プラン（送信数に応じる）

トランザクションメールの到達率と速さを重視したサービスである。マーケティングメールは別の配信経路（Message Stream）に分けることが求められ、送信元の評判が保たれる。

scripts/paid/63_postmark.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Postmark: REST API でメールを送信する。

環境変数 POSTMARK_SERVER_TOKEN, POSTMARK_FROM (署名済み送信元), POSTMARK_TO を設定する。
"""
import os
import sys
```

```

import httpx

need = ["POSTMARK_SERVER_TOKEN", "POSTMARK_FROM", "POSTMARK_TO"]
if not all(os.environ.get(k) for k in need):
    print(f"[SKIP] 環境変数 {' / '.join(need)} が未設定です。")
    sys.exit(0)

r = httpx.post("https://api.postmarkapp.com/email", timeout=20,
               headers={"X-Postmark-Server-Token": os.environ["POSTMARK_SERVER_TOKEN"],
                        "Accept": "application/json"},
               json={"From": os.environ["POSTMARK_FROM"], "To": os.environ["POSTMARK_TO"],
                     "Subject": "Postmark からのテストメール",
                     "TextBody": "Postmark のテストです。", "MessageStream": "outbound"})
print(r.json())
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 POSTMARK_SERVER_TOKEN / POSTMARK_FROM / POSTMARK_TO が未設定です。
```

14.5 Mailgun

公式 https://www.mailgun.com/

認証 API キーとドメイン

料金 月額プラン（少量の無料枠あり）

メールの送信に加えて、受信したメールを解析して Webhook で転送する機能を持つ。

scripts/paid/64_mailgun.py

```
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Mailgun: REST API でメールを送信する。

環境変数 MAILGUN_API_KEY, MAILGUN_DOMAIN, MAILGUN_TO を設定する。
EU リージョンの場合はベース URL を https://api.eu.mailgun.net に変える。
"""
import os
import sys

import httpx

need = ["MAILGUN_API_KEY", "MAILGUN_DOMAIN", "MAILGUN_TO"]
if not all(os.environ.get(k) for k in need):
    print(f"[SKIP] 環境変数 {' / '.join(need)} が未設定です。")
    sys.exit(0)

domain = os.environ["MAILGUN_DOMAIN"]
r = httpx.post(f"https://api.mailgun.net/v3/{domain}/messages", timeout=20,
               auth=("api", os.environ["MAILGUN_API_KEY"]),
               data={"from": f"API Guide <postmaster@{domain}>", "to": os.environ["MAILGUN_TO"],
                     "subject": "Mailgun からのテストメール", "text": "Mailgun のテストです。"})
print(r.status_code, r.json())
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 MAILGUN_API_KEY / MAILGUN_DOMAIN / MAILGUN_TO が未設定です。
```

認証は、ユーザー名 `api`、パスワードに API キーを使う Basic 認証である。EU リージョンのアカウントでは、ベース URL が `api.eu.mailgun.net` になる。

第 15 章　SMS・通話・メッセージング

15.1　Twilio

公式　https://www.twilio.com/

認証　Account SID と Auth Token

料金　送信 1 通ごとの従量課金（トライアルクレジットあり）

SMS・音声通話・ビデオ・WhatsApp などの通信機能を API で提供する。

scripts/paid/65_twilio.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["twilio"]
# ///
"""Twilio: SMS を送信する。

環境変数 TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_FROM (Twilio の電話番号),
TWILIO_TO (送信先, +81 から始まる E.164 形式) を設定する。
トライアル中は検証済みの番号にのみ送信できる。
"""
import os
import sys

from twilio.rest import Client

need = ["TWILIO_ACCOUNT_SID", "TWILIO_AUTH_TOKEN", "TWILIO_FROM", "TWILIO_TO"]
if not all(os.environ.get(k) for k in need):
    print(f"[SKIP] 環境変数 {' / '.join(need)} が未設定です。")
    sys.exit(0)

client = Client(os.environ["TWILIO_ACCOUNT_SID"], os.environ["TWILIO_AUTH_TOKEN"])
msg = client.messages.create(from_=os.environ["TWILIO_FROM"], to=os.environ["TWILIO_TO"],
                             body="【API ガイド】認証コード: 123456")
print("SID:", msg.sid, "status:", msg.status)
# 2 段階認証には専用の Verify API (client.verify.v2.services(...).verifications) が便利
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 TWILIO_ACCOUNT_SID / TWILIO_AUTH_TOKEN / TWILIO_FROM / TWILIO_TO が未設定で
↪  す。
```

電話番号は `+81...` の E.164 形式で指定する。`from_` とアンダースコアが付くのは、`from` が Python の予約語だからである。SMS による 2 段階認証を実装するなら、コードの生成・送信・照合をまとめて行う Verify API が便利である。

15.2 Vonage

公式 https://developer.vonage.com/

認証 API キーとシークレット

料金 送信 1 通ごとの従量課金（トライアルクレジットあり）

旧 Nexmo。SMS・音声・ビデオ API を提供する。

scripts/paid/66_vonage.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Vonage (旧 Nexmo): SMS API でメッセージを送信する。

環境変数 VONAGE_API_KEY, VONAGE_API_SECRET, VONAGE_TO (81 から始まる番号) を設定する。
"""
import os
import sys

import httpx

need = ["VONAGE_API_KEY", "VONAGE_API_SECRET", "VONAGE_TO"]
if not all(os.environ.get(k) for k in need):
    print(f"[SKIP] 環境変数 {' / '.join(need)} が未設定です。")
    sys.exit(0)

r = httpx.post("https://rest.nexmo.com/sms/json", timeout=20, data={
    "api_key": os.environ["VONAGE_API_KEY"], "api_secret": os.environ["VONAGE_API_SECRET"],
    "from": "APIGuide", "to": os.environ["VONAGE_TO"],
    "text": "Vonage SMS API のテストです", "type": "unicode",  # 日本語は unicode を指定
})
for m in r.json()["messages"]:
    print("status:", m["status"], m.get("message-id") or m.get("error-text"))
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 VONAGE_API_KEY / VONAGE_API_SECRET / VONAGE_TO が未設定です。
```

日本語の SMS を送るには `type=unicode` を指定する。指定しないと文字化けする。

15.3 LINE Messaging API

公式 https://developers.line.biz/ja/services/messaging-api/

認証 チャネルアクセストークン

料金 LINE 公式アカウントのプランに依存（無料プランは月 200 通まで）

LINE 公式アカウントを通じて、ユーザーにメッセージを送ったり、チャットボットを作ったりできる。日本では利用者が多いため、予約確認や通知の手段として有力である。

scripts/paid/67_line_messaging.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""LINE Messaging API: ボット情報と今月の送信可能数を確認し、メッセージを送る。

環境変数 LINE_CHANNEL_ACCESS_TOKEN (長期のチャネルアクセストークン) を設定する。
LINE_TO (送信先のユーザー ID) も設定するとプッシュメッセージを送信する。
"""
import os
import sys

import httpx

token = os.environ.get("LINE_CHANNEL_ACCESS_TOKEN")
if not token:
    print("[SKIP] 環境変数 LINE_CHANNEL_ACCESS_TOKEN が未設定です。")
    sys.exit(0)

with httpx.Client(base_url="https://api.line.me/v2/bot", timeout=20,
                  headers={"Authorization": f"Bearer {token}"}) as client:
    info = client.get("/info").json()
    print("ボット:", info["displayName"], info["basicId"])
    quota = client.get("/message/quota").json()
    used = client.get("/message/quota/consumption").json()
    print(f"今月の送信数: {used['totalUsage']} / 上限 {quota.get('value', '無制限')}")

    if to := os.environ.get("LINE_TO"):
        r = client.post("/message/push", json={
            "to": to, "messages": [{"type": "text", "text": "API ガイドからのテスト通知です"}]})
        print("push:", r.status_code)
# ユーザーからのメッセージは Webhook (POST) で受け取り、reply API で返信する
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 LINE_CHANNEL_ACCESS_TOKEN が未設定です。
```

スクリプトはまず読み取り専用の API でボット情報と今月の送信数を確認し、環境変数 `LINE_TO` が設定されている場合にだけプッシュメッセージを送る。プッシュメッセージは送信数の上限にカウントされるが、ユーザーの発言に応答する reply メッセージはカウントされない。

15.4 Slack API

公式 `https://api.slack.com/`

認証 Incoming Webhook URL またはボットトークン

料金 API 自体は無料（ワークスペースのプランに依存）

Slack への通知やボットの作成ができる。最も手軽なのは Incoming Webhook で、URL に JSON を POST するだけでチャンネルに投稿できる。

scripts/paid/68_slack.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Slack: Incoming Webhook でチャンネルに通知を送る。

環境変数 SLACK_WEBHOOK_URL に Incoming Webhook の URL を設定する。
デプロイ完了やエラー発生の通知に便利。Block Kit で見た目を整えられる。
"""
import os
import sys

import httpx

url = os.environ.get("SLACK_WEBHOOK_URL")
if not url:
    print("[SKIP] 環境変数 SLACK_WEBHOOK_URL が未設定です。")
    sys.exit(0)

payload = {
    "text": "デプロイが完了しました",  # 通知用のフォールバックテキスト
    "blocks": [
        {"type": "header", "text": {"type": "plain_text", "text": ":rocket: デプロイ完了"}},
        {"type": "section", "fields": [
            {"type": "mrkdwn", "text": "*環境:*\nproduction"},
            {"type": "mrkdwn", "text": "*バージョン:*\nv1.2.3"},
        ]},
    ],
}
r = httpx.post(url, json=payload, timeout=20)
print(r.status_code, r.text)  # 成功すると 200 ok
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 SLACK_WEBHOOK_URL が未設定です。
```

`blocks` には Block Kit 形式でリッチなレイアウトを記述する（Block Kit Builder で見た目を確認しながら作れる）。`text` は通知のプレビューや、ブロックを表示できない環境で使われる。Webhook の URL 自体が秘密情報なので、リポジトリにコミットしないこと。

第 16 章　地図・位置情報（有償）

16.1　Google Maps Platform

公式　https://developers.google.com/maps

認証　API キー（GOOGLE_MAPS_API_KEY）

料金　SKU（機能）ごとの月間無料枠。超過分はリクエスト数の従量課金

地図表示（Maps JavaScript API）、ジオコーディング、場所の検索（Places API）、経路検索（Routes API）などを提供する。日本国内の地図データとお店の情報の充実度は群を抜いている。

scripts/paid/69_google_maps.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Google Maps Platform: Geocoding API と Places API (New) のテキスト検索。

環境変数 GOOGLE_MAPS_API_KEY を設定する。Cloud Console で各 API を有効化し、
キーには API 制限・リファラ制限をかけておく。
"""
import os
import sys

import httpx

key = os.environ.get("GOOGLE_MAPS_API_KEY")
if not key:
    print("[SKIP] 環境変数 GOOGLE_MAPS_API_KEY が未設定です。")
    sys.exit(0)

with httpx.Client(timeout=20) as client:
    geo = client.get("https://maps.googleapis.com/maps/api/geocode/json",
                     params={"address": "名古屋市瑞穂区瑞穂町山の畑1", "language": "ja",
                             "key": key}).json()
    loc = geo["results"][0]["geometry"]["location"]
    print("ジオコーディング:", geo["results"][0]["formatted_address"], loc)

    places = client.post("https://places.googleapis.com/v1/places:searchText",
                         headers={"X-Goog-Api-Key": key,
                                  "X-Goog-FieldMask": "places.displayName,places.rating,places.
                                      ↪ formattedAddress"},
                         json={"textQuery": "名古屋駅 ひつまぶし", "languageCode": "ja",
                               "maxResultCount": 3}).json()
    for p in places.get("places", []):
        print(f"  {p['displayName']['text']} ★{p.get('rating')}  {p['formattedAddress']}")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 GOOGLE_MAPS_API_KEY が未設定です。
```

Places API (New) では、ヘッダー `X-Goog-FieldMask` で返す項目を必ず指定する。指定した項目によって料金の区分（SKU）が変わるため、必要な項目だけを指定するのが節約の基本である。API キーには、使う API とリファラ（呼び出し元のドメイン）の制限を必ずかけておく。

16.2 Mapbox

公式 `https://www.mapbox.com/`

認証 アクセストークン（`MAPBOX_ACCESS_TOKEN`）

料金 無料枠あり。超過分は地図の読み込み回数などの従量課金

デザインを細かく調整できるベクトル地図と、ジオコーディング・経路検索・静的地図画像の API を提供する。

scripts/paid/70_mapbox.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Mapbox: Geocoding API (v6) と Static Images API。

環境変数 MAPBOX_ACCESS_TOKEN (pk.... 公開トークン) を設定する。
"""
import os
import sys
from pathlib import Path

import httpx

token = os.environ.get("MAPBOX_ACCESS_TOKEN")
if not token:
    print("[SKIP] 環境変数 MAPBOX_ACCESS_TOKEN が未設定です。")
    sys.exit(0)

with httpx.Client(timeout=20) as client:
    g = client.get("https://api.mapbox.com/search/geocode/v6/forward",
                   params={"q": "名古屋城", "language": "ja", "limit": 1,
                           "access_token": token}).json()
    f = g["features"][0]
    lon, lat = f["geometry"]["coordinates"]
    print("名古屋城:", f["properties"]["full_address"], lat, lon)

    url = (f"https://api.mapbox.com/styles/v1/mapbox/streets-v12/static/"
           f"pin-s+e55e5e({lon},{lat})/{lon},{lat},14/600x400")
    img = client.get(url, params={"access_token": token})
    out = Path(__file__).resolve().parents[2] / "outputs" / "img" / "mapbox_static.png"
    out.write_bytes(img.content)
    print("静的地図を保存:", out.name)
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 MAPBOX_ACCESS_TOKEN が未設定です。
```

Static Images API は、URL にピンの位置・中心・ズーム・サイズを書くだけで地図画像を返す。メールや PDF のように JavaScript が動かない場所に地図を載せたいときに便利である。

16.3 HERE

公式 https://www.here.com/developer

認証 API キー（HERE_API_KEY）

料金 無料枠あり。超過分は従量課金

カーナビ向けの地図データで実績のある HERE の、ジオコーディング・経路検索・交通情報の API である。

scripts/paid/71_here.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""HERE: Geocoding & Search API と Routing API (v8)。

環境変数 HERE_API_KEY を設定する。
"""
import os
import sys

import httpx

key = os.environ.get("HERE_API_KEY")
if not key:
    print("[SKIP] 環境変数 HERE_API_KEY が未設定です。")
    sys.exit(0)

with httpx.Client(timeout=20) as client:
    g = client.get("https://geocode.search.hereapi.com/v1/geocode",
                   params={"q": "名古屋駅", "lang": "ja", "apiKey": key}).json()
    pos = g["items"][0]["position"]
    print("名古屋駅:", pos)

    r = client.get("https://router.hereapi.com/v8/routes", params={
        "transportMode": "car", "origin": f"{pos['lat']},{pos['lng']}",
        "destination": "35.1847,136.8995", "return": "summary", "apiKey": key}).json()
    s = r["routes"][0]["sections"][0]["summary"]
    print(f"車で名古屋城まで: {s['length'] / 1000:.1f} km, 約 {s['duration'] // 60} 分")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 HERE_API_KEY が未設定です。
```

第 17 章　ストレージ・画像処理

17.1　Amazon S3

公式　https://aws.amazon.com/s3/

認証　AWS 認証情報

料金　保存容量・リクエスト数・転送量の従量課金

最も広く使われているオブジェクトストレージで、ユーザーがアップロードした画像やバックアップの保存先として定番である。

scripts/paid/72_amazon_s3.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["boto3"]
# ///
"""Amazon S3: ファイルのアップロードと署名付き URL の発行。

AWS 認証情報と、環境変数 S3_BUCKET (バケット名) を設定する。
署名付き URL を使うと、バケットを公開せずに期限付きでダウンロードさせられる。
"""
import os
import sys

import boto3

bucket = os.environ.get("S3_BUCKET")
if not bucket:
    print("[SKIP] 環境変数 S3_BUCKET (と AWS 認証情報) が未設定です。")
    sys.exit(0)

s3 = boto3.client("s3")
s3.put_object(Bucket=bucket, Key="guide/hello.txt", Body="こんにちは S3".encode(),
              ContentType="text/plain; charset=utf-8")
url = s3.generate_presigned_url("get_object", Params={"Bucket": bucket, "Key": "guide/hello.txt"},
                                ExpiresIn=600)
print("署名付き URL (10 分有効):", url[:80], "...")
for obj in s3.list_objects_v2(Bucket=bucket, Prefix="guide/").get("Contents", []):
    print(obj["Key"], obj["Size"], "bytes")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 S3_BUCKET (と AWS 認証情報) が未設定です。
```

署名付き URL（presigned URL）を発行すると、バケットを公開しないまま、期限付きでファイルをダウンロードさせられる。同様に `put_object` 用の署名付き URL を発行すれば、ブラウザから S3 へ直接アップロードさせることもでき、大きなファイルが自前のサーバーを経由しなくて済む。

17.2 Cloudflare R2

公式 https://www.cloudflare.com/developer-platform/r2/

認証 アクセスキー（R2_ACCESS_KEY_ID ほか）

料金 無料枠あり（10 GB 程度）。データ転送（エグレス）料金は無料

S3 互換のオブジェクトストレージで、ダウンロードの転送料金がかからないのが最大の特徴である。

scripts/paid/73_cloudflare_r2.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["boto3"]
# ///
"""Cloudflare R2: S3 互換 API でアップロードする（エンドポイントを変えるだけ）。

環境変数 R2_ACCOUNT_ID, R2_ACCESS_KEY_ID, R2_SECRET_ACCESS_KEY, R2_BUCKET を設定する。
R2 はデータ転送（エグレス）料金が無料なのが特徴。
"""
import os
import sys

import boto3

need = ["R2_ACCOUNT_ID", "R2_ACCESS_KEY_ID", "R2_SECRET_ACCESS_KEY", "R2_BUCKET"]
if not all(os.environ.get(k) for k in need):
    print(f"[SKIP] 環境変数 {' / '.join(need)} が未設定です。")
    sys.exit(0)

r2 = boto3.client(
    "s3",
    endpoint_url=f"https://{os.environ['R2_ACCOUNT_ID']}.r2.cloudflarestorage.com",
    aws_access_key_id=os.environ["R2_ACCESS_KEY_ID"],
    aws_secret_access_key=os.environ["R2_SECRET_ACCESS_KEY"],
    region_name="auto",
)
bucket = os.environ["R2_BUCKET"]
r2.put_object(Bucket=bucket, Key="guide/hello.json", Body=b'{"hello": "R2"}',
              ContentType="application/json")
print(r2.get_object(Bucket=bucket, Key="guide/hello.json")["Body"].read().decode())
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 R2_ACCOUNT_ID / R2_ACCESS_KEY_ID / R2_SECRET_ACCESS_KEY / R2_BUCKET が未設
↪  定です。
```

S3 用のライブラリ boto3 が、`endpoint_url` を R2 のものに変えるだけでそのまま使える。動画や画像のように転送量の多いファイルの配信では、S3 より大幅に安くなることがある。

17.3 Cloudinary

公式 https://cloudinary.com/

認証 CLOUDINARY_URL

料金 無料枠あり（クレジット制）

画像・動画のアップロード、変換（リサイズ・切り抜き・形式変換・画質調整）、CDN 配信をまとめて提供する。

scripts/paid/74_cloudinary.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["cloudinary"]
# ///
"""Cloudinary: 画像をアップロードし、URL で変換（リサイズ・形式変換）する。

環境変数 CLOUDINARY_URL (cloudinary://<key>:<secret>@<cloud_name>) を設定する。
"""
import os
import sys

import cloudinary
import cloudinary.uploader
from cloudinary import CloudinaryImage

if not os.environ.get("CLOUDINARY_URL"):
    print("[SKIP] 環境変数 CLOUDINARY_URL が未設定です。")
    sys.exit(0)

cloudinary.config(secure=True)
res = cloudinary.uploader.upload("https://picsum.photos/seed/kklab/1200/800",
                                 public_id="guide/sample", overwrite=True)
print("アップロード:", res["secure_url"], f"{res['width']}x{res['height']}")

# 幅 400px・自動フォーマット (WebP/AVIF)・自動画質 の URL を生成
print("変換 URL:", CloudinaryImage("guide/sample").build_url(
    width=400, crop="fill", fetch_format="auto", quality="auto"))
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 CLOUDINARY_URL が未設定です。
```

変換は URL のパラメータで指定する。`fetch_format="auto"` はブラウザが対応していれば WebP や AVIF で配信する指定で、画像の転送量を大きく減らせる。

17.4 imgix

公式 https://www.imgix.com/

認証 ソースのドメイン（署名を使う場合はトークン）

料金 月額プラン（トライアルあり）

S3 などに置いた元画像を、URL のパラメータに応じてリアルタイムに変換して CDN で配信する。URL の組み立てはローカルで完結するため、ここでは imgix 公式のデモ用

ドメインを使って実行した。

scripts/paid/75_imgix.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["imgix", "httpx"]
# ///
"""imgix: URL パラメータで画像をリアルタイムに変換する。

URL の組み立てはローカルで完結する。ここでは imgix 公式のデモ用ドメインを使う。
自分の画像を使うには imgix でソース (S3 など) を登録し、IMGIX_DOMAIN を設定する。
"""
import os
from pathlib import Path

import httpx
from imgix import UrlBuilder

domain = os.environ.get("IMGIX_DOMAIN", "assets.imgix.net")
builder = UrlBuilder(domain, include_library_param=False,
                     sign_key=os.environ.get("IMGIX_SECURE_TOKEN"))  # 署名付き URL (任意)

url = builder.create_url("unsplash/bridge.jpg", {"w": 480, "h": 320, "fit": "crop",
                                                 "sat": -100, "auto": "format,compress"})
print("変換 URL:", url)

# レスポンシブ画像用の srcset も自動生成できる
srcset = builder.create_srcset("unsplash/bridge.jpg", {"ar": "3:2", "fit": "crop"},
                               widths=[320, 640, 960])
print("srcset:", srcset.replace(",\n", ", "))

r = httpx.get(url, timeout=30)
r.raise_for_status()
out = Path(__file__).resolve().parents[2] / "outputs" / "img" / "imgix.jpg"
out.write_bytes(r.content)
print(f"取得: {out.name} ({len(r.content):,} bytes, {r.headers['content-type']})")
```

実行結果　`uv run scripts/paid/75_imgix.py`

```
変換 URL: https://assets.imgix.net/unsplash/bridge.jpg?auto=format%2Ccompress&fit=crop&h=3
↪  20&sat=-100&w=480
srcset: https://assets.imgix.net/unsplash/bridge.jpg?ar=3%3A2&fit=crop&w=320 320w,
↪  https://assets.imgix.net/unsplash/bridge.jpg?ar=3%3A2&fit=crop&w=640 640w,
↪  https://assets.imgix.net/unsplash/bridge.jpg?ar=3%3A2&fit=crop&w=960 960w
取得: imgix.jpg (19,300 bytes, image/jpeg)
```



`w`・`h`・`fit=crop` で 480×320 に切り抜き、`sat=-100` で彩度を下げてモノクロにした。`auto=format,compress` は、ブラウザに応じた最適な形式と圧縮率を自動で選ぶ指定で

ある。`create_srcset` は、レスポンシブ画像用の `srcset` 属性の値を生成する。これを `<img srcset="...">` に入れると、ブラウザが画面幅に合った大きさの画像を選んで読み込む。

17.5 UploadThing

公式 `https://uploadthing.com/`

認証 API キー（`UPLOADTHING_SECRET`）

料金 無料枠あり。月額プラン

Next.js などの TypeScript アプリ向けに、ファイルアップロードの機能一式（UI・サーバー処理・保存）を提供する。公式の Python SDK はないため、ここでは REST API でファイル一覧を取得している。

scripts/paid/76_uploadthing.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""UploadThing: REST API でアップロード済みファイルの一覧を取得する。

環境変数 UPLOADTHING_SECRET (sk_live_... の API キー) を設定する。
アップロード処理そのものは Next.js などのフロントエンド SDK を使うのが一般的。
"""
import os
import sys

import httpx

key = os.environ.get("UPLOADTHING_SECRET")
if not key:
    print("[SKIP] 環境変数 UPLOADTHING_SECRET が未設定です。")
    sys.exit(0)

r = httpx.post("https://api.uploadthing.com/v6/listFiles", timeout=20,
               headers={"X-Uploadthing-Api-Key": key}, json={"limit": 5})
r.raise_for_status()
for f in r.json()["files"]:
    print(f["key"], f["name"], f["status"])
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 UPLOADTHING_SECRET が未設定です。
```

第 18 章　検索

データベースの LIKE 検索では、表記ゆれやタイプミスへの対応、関連度による並べ替え、入力途中のサジェストが難しい。専用の検索エンジンを使うと、これらを高速に実現できる。

18.1　Algolia

公式　https://www.algolia.com/

認証　Application ID と API キー

料金　無料枠あり。超過分は検索リクエスト数の従量課金

ホスト型の全文検索サービスで、入力のたびに結果を更新するような高速な検索 UI を作れる。

scripts/paid/77_algolia.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["algoliasearch>=4"]
# ///
"""Algolia: インデックスにレコードを登録して全文検索する。

環境変数 ALGOLIA_APP_ID と ALGOLIA_API_KEY (Write API Key) を設定する。
ブラウザからの検索には Search-Only API Key を使う。
"""
import os
import sys

from algoliasearch.search.client import SearchClientSync

app_id, key = os.environ.get("ALGOLIA_APP_ID"), os.environ.get("ALGOLIA_API_KEY")
if not (app_id and key):
    print("[SKIP] 環境変数 ALGOLIA_APP_ID / ALGOLIA_API_KEY が未設定です。")
    sys.exit(0)

client = SearchClientSync(app_id, key)
records = [
    {"objectID": "1", "title": "Python で始める Web API", "tags": ["python", "api"]},
    {"objectID": "2", "title": "JavaScript の非同期処理", "tags": ["javascript"]},
    {"objectID": "3", "title": "REST API 設計入門", "tags": ["api", "design"]},
]
res = client.save_objects(index_name="guide_articles", objects=records, wait_for_tasks=True)
hits = client.search_single_index(index_name="guide_articles",
                                  search_params={"query": "API", "hitsPerPage": 5})
for h in hits.hits:
    print(h.object_id, h.to_dict().get("title"))
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 ALGOLIA_APP_ID / ALGOLIA_API_KEY が未設定です。
```

データの登録には書き込み用の Write API Key を使い、ブラウザからの検索には検索専用の Search-Only API Key を使う。後者はフロントエンドに置いてよいが、前者は絶対に置かない。InstantSearch というフロントエンド用ライブラリで、検索ボックスや絞り込み UI を簡単に組み立てられる。

18.2 Meilisearch

公式 https://www.meilisearch.com/

認証 URL とマスターキー / API キー

料金 OSS 版は無料（セルフホスト）。Cloud は月額プラン

Rust で書かれた OSS の検索エンジンで、設定なしでもタイプミスに強い検索ができる。Docker で手元に立てて試せるのが魅力である。

scripts/paid/78_meilisearch.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["meilisearch"]
# ///
"""Meilisearch: ドキュメントを登録して検索する (タイプミスにも強い)。

Meilisearch Cloud なら MEILI_URL と MEILI_API_KEY を設定する。
ローカルでは `docker run -p 7700:7700 getmeili/meilisearch` で起動できる (OSS 版は無料)。
"""
import os
import sys

import meilisearch

url = os.environ.get("MEILI_URL")
if not url:
    print("[SKIP] 環境変数 MEILI_URL (と MEILI_API_KEY) が未設定です。")
    sys.exit(0)

client = meilisearch.Client(url, os.environ.get("MEILI_API_KEY"))
index = client.index("movies")
task = index.add_documents([
    {"id": 1, "title": "千と千尋の神隠し", "year": 2001},
    {"id": 2, "title": "となりのトトロ", "year": 1988},
    {"id": 3, "title": "もののけ姫", "year": 1997},
])
client.wait_for_task(task.task_uid)
for hit in index.search("トトロ")["hits"]:
    print(hit)
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 MEILI_URL (と MEILI_API_KEY) が未設定です。
```

ドキュメントの追加は非同期のタスクとして処理されるため、`wait_for_task` で完了を待ってから検索している。日本語の検索にも対応している。

18.3 Typesense

公式 `https://typesense.org/`

認証 ホスト名と API キー

料金 OSS 版は無料（セルフホスト）。Cloud はクラスタの時間課金

Meilisearch と同様の OSS 検索エンジンで、スキーマを事前に定義する点が異なる。

scripts/paid/79_typesense.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Typesense: コレクションを作成し、ドキュメントを登録・検索する (REST API)。

環境変数 TYPESENSE_HOST (例: xxx.a1.typesense.net) と TYPESENSE_API_KEY を設定する。
"""
import os
import sys

import httpx

host, key = os.environ.get("TYPESENSE_HOST"), os.environ.get("TYPESENSE_API_KEY")
if not (host and key):
    print("[SKIP] 環境変数 TYPESENSE_HOST / TYPESENSE_API_KEY が未設定です。")
    sys.exit(0)

with httpx.Client(base_url=f"https://{host}", timeout=20,
                  headers={"X-TYPESENSE-API-KEY": key}) as client:
    client.post("/collections", json={"name": "books", "fields": [
        {"name": "title", "type": "string", "locale": "ja"},
        {"name": "year", "type": "int32"}]})  # 既に存在する場合は 409 が返るだけ
    client.post("/collections/books/documents/import", params={"action": "upsert"},
                content='{"id":"1","title":"国富論","year":1776}\n'
                        '{"id":"2","title":"雇用・利子および貨幣の一般理論","year":1936}')
    res = client.get("/collections/books/documents/search",
                     params={"q": "国富", "query_by": "title"}).json()
    for h in res["hits"]:
        print(h["document"])
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 TYPESENSE_HOST / TYPESENSE_API_KEY が未設定です。
```

フィールドに `"locale": "ja"` を指定すると日本語用の分かち書きが使われる。一括登録は JSON Lines 形式（1 行 1 ドキュメント）で送る。

第 19 章　翻訳・音声

19.1　DeepL API

公式　https://www.deepl.com/pro-api

認証　認証キー（DEEPL_AUTH_KEY）

料金　Free 版は月 50 万文字まで無料。Pro 版は文字数の従量課金

自然な訳文に定評のある機械翻訳 API である。

scripts/paid/80_deepl.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["deepl"]
# ///
"""DeepL API: 高品質な機械翻訳。

環境変数 DEEPL_AUTH_KEY を設定する（Free 版のキーは末尾が :fx）。
"""
import os
import sys

import deepl

key = os.environ.get("DEEPL_AUTH_KEY")
if not key:
    print("[SKIP] 環境変数 DEEPL_AUTH_KEY が未設定です。")
    sys.exit(0)

translator = deepl.DeepLClient(key)
for text in ["API キーはサーバー側で管理してください。", "レート制限に達しました。"]:
    result = translator.translate_text(text, target_lang="EN-US")
    print(f"{text}\n  -> {result.text}")
usage = translator.get_usage()
print(f"今月の使用文字数: {usage.character.count:,} / {usage.character.limit:,}")
```

未実行（認証情報が未設定）

```
[SKIP] 環境変数 DEEPL_AUTH_KEY が未設定です。
```

Free 版のキーは末尾が `:fx` で、SDK がキーを見て接続先（Free / Pro）を自動で切り替える。`get_usage()` で今月の使用文字数がわかるので、上限の手前で警告を出すといった運用ができる。用語集（glossary）を登録すれば、専門用語の訳語を統一できる。

19.2 Google Cloud Translation

公式 https://cloud.google.com/translate

認証 API キー

料金 月 50 万文字まで無料。超過分は文字数の従量課金

Google の機械翻訳 API で、対応言語の多さが特徴である。ここではシンプルな Basic 版（v2）を使い、翻訳元の言語を自動判定させている。

scripts/paid/81_google_translate.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Google Cloud Translation API (Basic v2): API キーで翻訳する。

環境変数 GOOGLE_TRANSLATE_API_KEY を設定し、Cloud Console で API を有効化しておく。
"""
import os
import sys

import httpx

key = os.environ.get("GOOGLE_TRANSLATE_API_KEY")
if not key:
    print("[SKIP] 環境変数 GOOGLE_TRANSLATE_API_KEY が未設定です。")
    sys.exit(0)

r = httpx.post("https://translation.googleapis.com/language/translate/v2", params={"key": key},
               json={"q": ["おはようございます", "API の利用ガイド"], "target": "en",
                     "format": "text"}, timeout=20)
r.raise_for_status()
for t in r.json()["data"]["translations"]:
    print(t["detectedSourceLanguage"], "->", t["translatedText"])
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 GOOGLE_TRANSLATE_API_KEY が未設定です。
```

19.3 ElevenLabs

公式 https://elevenlabs.io/

認証 API キー（ELEVENLABS_API_KEY）

料金 無料枠あり（少量）。月額プラン

自然な抑揚の音声合成で知られ、日本語を含む多言語に対応する。

scripts/paid/82_elevenlabs.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["elevenlabs"]
```

```python
# ///
"""ElevenLabs: テキストを音声 (MP3) に変換する。

環境変数 ELEVENLABS_API_KEY を設定する。voice_id は Voice Library で確認できる。
"""
import os
import sys
from pathlib import Path

from elevenlabs.client import ElevenLabs

if not os.environ.get("ELEVENLABS_API_KEY"):
    print("[SKIP] 環境変数 ELEVENLABS_API_KEY が未設定です。")
    sys.exit(0)

client = ElevenLabs()  # ELEVENLABS_API_KEY を自動で読み込む
audio = client.text_to_speech.convert(
    voice_id=os.environ.get("ELEVENLABS_VOICE_ID", "JBFqnCBsd6RMkjVDRZzb"),
    model_id="eleven_multilingual_v2",
    text="こんにちは。これは ElevenLabs による音声合成のテストです。",
    output_format="mp3_44100_128",
)
out = Path(__file__).resolve().parents[2] / "outputs" / "elevenlabs.mp3"
out.write_bytes(b"".join(audio))
print("保存:", out.name, f"{out.stat().st_size:,} bytes")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 ELEVENLABS_API_KEY が未設定です。
```

`convert` は音声データを少しずつ返すイテレータなので、`b"".join()` でまとめてファイルに保存している。Web アプリでは、このストリームをそのままレスポンスとして返せば、生成の途中から再生を始められる。

19.4 Google Cloud Speech-to-Text

公式 `https://cloud.google.com/speech-to-text`

認証 API キー

料金 月 60 分まで無料。超過分は音声の長さの従量課金

音声を文字に起こす API である。ここでは Google が公開しているサンプル音声（英語）を使う。

scripts/paid/83_google_speech.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Google Cloud Speech-to-Text (v1): 公開サンプル音声を文字起こしする。

環境変数 GOOGLE_SPEECH_API_KEY を設定し、Cloud Console で API を有効化しておく。
"""
import os
import sys
```

```

import httpx

key = os.environ.get("GOOGLE_SPEECH_API_KEY")
if not key:
    print("[SKIP] 環境変数 GOOGLE_SPEECH_API_KEY が未設定です。")
    sys.exit(0)

r = httpx.post("https://speech.googleapis.com/v1/speech:recognize", params={"key": key},
               timeout=60, json={
                   "config": {"encoding": "FLAC", "sampleRateHertz": 16000,
                              "languageCode": "en-US"},
                   "audio": {"uri": "gs://cloud-samples-tests/speech/brooklyn.flac"},
               })
r.raise_for_status()
for res in r.json()["results"]:
    alt = res["alternatives"][0]
    print(f"{alt['transcript']}  (信頼度 {alt['confidence']:.2f})")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 GOOGLE_SPEECH_API_KEY が未設定です。
```

1 分を超える音声には `longrunningrecognize`（非同期処理）を使う。日本語の場合は `languageCode` を `ja-JP` にする。

第 20 章　監視・分析

20.1　Sentry

公式　https://sentry.io/

認証　DSN（SENTRY_DSN）

料金　個人向けの無料プランあり。イベント数に応じた月額プラン

アプリで発生したエラーを収集し、スタックトレースや発生状況とともに通知するエラー監視サービスである。

scripts/paid/84_sentry.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["sentry-sdk"]
# ///
"""Sentry: 例外とメッセージをエラー監視サービスに送る。

環境変数 SENTRY_DSN を設定する。Flask / FastAPI / Django なら init するだけで
未処理の例外が自動的に送信される。
"""
import os
import sys

import sentry_sdk

dsn = os.environ.get("SENTRY_DSN")
if not dsn:
    print("[SKIP] 環境変数 SENTRY_DSN が未設定です。")
    sys.exit(0)

sentry_sdk.init(dsn=dsn, environment="guide", traces_sample_rate=1.0, send_default_pii=False)
sentry_sdk.set_tag("feature", "api-guide")
sentry_sdk.capture_message("API ガイドからのテストメッセージ")
try:
    1 / 0
except ZeroDivisionError as e:
    event_id = sentry_sdk.capture_exception(e)
    print("送信したイベント ID:", event_id)
sentry_sdk.flush()
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 SENTRY_DSN が未設定です。
```

Flask・FastAPI・Django などのフレームワークでは `sentry_sdk.init()` を呼ぶだけで、未処理の例外が自動的に送信される。`send_default_pii=False` は、IP アドレスなどの個人情報を送らない設定である。DSN は送信専用の識別子なので、フロントエンドの JavaScript に置いてもよい。

20.2 PostHog

公式 https://posthog.com/

認証 プロジェクト API キー（POSTHOG_API_KEY）

料金 無料枠が大きい（月 100 万イベント程度）。超過分は従量課金

プロダクト分析、セッション録画、機能フラグ、A/B テストなどをまとめて提供する。OSS 版のセルフホストも可能である。

scripts/paid/85_posthog.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["posthog"]
# ///
"""PostHog: プロダクト分析イベントの送信と機能フラグの判定。

環境変数 POSTHOG_API_KEY（プロジェクト API キー, phc_...）と POSTHOG_HOST
（例: https://us.i.posthog.com）を設定する。
"""
import os
import sys

from posthog import Posthog

key = os.environ.get("POSTHOG_API_KEY")
if not key:
    print("[SKIP] 環境変数 POSTHOG_API_KEY が未設定です。")
    sys.exit(0)

ph = Posthog(key, host=os.environ.get("POSTHOG_HOST", "https://us.i.posthog.com"))
ph.capture("guide_sample_run", distinct_id="user-123", properties={"plan": "free"})
print("機能フラグ new-ui:", ph.feature_enabled("new-ui", "user-123"))
ph.shutdown()  # バッファ内のイベントを送信してから終了
print("イベントを送信しました")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 POSTHOG_API_KEY が未設定です。
```

SDK はイベントをバッファにためて一括送信するため、スクリプトの最後に `shutdown()` を呼んで送信を完了させている。機能フラグを使うと、新機能を一部のユーザーにだけ公開して反応を見る、といった段階的なリリースができる。

20.3 Mixpanel

公式 https://mixpanel.com/

認証 プロジェクトトークン（MIXPANEL_TOKEN）

料金 無料枠あり。イベント数に応じた月額プラン

ユーザーの行動（イベント）を記録し、ファネル分析や継続率の分析を行う。

scripts/paid/86_mixpanel.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Mixpanel: Ingestion API でユーザー行動イベントを送信する。

環境変数 MIXPANEL_TOKEN (プロジェクトトークン) を設定する。
(公式 Python SDK `mixpanel` もあるが、ここでは HTTP API を直接呼ぶ)
"""
import os
import sys
import time
import uuid

import httpx

token = os.environ.get("MIXPANEL_TOKEN")
if not token:
    print("[SKIP] 環境変数 MIXPANEL_TOKEN が未設定です。")
    sys.exit(0)

event = {"event": "Signed Up", "properties": {
    "token": token, "distinct_id": "user-123", "time": int(time.time()),
    "$insert_id": str(uuid.uuid4()),  # 重複排除用の ID
    "source": "api-guide", "plan": "free"}}
r = httpx.post("https://api.mixpanel.com/track", json=[event], timeout=20,
               headers={"Accept": "text/plain"})
print("track:", r.status_code, r.text)  # "1" なら受付成功
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 MIXPANEL_TOKEN が未設定です。
```

公式の Python SDK（`mixpanel`）もあるが、執筆時点では Python 3.14 の環境で依存パッケージのビルドに失敗したため、HTTP の Ingestion API を直接呼ぶ形にした。`$insert_id` を付けておくと、再送などで同じイベントが重複して記録されるのを防げる。

20.4 Datadog

公式 `https://www.datadoghq.com/`

認証 API キー（`DD_API_KEY`）

料金 機能ごと・ホストごとの月額課金（無料プランは制限付き）

サーバー・コンテナ・アプリケーションのメトリクス、ログ、トレースを統合して監視するサービスである。

scripts/paid/87_datadog.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Datadog: カスタムメトリクスとログを送信する (REST API)。

```

```
環境変数 DD_API_KEY と DD_SITE (例: datadoghq.com, ap1.datadoghq.com) を設定する。
"""
import os
import sys
import time

import httpx

key = os.environ.get("DD_API_KEY")
if not key:
    print("[SKIP] 環境変数 DD_API_KEY が未設定です。")
    sys.exit(0)

site = os.environ.get("DD_SITE", "datadoghq.com")
headers = {"DD-API-KEY": key}
with httpx.Client(timeout=20, headers=headers) as client:
    r = client.post(f"https://api.{site}/api/v2/series", json={"series": [{
        "metric": "guide.signups", "type": 1,  # 1 = count
        "points": [{"timestamp": int(time.time()), "value": 3}],
        "tags": ["env:guide"]}]})
    print("metrics:", r.status_code, r.json())
    r = client.post(f"https://http-intake.logs.{site}/api/v2/logs", json=[{
        "message": "API ガイドからのテストログ", "service": "api-guide", "ddsource": "python"}])
    print("logs:", r.status_code)
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 DD_API_KEY が未設定です。
```

通常はサーバーに Datadog Agent を入れて自動で収集するが、ここでは HTTP API でカスタムメトリクスとログを直接送信している。アカウントのリージョン（`DD_SITE`）によって送信先のドメインが異なり、日本リージョンは `ap1.datadoghq.com` である。

第 21 章　リアルタイム通信

チャット、通知、共同編集、ライブ配信のように、サーバーからブラウザへ即座にデータを届ける機能は、WebSocket などの常時接続を管理する必要があり、自前で大規模に運用するのは難しい。

21.1　Pusher Channels

公式　`https://pusher.com/channels`

認証　App ID / Key / Secret / Cluster

料金　無料プランあり（同時接続数の上限付き）。月額プラン

サーバーからチャンネルにイベントを送ると、そのチャンネルを購読しているすべてのブラウザに即座に届く。

scripts/paid/88_pusher.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["pusher"]
# ///
"""Pusher Channels: サーバーからブラウザへリアルタイムにイベントを送る。

環境変数 PUSHER_APP_ID, PUSHER_KEY, PUSHER_SECRET, PUSHER_CLUSTER (例: ap3) を設定する。
ブラウザ側は pusher-js で同じチャンネルを subscribe しておく。
"""
import os
import sys

import pusher

need = ["PUSHER_APP_ID", "PUSHER_KEY", "PUSHER_SECRET", "PUSHER_CLUSTER"]
if not all(os.environ.get(k) for k in need):
    print(f"[SKIP] 環境変数 {' / '.join(need)} が未設定です。")
    sys.exit(0)

client = pusher.Pusher(app_id=os.environ["PUSHER_APP_ID"], key=os.environ["PUSHER_KEY"],
                       secret=os.environ["PUSHER_SECRET"], cluster=os.environ["PUSHER_CLUSTER"],
                       ssl=True)
client.trigger("orders", "order-created", {"id": 42, "total": 1500})
print("イベント order-created を送信しました")
# ブラウザ側:
#   const ch = new Pusher(KEY, {cluster: CLUSTER}).subscribe('orders');
#   ch.bind('order-created', data => console.log(data));
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 PUSHER_APP_ID / PUSHER_KEY / PUSHER_SECRET / PUSHER_CLUSTER が未設定です。
```

サーバーは `trigger` でイベントを送るだけで、WebSocket の接続管理はすべて Pusher

が担う。ブラウザ側のコードはスクリプト末尾のコメントのとおり数行で済む。

21.2 Ably

公式 https://ably.com/

認証 API キー（ABLY_API_KEY）

料金 無料プランあり。メッセージ数・接続時間の従量課金

Pub/Sub 型のリアルタイムメッセージング基盤で、メッセージの順序保証や履歴の保存に対応する。

scripts/paid/89_ably.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Ably: REST API でチャンネルにメッセージを publish し、履歴を取得する。

環境変数 ABLY_API_KEY (appId.keyId:secret 形式) を設定する。
"""
import os
import sys

import httpx

key = os.environ.get("ABLY_API_KEY")
if not key:
    print("[SKIP] 環境変数 ABLY_API_KEY が未設定です。")
    sys.exit(0)

user, secret = key.split(":", 1)
with httpx.Client(base_url="https://rest.ably.io", auth=(user, secret), timeout=20) as client:
    r = client.post("/channels/guide-chat/messages",
                    json={"name": "message", "data": "こんにちは Ably"})
    print("publish:", r.status_code)
    for m in client.get("/channels/guide-chat/messages", params={"limit": 3}).json():
        print("履歴:", m["name"], m["data"])
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 ABLY_API_KEY が未設定です。
```

API キーは `appId.keyId:secret` の形式で、コロンの前後をユーザー名とパスワードとする Basic 認証に使える。ブラウザには API キーそのものではなく、サーバーで発行した期限付きのトークンを渡す。

21.3 LiveKit

公式 https://livekit.io/

認証 API キーとシークレット

料金 OSS 版は無料（セルフホスト）。Cloud は無料枠あり、超過分は従量課金

WebRTC によるビデオ・音声通話の基盤で、OSS としても公開されている。音声で対

話する AI エージェントの構築基盤としても使われている。

scripts/paid/90_livekit.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["livekit-api"]
# ///
"""LiveKit: ビデオ通話ルームへの参加トークン（JWT）を発行する。

トークンの発行はローカルで完結する。環境変数 LIVEKIT_API_KEY / LIVEKIT_API_SECRET が
未設定の場合は、ローカル開発サーバー（`livekit-server --dev`）の既定値 devkey / secret を使う。
"""
import os
import warnings
from datetime import timedelta

from livekit import api

key = os.environ.get("LIVEKIT_API_KEY", "devkey")
secret = os.environ.get("LIVEKIT_API_SECRET", "secret")
if secret == "secret":
    # 開発用の短い鍵では警告が出る。本番では 32 バイト以上のランダムな秘密鍵を使う
    warnings.filterwarnings("ignore", message=".*HMAC key.*")

token = (api.AccessToken(key, secret)
         .with_identity("user-123")
         .with_name("テストユーザー")
         .with_ttl(timedelta(hours=1))
         .with_grants(api.VideoGrants(room_join=True, room="guide-room")))
jwt = token.to_jwt()
print("参加トークン:", jwt[:60], "...")

# 検証（サーバー側で受け取ったトークンを確認する例）
claims = api.TokenVerifier(key, secret).verify(jwt)
print("identity:", claims.identity, " room:", claims.video.room, " join:", claims.video.room_join)
# フロントエンドでは livekit-client の room.connect(LIVEKIT_URL, jwt) で接続する
```

実行結果　`uv run scripts/paid/90_livekit.py`

```
参加トークン: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1lIjoiXHUzMGM2XHU ...
identity: user-123  room: guide-room  join: True
```

参加者がルームに入るには、サーバーが発行した JWT（参加トークン）が必要である。トークンには、参加者の ID・ルーム名・権限・有効期限が署名付きで埋め込まれる。トークンの発行はローカルの計算だけで完結するため、開発用サーバー（`livekit-server --dev`）の既定の鍵 `devkey` / `secret` で実行した。`TokenVerifier` で検証すると、埋め込んだ ID とルーム名がそのまま取り出せる。開発用の短い鍵ではライブラリが「鍵が短すぎる」という警告を出すので、本番では 32 バイト以上のランダムな秘密鍵を使う。

第 22 章　その他データ

22.1　OpenWeather

公式　https://openweathermap.org/api

認証　API キー（OPENWEATHER_API_KEY）

料金　無料枠あり（1 日 1,000 回など、プランによる）。従量・月額プラン

世界中の現在の天気・予報・大気汚染などを提供する、広く使われている天気 API である。

scripts/paid/91_openweather.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""OpenWeather: 現在の天気と 5 日間予報を取得する。

環境変数 OPENWEATHER_API_KEY を設定する（発行直後は有効になるまで少し時間がかかる）。
"""
import os
import sys

import httpx

key = os.environ.get("OPENWEATHER_API_KEY")
if not key:
    print("[SKIP] 環境変数 OPENWEATHER_API_KEY が未設定です。")
    sys.exit(0)

common = {"lat": 35.1815, "lon": 136.9066, "units": "metric", "lang": "ja", "appid": key}
with httpx.Client(base_url="https://api.openweathermap.org/data/2.5", timeout=20) as client:
    cur = client.get("/weather", params=common).json()
    print(f"名古屋の現在: {cur['weather'][0]['description']}, {cur['main']['temp']}℃")
    fc = client.get("/forecast", params=common | {"cnt": 4}).json()
    for item in fc["list"]:
        print(f"  {item['dt_txt']}  {item['weather'][0]['description']}  {item['main']['temp']}℃")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 OPENWEATHER_API_KEY が未設定です。
```

`units=metric` で摂氏、`lang=ja` で天気の説明が日本語になる。API キーは発行してから有効になるまで、最大で数時間かかることがある。キー不要の Open-Meteo（第 I 部）と使い比べてみるとよい。

22.2 Alpha Vantage

公式 https://www.alphavantage.co/

認証 API キー（デモ用の demo キーあり）

料金 無料枠は 1 日 25 リクエスト程度。月額プラン

株価・為替・暗号資産・経済指標・企業の財務データを取得できる。公式ドキュメントのデモ用キー demo では、IBM などの一部の銘柄に限ってキーなしで試せるため、実際に実行した。

scripts/paid/92_alpha_vantage.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""Alpha Vantage: 株価・為替データを取得する。

環境変数 ALPHAVANTAGE_API_KEY を設定する。未設定の場合は公式ドキュメントの
デモ用キー "demo" を使う (IBM などデモ対象の銘柄のみ取得できる)。
"""
import os

import httpx

key = os.environ.get("ALPHAVANTAGE_API_KEY", "demo")
with httpx.Client(base_url="https://www.alphavantage.co", timeout=30) as client:
    q = client.get("/query", params={"function": "GLOBAL_QUOTE", "symbol": "IBM",
                                     "apikey": key}).json()["Global Quote"]
    print(f"IBM: ${q['05. price']}  前日比 {q['10. change percent']}  ({q['07. latest trading day']})")

    daily = client.get("/query", params={"function": "TIME_SERIES_DAILY", "symbol": "IBM",
                                         "apikey": key}).json()["Time Series (Daily)"]
    for d in list(daily)[:3]:
        print(f"  {d}  終値 ${daily[d]['4. close']}  出来高 {int(daily[d]['5. volume']):,}")
```

実行結果 uv run scripts/paid/92_alpha_vantage.py

```
IBM: $226.6100  前日比 2.7663%  (2026-10-08)
  2026-10-08  終値 $226.6100  出来高 8,563,521
  2026-10-07  終値 $220.5100  出来高 3,895,497
  2026-10-06  終値 $221.2900  出来高 3,938,659
```

GLOBAL_QUOTE で最新の株価と前日比、TIME_SERIES_DAILY で日足の時系列が得られる。レスポンスのキーが "05. price" のように番号付きの独特な形式なので、扱う前に辞書の構造を確認する。無料枠の 1 日あたりの回数は少ないため、取得したデータはキャッシュして再利用する。

22.3 NewsAPI

公式 https://newsapi.org/

認証 API キー（NEWSAPI_KEY）

料金 無料の Developer プランは開発用途のみ。本番利用は有料

世界中のニュースサイトの記事を、キーワード・言語・期間で検索できる。

scripts/paid/93_newsapi.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["httpx"]
# ///
"""NewsAPI: ニュース記事を検索する。

環境変数 NEWSAPI_KEY を設定する。無料の Developer プランは開発用途のみ
(本番サイトでの利用は不可、記事は 24 時間遅れ)。
"""
import os
import sys

import httpx

key = os.environ.get("NEWSAPI_KEY")
if not key:
    print("[SKIP] 環境変数 NEWSAPI_KEY が未設定です。")
    sys.exit(0)

r = httpx.get("https://newsapi.org/v2/everything", headers={"X-Api-Key": key}, timeout=20,
              params={"q": "生成AI", "language": "jp", "sortBy": "publishedAt", "pageSize": 5})
r.raise_for_status()
for a in r.json()["articles"]:
    print(f"{a['publishedAt'][:10]}  {a['title']}  ({a['source']['name']})")
```

未実行 (認証情報が未設定)

```
[SKIP] 環境変数 NEWSAPI_KEY が未設定です。
```

無料プランは開発・テスト用途に限られ、記事も 24 時間遅れになる。公開する Web サイトで使うには有料プランが必要である。

22.4 X (Twitter) API

公式 https://developer.x.com/

認証 OAuth 1.0a（API キー・アクセストークンの 4 点）

料金 従量課金（呼び出し回数に応じる）

X の投稿の取得・投稿、ユーザー情報の取得などができる。2023 年以降、無料で使える範囲は大きく縮小された。

scripts/paid/94_x_api.py

```python
# /// script
# requires-python = ">=3.11"
# dependencies = ["tweepy"]
# ///
"""X (Twitter) API v2: 認証ユーザーの情報を取得する（読み取りのみ）。

環境変数 X_API_KEY, X_API_SECRET, X_ACCESS_TOKEN, X_ACCESS_TOKEN_SECRET を設定する。
X API は従量課金のため、呼び出し回数に注意する。
"""
import os
import sys

import tweepy

need = ["X_API_KEY", "X_API_SECRET", "X_ACCESS_TOKEN", "X_ACCESS_TOKEN_SECRET"]
if not all(os.environ.get(k) for k in need):
    print(f"[SKIP] 環境変数 {' / '.join(need)} が未設定です。")
    sys.exit(0)

client = tweepy.Client(consumer_key=os.environ["X_API_KEY"],
                       consumer_secret=os.environ["X_API_SECRET"],
                       access_token=os.environ["X_ACCESS_TOKEN"],
                       access_token_secret=os.environ["X_ACCESS_TOKEN_SECRET"])
me = client.get_me(user_fields=["created_at", "public_metrics"]).data
m = me.public_metrics
print(f"@{me.username}  フォロワー {m['followers_count']:,}  投稿 {m['tweet_count']:,}")
# 投稿する場合: client.create_tweet(text="...")  （書き込み権限が必要）
```

実行結果　`uv run scripts/paid/94_x_api.py`

```
@katzkawai  フォロワー 2,328  投稿 38,342
```

tweepy の `Client` に 4 つの認証情報を渡すと、ユーザーとして API を呼べる。`get_me()` は認証したユーザー自身の情報を返し、`user_fields` で追加の項目（作成日、フォロワー数など）を指定する。書き込み（投稿）にはアプリに書き込み権限が必要で、従量課金のため呼び出し回数にも注意する。

付録 A　実行結果の一覧

本書の全スクリプトの実行結果を次の表にまとめる（成功 41 件、未実行 52 件、エラー 1 件）。「未実行」は、API キーなどの認証情報が未設定のため `[SKIP]` を表示して終了したものである。この表は `uv run tools/make_summary.py` で `outputs/` から自動生成した。

スクリプト	API	依存パッケージ	結果
`free/01_jsonplaceholder`	JSONPlaceholder	`httpx`	成功
`free/02_dummyjson`	DummyJSON	`httpx`	成功
`free/03_httpbin`	httpbin	`httpx`	成功
`free/04_randomuser`	Random User Generator	`httpx`	成功
`free/05_picsum`	Lorem Picsum	`httpx`	成功
`free/06_osm_tiles`	OpenStreetMap タイル	`httpx`	成功
`free/07_nominatim`	Nominatim	`httpx`	成功
`free/08_gsi_tiles`	国土地理院 地理院タイル	`httpx`	成功
`free/09_gsi_address_elevation`	国土地理院 API	`httpx`	成功
`free/10_restcountries`	REST Countries	`httpx`	成功
`free/11_ip_api`	ip-api	`httpx`	成功
`free/12_open_meteo`	Open-Meteo	`httpx`	成功
`free/13_jma`	気象庁 防災情報 JSON	`httpx`	成功
`free/14_usgs_earthquake`	USGS Earthquake API	`httpx`	成功
`free/15_nasa`	NASA Open APIs	`httpx`	成功
`free/16_sunrise_sunset`	Sunrise-Sunset	`httpx`	成功
`free/17_zipcloud`	zipcloud 郵便番号検索 API	`httpx`	成功
`free/18_holidays_jp`	Holidays JP API	`httpx`	成功
`free/19_heartrails_express`	HeartRails Express	`httpx`	成功
`free/20_estat`	e-Stat API	`httpx`	未実行
`free/21_ndl_search`	国立国会図書館サーチ API (OpenSearch)	`httpx`	成功
`free/22_rakuten`	楽天ウェブサービス	`httpx`	未実行
`free/23_yahoo_shopping`	Yahoo!デベロッパーネットワーク	`httpx`	未実行
`free/24_unsplash`	Unsplash API	`httpx`	未実行
`free/25_pexels`	Pexels API	`httpx`	未実行
`free/26_pixabay`	Pixabay API	`httpx`	未実行
`free/27_dicebear`	DiceBear	`httpx`	成功
`free/28_iconify`	Iconify API	`httpx`	成功
`free/29_wikipedia`	Wikipedia / MediaWiki API	`httpx`	成功

スクリプト	API	依存パッケージ	結果
free/30_open_library	Open Library API	httpx	成功
free/31_google_books	Google Books API	httpx	エラー
free/32_hacker_news	Hacker News API (Firebase)	httpx	成功
free/33_pokeapi	PokéAPI	httpx	成功
free/34_github	GitHub REST API	httpx	成功
free/35_frankfurter	Frankfurter	httpx	成功
free/36_coingecko	CoinGecko API	httpx	成功
free/37_qrserver	goqr.me QR Code API	httpx	成功
free/38_microlink	Microlink	httpx	成功
free/39_google_fonts	Google Fonts (CSS API)	httpx	成功
free/40_worldtimeapi	World Time API / Time API	httpx	成功
paid/41_anthropic_claude	Claude API (Anthropic)	anthropic	成功
paid/42_openai	OpenAI API	openai	成功
paid/43_gemini	Google Gemini API	google-genai	成功
paid/44_replicate	Replicate	replicate	未実行
paid/45_huggingface	Hugging Face Inference Providers	huggingface_hub, numpy	成功
paid/46_stripe	Stripe	stripe	未実行
paid/47_payjp	PAY.JP	httpx	未実行
paid/48_square	Square	httpx	未実行
paid/49_paypal	PayPal	httpx	未実行
paid/50_komoju	KOMOJU	httpx	未実行
paid/51_firebase_auth	Firebase Authentication	httpx	未実行
paid/52_auth0	Auth0	httpx	未実行
paid/53_clerk	Clerk	httpx	未実行
paid/54_supabase_auth	Supabase Auth	supabase	未実行
paid/55_firestore	Cloud Firestore	google-cloud-firestore	未実行
paid/56_supabase_db	Supabase (PostgreSQL)	supabase	未実行
paid/57_neon	Neon	psycopg[binary]	未実行
paid/58_upstash	Upstash Redis	upstash-redis	未実行
paid/59_mongodb_atlas	MongoDB Atlas	pymongo	未実行
paid/60_resend	Resend	resend	未実行
paid/61_amazon_ses	Amazon SES (v2 API)	boto3	未実行
paid/62_sendgrid	SendGrid	sendgrid	未実行
paid/63_postmark	Postmark	httpx	未実行
paid/64_mailgun	Mailgun	httpx	未実行
paid/65_twilio	Twilio	twilio	未実行
paid/66_vonage	Vonage (旧 Nexmo)	httpx	未実行
paid/67_line_messaging	LINE Messaging API	httpx	未実行

スクリプト	API	依存パッケージ	結果
paid/68_slack	Slack	httpx	未実行
paid/69_google_maps	Google Maps Platform	httpx	未実行
paid/70_mapbox	Mapbox	httpx	未実行
paid/71_here	HERE	httpx	未実行
paid/72_amazon_s3	Amazon S3	boto3	未実行
paid/73_cloudflare_r2	Cloudflare R2	boto3	未実行
paid/74_cloudinary	Cloudinary	cloudinary	未実行
paid/75_imgix	imgix	imgix, httpx	成功
paid/76_uploadthing	UploadThing	httpx	未実行
paid/77_algolia	Algolia	algoliasearch>=4	未実行
paid/78_meilisearch	Meilisearch	meilisearch	未実行
paid/79_typesense	Typesense	httpx	未実行
paid/80_deepl	DeepL API	deepl	未実行
paid/81_google_translate	Google Cloud Translation API (Basic v2)	httpx	未実行
paid/82_elevenlabs	ElevenLabs	elevenlabs	未実行
paid/83_google_speech	Google Cloud Speech-to-Text (v1)	httpx	未実行
paid/84_sentry	Sentry	sentry-sdk	未実行
paid/85_posthog	PostHog	posthog	未実行
paid/86_mixpanel	Mixpanel	httpx	未実行
paid/87_datadog	Datadog	httpx	未実行
paid/88_pusher	Pusher Channels	pusher	未実行
paid/89_ably	Ably	httpx	未実行
paid/90_livekit	LiveKit	livekit-api	成功
paid/91_openweather	OpenWeather	httpx	未実行
paid/92_alpha_vantage	Alpha Vantage	httpx	成功
paid/93_newsapi	NewsAPI	httpx	未実行
paid/94_x_api	X (Twitter) API v2	tweepy	成功

付録 B　リポジトリの構成と再現手順

```
kklab-api-list/
├── README.md            API リスト (無償 / 有償)
├── scripts/
│   ├── free/             無償 API のスクリプト (01–40)
│   └── paid/             有償 API のスクリプト (41–94)
├── outputs/             実行結果 (*.txt / *.err / *.skip) と画像 (img/)
├── tools/
│   ├── run_all.py       全スクリプトを実行して outputs/ に保存
│   └── make_summary.py  付録の一覧表を生成
├── guide/               本書のソース
└── Makefile
```

実行結果は、次のコマンドで取得し直せる。

```
$ make run     # uv run tools/run_all.py && uv run tools/make_summary.py
```

実行環境

- 実行日：2026 年 10 月 9 日
- OS：Linux 6.6（x86_64）
- uv 0.12.9 / Python 3.14.4

参考資料

- PEP 723 – Inline script metadata：`https://peps.python.org/pep-0723/`
- uv ドキュメント「Running scripts」：`https://docs.astral.sh/uv/guides/scripts/`
- HTTPX：`https://www.python-httpx.org/`
- public-apis/public-apis：`https://github.com/public-apis/public-apis`
- free-for.dev：`https://free-for.dev/`

付録 C　用語集

本書に登場する主な用語を、アルファベット順、五十音順に並べる。各用語は巻末の索引にも収録しており、本文中の該当ページを引ける。

API

Application Programming Interface。プログラムから別のソフトウェアやサービスの機能を呼び出すための取り決め。本書では主に、HTTP を通じて呼び出す Web API を指す。

API キー

API の利用者を識別するための文字列。リクエストのヘッダーやクエリパラメータに付けて送る。秘密のキーはサーバー側でのみ扱い、フロントエンドや Git リポジトリに含めてはならない。

Basic 認証

ユーザー名とパスワードを「ユーザー名: パスワード」の形で Base64 符号化し、`Authorization` ヘッダーで送る認証方式。PAY.JP や Mailgun のように、API キーをユーザー名として使う API がある。

Bearer トークン

`Authorization: Bearer <トークン>` の形でヘッダーに付けて送るアクセストークン。トークンを持っている者（bearer）にアクセスを許可する。

CDN

Content Delivery Network。世界各地に置いたサーバーから、利用者に近い場所でファイルを配信する仕組み。画像や動画の表示を速くする。

CORS

Cross-Origin Resource Sharing。ブラウザが、別のオリジン（ドメイン・プロトコル・ポートの組）の API を呼ぶことを許可するかどうかを、サーバーの応答ヘッダーで決める仕組み。

E.164

国際電話番号の表記規格。`+81` のように国番号から始め、先頭の 0 を除いて書く（例：`+819012345678`）。

GeoJSON

地理空間データを表す JSON の形式。座標は `[経度, 緯度]` の順に書く。Leaflet などの地図ライブラリでそのまま描画できる。

GraphQL

必要なデータの形をクエリとして送り、そのとおりの形で結果を受け取る API の方式。単一のエンドポイントで多様な取得ができる。

HTTP メソッド

リクエストの種類を表す語。取得の GET、作成の POST、更新の PUT / PATCH、削除の DELETE などがある。

httpx

Python の HTTP クライアントライブラリ。同期と非同期（`AsyncClient`）の両方の書き方に対応する。本書の多くのスクリプトで使用した。

ISO 8601

日付と時刻の国際規格の表記。例：`2026-10-09T05:51:58+09:00`。末尾の `+09:00` は UTC との時差を表す。

JSON

JavaScript Object Notation。キーと値の組、配列、文字列、数値などでデータを表すテキスト形式。Web API の応答形式として最も広く使われる。

JSONP

CORS が普及する前に使われた、別ドメインのデータを `<script>` タグ経由で読み込む手法。現在は CORS を使うのが一般的。

JWT

JSON Web Token。ユーザー ID や有効期限などの情報を JSON で表し、署名を付けて改ざんを検出できるようにしたトークン。

OAuth 2.0

利用者のパスワードを渡さずに、アプリに限定的なアクセス権を与えるための認可の仕組み。サーバー間連携では Client Credentials フローがよく使われる。

OGP

Open Graph Protocol。Web ページのタイトル・説明・画像を `<meta>` タグで記述する仕様。SNS のリンクカードに使われる。

PEP 723

Python スクリプトの先頭に、必要な Python のバージョンと依存パッケージをコメントとして書く仕様（Inline script metadata）。uv などのツールが読み取って実行環境を用意する。

Pub/Sub

Publish / Subscribe。送り手はチャンネルにメッセージを発行（publish）し、そのチャンネルを購読（subscribe）している受け手全員に配信される通信モデル。

REST

URL でリソースを表し、HTTP メソッドでそのリソースに対する操作を表す API の設計様式。

RLS

Row Level Security（行レベルセキュリティ）。データベースの行ごとに、どのユーザーが読み書きできるかをポリシーで制御する PostgreSQL の機能。

RSS

Web サイトの更新情報を配信するための XML 形式。国立国会図書館サーチの

OpenSearch はこの形式で結果を返す。

S3 互換

Amazon S3 と同じ API で操作できること。Cloudflare R2 などは S3 用のライブラリをそのまま使える。

SDK

Software Development Kit。特定のサービスを使うためのライブラリ一式。認証、再試行、ページネーションなどを肩代わりする。

SPF / DKIM / DMARC

メールの送信元が正当であることを示す 3 つの仕組み。送信ドメインに設定すると、迷惑メールに分類されにくくなる。

srcset

`<img>` タグの属性で、大きさの異なる複数の画像を候補として示し、ブラウザに画面に合ったものを選ばせる。

SSRF

Server-Side Request Forgery。利用者が指定した URL をサーバーに取得させる機能を悪用して、内部ネットワークなどへ不正にアクセスさせる攻撃。

UNIX 時間

1970 年 1 月 1 日 0 時（UTC）からの経過秒数で時刻を表す方法。ミリ秒単位で表す API もある。

User-Agent

リクエストを送ったソフトウェアを示す HTTP ヘッダー。連絡先を含めた指定を利用条件とする API がある。

uv Astral 社の Python パッケージ・プロジェクト管理ツール。Python 本体・仮想環境・パッケージの管理を高速に行う。`uv run` で PEP 723 形式のスクリプトを実行できる。

Web メルカトル図法

Web 地図で標準的に使われる地図投影法。世界を正方形に投影し、ズームレベルごとにタイルに分割する。

Webhook

サービス側でイベント（決済完了、メッセージ受信など）が起きたときに、あらかじめ登録した URL へ HTTP リクエストで通知する仕組み。

WebRTC

ブラウザ間で音声・映像・データをリアルタイムにやり取りするための技術。ビデオ通話に使われる。

WebSocket

ブラウザとサーバーの間で接続を保ったまま双方向に通信するためのプロトコル。チャットや通知に使われる。

XYZ タイル

ズームレベル z とタイル番号 (x, y) を URL に含めて地図画像を配信する方式。OSM

や地理院タイルが採用している。

埋め込み

文章などを、意味の近さが距離に反映される数値ベクトルに変換したもの。意味検索や分類に使う。

エグレス料金

クラウドからインターネットへデータを送り出す（ダウンロードさせる）ときにかかる転送料金。

エンドポイント

API の個々の機能に対応する URL。例：`/todos/1`、`/v1/messages`。

オブジェクトストレージ

ファイルをキー（名前）で保存・取得するストレージ。容量の上限がなく、画像やバックアップの保存に使われる。

仮想環境

プロジェクトやスクリプトごとに独立した Python のパッケージ環境。uv は PEP 723 スクリプトのために自動で作成する。

環境変数

OS がプログラムに渡す名前と値の組。API キーなどの秘密情報をコードに書かずに渡すのに使う。

機能フラグ

コードを変更せずに、機能の有効・無効をユーザーごとに切り替える仕組み。段階的なリリースや A/B テストに使う。

クエリパラメータ

URL の ? 以降に `key=value` の形で付ける値。検索条件や表示件数の指定に使う。

サンドボックス

実際の取引や送信を発生させずに API を試せる、本番から切り離された環境。テストモードとも呼ばれる。

サーバーレス

サーバーの管理を利用者が行わず、リクエストや使用量に応じて自動で計算資源が割り当てられる提供形態。

署名付き URL

有効期限と署名を含み、その期間だけ特定のファイルへのアクセスを許可する URL。

ジオコーディング

住所や地名から緯度経度を求めること。逆に、緯度経度から住所を求めることを逆ジオコーディング（逆引き）という。

従量課金

リクエスト数・トークン数・転送量など、使った量に応じて料金が決まる課金方式。

ステータスコード

HTTP の応答の結果を示す 3 桁の数値。2xx は成功、4xx はクライアント側の誤り、

5xx はサーバー側の誤り。

全文検索

文書の本文全体を対象に、キーワードを含むものを関連度の高い順に探す検索。表記ゆれやタイプミスへの対応も含む。

タイムアウト

一定時間内に応答がない場合に処理を打ち切ること。外部 API を呼ぶときは必ず設定する。

トランザクションメール

会員登録の確認やパスワード再設定など、利用者の操作をきっかけに個別に送るメール。

トークン（生成 AI）

生成 AI が文章を処理する単位。日本語では 1 文字が 1 トークン前後になる。生成 AI の API はトークン数で課金される。

非同期処理

ある処理の完了を待たずに別の処理を進める書き方。多数の API 呼び出しを並行して行うと全体の時間を短縮できる。

フォールバック

主な手段が失敗したときに、代わりの手段に切り替えること。

冪等キー

同じリクエストを複数回送っても一度しか処理されないようにするために付ける一意な値。決済 API で重要。

ページネーション

大量の結果を一定件数ずつ分けて取得すること。`limit` と `offset`（`skip`）、またはカーソルで指定する。

無料枠

有償 API で、一定量まで料金がかからない利用範囲。超過分から課金される。

リダイレクト

サーバーが別の URL へ移動するよう指示する応答（3xx）。httpx は既定では追従しないため `follow_redirects=True` を指定する。

レート制限

一定時間あたりのリクエスト数の上限。超えると HTTP 429 Too Many Requests が返る。

索引

A

Ably 105
Algolia 93
Alpha Vantage 108
Amazon S3 88
Amazon SES 77
API **115**
API キー **115**
Auth0 68

B

Basic 認証 63, 80, 105, **115**
Bearer トークン **115**

C

CDN 90, **115**
Claude API (Anthropic) 55
Clerk 69
Cloud Firestore (Firebase) 71
Cloudflare R2 89
Cloudinary 90
CoinGecko API 48
CORS **115**

D

Datadog 102
DeepL API 96
DiceBear 38
DummyJSON 11

E

e-Stat API 32
E.164 ..... 81, **115**
ElevenLabs 97

F

Firebase Authentication 67
Frankfurter 47

G

GeoJSON 20, 25, **115**
GitHub REST API 45
Google Books API 43
Google Cloud Speech-to-Text 98
Google Cloud Translation 97
Google Fonts 50
Google Gemini API 57
Google Maps Platform 85
goqr.me QR Code API 49
GraphQL **115**

H

Hacker News API 44
HeartRails Express 31
HERE 87
Holidays JP API 30
HTTP メソッド 10, 83, **116**
httpbin 12
httpx 7, 15, 63, **116**
Hugging Face Inference Providers 59

I

Iconify API 39
imgix 90
ip-api 21
ISO 8601 ..... **116**

J

JSON 12, 21, 24, 30, 44, 74, 83, 95, **116**
JSONP 29, **116**
JSONPlaceholder 10
JWT 11, 106, **116**

K

KOMOJU 65

L

LINE Messaging API 82
LiveKit 105
Lorem Picsum 14

M

Mailgun 79
Mapbox 86
Meilisearch 94
Microlink 50
Mixpanel 101
MongoDB Atlas 74

N

NASA Open APIs 26
Neon 72
NewsAPI 109
Nominatim 17

O

OAuth 2.0 . . . . . 65, **116**
OGP 50, **116**
Open Library API 42
Open-Meteo 23
OpenAI API 56
OpenStreetMap タイル + Leaflet 16
OpenWeather 107

P

PAY.JP 62
PayPal 64
PEP 723 . . . . . 6, **116**
Pexels API 37
Pixabay API 37
PokéAPI 45
PostHog 101
Postmark 78
Pub/Sub 105, **116**
Pusher Channels 104

R

Random User Generator 13
Replicate 58
Resend 76
REST 7, 10, 41, 68, 92, **116**
REST Countries 20
RLS 69, 72, **116**
RSS 33, **116**

S

S3 互換 89, **117**
SDK 8, 55, 68, 92, 96, 101, **117**
SendGrid 77
Sentry 100
Slack API 83
SPF / DKIM / DMARC 76, **117**
Square 63
srcset **117**
SSRF 50, **117**
Stripe 61
Sunrise-Sunset 27
Supabase 72
Supabase Auth 69

T

Twilio 81
Typesense 95

U

UNIX 時間 **117**
Unsplash API 36
UploadThing 92
Upstash 73
User-Agent 7, 12, **117**
USGS Earthquake API 25
uv 5, **117**

V

Vonage 82

W

Web メルカトル図法 **117**
Webhook 62, 78, 83, **117**
WebRTC 105, **117**
WebSocket 104, **117**
Wikipedia / MediaWiki API 41
World Time API / Time API 52

X

X (Twitter) API 109

XYZ タイル 16, **117**

Y

Yahoo!デベロッパーネットワーク 34

Z

zipcloud 郵便番号検索 API 29

あ

埋め込み 56, **118**
エグレス料金 **118**
エンドポイント 12, 36, **118**
オブジェクトストレージ 88, **118**

か

仮想環境 5, **118**
環境変数 7, 55, 83, **118**
気象庁 防災情報 JSON 24
機能フラグ 101, **118**
クエリパラメータ 7, **118**
国土地理院 住所検索・標高 API 20
国土地理院 地理院タイル 18
国立国会図書館サーチ API 32

さ

サーバーレス 59, 71, **118**
サンドボックス 61, 77, **118**
ジオコーディング 17, 85, **118**
従量課金 5, 54, 110, **118**
署名付き URL 88, **118**
ステータスコード 7, 21, **118**
全文検索 42, 93, **119**

た

タイムアウト 7, **119**
トークン（生成 AI） **119**
トランザクションメール 76, **119**

は

非同期処理 7, 94, 99, **119**
フォールバック 52, **119**
ページネーション 8, 11, **119**
冪等キー 64, **119**

ま

無料枠 5, 54, 57, 108, **119**

ら

楽天ウェブサービス 33
リダイレクト 15, 21, 62, **119**
レート制限 7, **119**